

EDADSL Documentation

Andreas G.

15th of April 2026

Contents

1	Introduction	10
1.1	Abstract	10
1.2	Reading this Document	10
I	Core Language	11
2	Design Principles	12
2.1	Design Goals	12
2.2	Applications and Application-Areas with special Priority during Design . . .	12
2.3	Design Philosophy	13
3	Lexical Structure	14
3.1	Character Set	14
3.2	Whitespace	14
3.3	Comments	14
3.4	Identifiers	15
3.5	Keywords	15
3.6	Literals	15
3.7	Operators	15
3.8	Delimiters	16
3.9	Tokenization Rules	16
3.10	Statement Separators	16
3.11	End of File	16
4	Formal Grammar	17
4.1	Primitives	17
4.2	General Grammar	17
4.3	Grammar relating to the electrical System	20
4.3.1	Make Statement	20
4.3.2	Electrical Statements	20
4.3.3	Physical Constraints	21
4.3.4	System Statements	22
4.3.5	Additional Grammar Rules	22
4.3.6	Common Rules	22

5	Types	24
5.1	Generic Types	24
5.1.1	Boolean	24
5.1.2	Integer	24
5.1.3	Real	25
5.1.4	Complex	27
5.1.5	Set	28
5.1.6	String	29
5.1.7	List	30
5.2	Electrical Types	33
5.2.1	Part	33
5.2.2	Pin	34
5.2.3	Connection	34
5.2.4	Net	35
6	Variables	36
6.1	Variable Declaration and Assignment	36
6.1.1	Type-Inferred Assignment	36
6.1.2	Variable Redeclaration	36
6.2	Mutability	37
6.3	Scope	37
6.4	Examples	38
7	Expressions	39
7.1	Expression Grammar	39
7.2	Operator Precedence	40
7.3	Evaluation Order	41
8	Operators	42
8.1	Built-in Operators	42
8.1.1	Arithmetic	42
8.1.2	Increment / Decrement	45
8.1.3	Compound Assignment	45
8.1.4	Comparison	48
8.1.5	Boolean Logic	50
8.1.6	Collection Operators	51
8.1.7	List Operators	52
8.1.8	Tensor (Ndarray) Operators	53
8.1.9	Set Comparison	55
8.1.10	Conditional Expression	55
8.1.11	Lambda Expressions	56
8.2	Operator Precedence	57
8.3	User-defined Operators	58
8.3.1	Definition	58
8.3.2	Examples	58
8.3.3	Precedence	59

9	Constants	60
9.1	Built-in Constants	60
9.1.1	Proper Constants	60
9.1.2	Pseudo-Constants	61
9.2	User-defined Constants	62
10	Control Flow	63
10.1	Conditional Execution	63
10.2	For Loops	64
10.3	While Loops	64
10.3.1	Basic While Loop	64
10.3.2	While with Else Clause	64
10.3.3	Do-While Loops	65
10.3.4	Break and Continue	65
10.3.5	Nested While Loops	66
10.3.6	Safety: Maximum Iteration Limit	66
10.3.7	Infinite Loops	67
10.4	Goto Statements	67
10.5	End of Program Statements	68
10.6	Output and Logging	68
11	Functions	69
11.1	Built-in Functions	69
11.1.1	Mathematical Functions	69
11.1.2	String Functions	85
11.1.3	List and Set Functions	86
11.1.4	Ndarray Functions	87
11.1.5	Higher-Order Functions	94
11.1.6	Signal Processing Functions	100
11.1.7	Shared Functions	102
11.1.8	Special Functions	102
11.2	User-defined Functions	104
11.2.1	Function Definition	104
11.2.2	Function Calls	105
11.2.3	Return Values	105
11.2.4	Local Scoping	105
II	Formal Semantics	107
12	Evaluation Model	108
12.1	Evaluation Order	108
12.2	Operator Precedence	108
12.3	Type Coercion	108

13 Execution Order	110
13.1 Sequential Execution	110
13.2 Expression Evaluation	110
13.3 Declaration Order	110
14 Constraint Resolution	111
14.1 Conflict Resolution	111
15 Type System Rules	112
15.1 Type Categories	112
15.2 Type Safety	112
III Core Formal Model	113
16 Structure of the Program	114
16.1 Program Entry Point	114
16.2 Global Scope	114
16.3 Execution Model	114
17 Structure of the Electrical System	115
17.1 Electrical Graph	115
17.2 Object Lifecycle	115
18 Structure of the Solver	116
18.1 Constraint Representation	116
18.2 Resolution Algorithm	116
IV Hardware Model	117
19 Circuit Model	118
19.1 Graph Structure	118
19.2 Hierarchy	118
20 Electrical Objects	119
20.1 Nets	119
20.1.1 Creation	119
20.1.2 Properties	119
20.2 Parts	119
20.2.1 Creation	120
20.2.2 PCB Text Visibility Flags	120
20.2.3 Properties	120
20.3 Pins	120
20.3.1 Creation	120
20.3.2 Reference Syntax	120
20.4 Connections	121
20.4.1 Creation	121

20.5	Models	121
20.5.1	Model Declaration Syntax	121
20.5.2	Built-in Models	122
20.5.3	Custom Model Examples	122
20.6	Electrical Modules	123
20.6.1	Definition	123
20.6.2	Instantiation	124
20.6.3	Example	124
20.7	Relationship between the Objects	124
21	Querying	125
21.1	Output and Input Quantifiers	125
21.2	Net based Querying	125
21.3	Tagged Net based Querying	125
21.4	Affiliation-based Querying	126
21.5	General Querying	126
21.5.1	Set Builder Notation	126
21.5.2	Set Query Notation	127
21.5.3	Nesting	127
V	Physical System	129
22	Physical Constraints	130
22.1	Units and SI Prefixes	130
22.2	Priority System	130
22.3	Proximity Constraint	131
22.4	Creepage Constraint	131
22.5	Layer Constraint	131
22.6	Board Definition	131
22.7	Footprint Unification	132
22.8	Net Shrink	132
22.9	Plane Creation	133
22.10	Text Annotation	133
22.11	Via Fill	133
22.11.1	Pattern Types	134
22.11.2	Examples	134
22.12	Connection Current	135
22.13	Alignment	135
22.14	Connection Matching	135
22.14.1	Properties	135
22.14.2	Value Calculation Methods	136
22.14.3	Matching Objective	138
22.14.4	Example	139

23 Solver Model	140
23.1 Overview	140
23.2 Algorithm Selection	140
23.2.1 Available Algorithms	140
23.2.2 Default Algorithm	148
23.3 Constraint Resolution	148
23.3.1 No Conflicts	148
23.3.2 Conflicts Between Different Priorities	148
23.3.3 Conflicts Between Same Priorities	149
23.4 Layout Generation Pipeline	149
23.4.1 Phase 1: Constraint Collection	149
23.4.2 Phase 2: Board Setup	149
23.4.3 Phase 3: Component Placement	149
23.4.4 Phase 4: Trace Routing	149
23.4.5 Phase 5: Via Placement	150
23.4.6 Phase 6: Plane Generation	150
23.4.7 Phase 7: Design Rule Check	150
23.4.8 Phase 8: File Export	150
23.5 Ordering of Global Set Constant *e	150
23.5.1 Canonical String Rules	150
23.5.2 Example	151
24 Flags	152
24.1 Available Flags	152
24.2 PCB Part Text Visibility	153
VI Toolchain	154
25 Compiler and Execution Pipeline	155
25.1 Compilation Phases	155
25.2 Entry Point	155
26 Code Generation	156
26.1 PCB Generation	156
26.2 Schematic Generation	156
26.3 Export	156
27 Standard Library	157
27.1 Component Models	157
27.2 Footprint Libraries	157
27.3 Material Library	157
28 External Library	158
28.1 Importing Libraries	158
28.2 Supported Formats	158

29 Debugging and Logging	159
29.1 Debug Mode	159
29.2 Console Output	159
29.3 Log File	159
29.4 Debug Print	159
29.5 Breakpoints	160
29.6 Graph Inspection	160
29.7 Resource Monitoring	160
 VII Reference	 161
30 Exceptions	162
30.1 Type Errors	162
30.2 Value Errors	162
30.3 Reference Errors	162
30.4 Constraint Errors	162
31 Error and Warning Model	163
31.1 Errors	163
31.2 Warnings	163
32 Future Improvements	164
33 Implementations	165
33.1 Reference Implementation	165
33.2 Build and Installation	165
34 References	166
34.1 Language Specification	166
34.2 Related Standards	166
 A Standard Libraries	 167
A.1 Material Libraries	167
A.1.1 PCB Substrate Material Library	167
A.1.2 PCB Conductor Material Library	167
A.1.3 PCB Surface-finish Material Library	167
A.2 Basic Model Libraries	167
A.2.1 Passive Component Model Library	167
A.2.2 Active Component Model Library	168
A.2.3 Integrated Circuit Component Model Library	168
A.2.4 Board Hardware Component Model Library	168
A.3 Additional Language Libraries	168
A.4 Footprint Libraries	168

B	Examples	169
B.1	LED Circuit	169
B.1.1	Intent	169
B.1.2	EDADSL Code	169
B.1.3	Explanation	169
B.2	Arithmetic and Type Coercion	170
B.2.1	Intent	170
B.2.2	EDADSL Code	170
B.2.3	Explanation	170
B.3	Conditionals and Classification	170
B.3.1	Intent	170
B.3.2	EDADSL Code	170
B.3.3	Explanation	171
B.4	Lambdas and Higher-Order Functions	171
B.4.1	Intent	171
B.4.2	EDADSL Code	171
B.4.3	Explanation	172
B.5	Complex Arithmetic and FFT	172
B.5.1	Intent	172
B.5.2	EDADSL Code	172
B.5.3	Explanation	173
B.6	Ndarray Matrix Operations	173
B.6.1	Intent	173
B.6.2	EDADSL Code	173
B.6.3	Explanation	174
B.7	Closures and Lambdas	174
B.7.1	Intent	174
B.7.2	EDADSL Code	174
B.7.3	Explanation	175
B.8	Design-Rule-Driven Component Placement	175
B.8.1	Intent	175
B.8.2	EDADSL Code	175
B.8.3	Explanation	176
B.9	Signal Processing Pipeline	176
B.9.1	Intent	176
B.9.2	EDADSL Code	176
B.9.3	Explanation	177
C	Diagrams	178
C.1	Execution-phase Diagram	178
C.2	Type Hierarchy Diagram	179
C.3	Operator Precedence Diagram	179
C.4	Example Circuit: LED Current-Limiting	180
C.5	FFT Pipeline Diagram	181
C.6	Ndarray Operations	182
C.7	Custom Model Definition and Usage	183

C.7.1	Intent	183
C.7.2	EDADSL Code	183
C.7.3	Explanation	184

Chapter 1

Introduction

1.1 Abstract

EDADSL (Electronic Design Automation Domain Specific Language) is a Domain-specific Language (DSL) specialized for programmatic Electronic Design Automation (EDA) and Hardware-Synthesis. It is based on an Electrical System and a constraint-driven optimisation engine. This Document is a full outline and as of now the authoritative Specification of the EDADSL Language as well as the systems underlying it.

1.2 Reading this Document

This document is organized into the following parts:

- **Part I: Core Language** – Defines the programming language constructs including lexical structure, types, variables, expressions, constants, control flow, and functions.
- **Part II: Formal Semantics** – Describes the evaluation model, execution order, constraint resolution, and type system rules.
- **Part III: Core Formal Model** – Defines the structure of programs, the electrical system, and the solver.
- **Part IV: Hardware Model** – Describes the circuit model, electrical objects, and querying system.
- **Part V: Physical System** – Defines physical constraints, the solver model, and system flags.
- **Part VI: Toolchain** – Describes the compiler pipeline, code generation, standard library, and debugging.
- **Part VII: Reference** – Contains exceptions, error model, future improvements, and references.
- **Appendices** – Contains standard libraries, examples, and diagrams.

Readers new to EDADSL should start with Parts I and IV, which define the language and electrical system. Those interested in physical layout should read Part V. Implementers should consult Parts II and III for formal specifications.

Part I

Core Language

Chapter 2

Design Principles

2.1 Design Goals

EDADSL was designed with the following primary goals:

1. **Programmatic PCB Design** – Enable engineers to describe circuits and layouts programmatically, enabling version control, automation, and reproducibility.
2. **Constraint-Driven Layout** – Allow physical constraints (clearance, creepage, alignment) to be specified declaratively, with the solver handling the complex optimization.
3. **Electrical Awareness** – Maintain a rich electrical model (parts, pins, connections, nets) that understands circuit topology, not just geometry.
4. **KiCad Integration** – Generate industry-standard KiCad files (schematics and PCBs) for further editing and manufacturing.
5. **Readability** – Use clear, prefixed syntax (elec.*, phys.*, syst.*) to make code self-documenting and easy to understand.
6. **Simplicity** – Provide a minimal set of powerful constructs rather than a large number of specialized features.
7. **Accessibility** – Serve both professional engineers and hobbyists with appropriate defaults while supporting advanced features for complex designs.

2.2 Applications and Application-Areas with special Priority during Design

EDADSL is particularly well-suited for:

- **Power Electronics** – Where creepage distances, high-current traces, and thermal management are critical. The constraint system supports isolation requirements between high-voltage and low-voltage sections.
- **High-Speed Digital Design** – Where trace length matching, impedance control, and signal integrity are important. The `phys.connmatch` constraint supports length and impedance matching.

- **Mixed-Signal Designs** – Where analog and digital sections must be carefully isolated. The constraint system supports separation of noisy and sensitive circuits.
- **Prototyping** – Where rapid iteration and parametric design enable quick exploration of design alternatives.
- **Automated Manufacturing** – Where programmatic generation enables batch production and design automation pipelines.

2.3 Design Philosophy

EDADSL follows a declarative-imperative hybrid philosophy:

- **Imperative Circuit Definition** – Components and connections are defined sequentially, allowing natural expression of circuit topology.
- **Declarative Physical Constraints** – Physical requirements are declared as constraints, letting the solver determine optimal placement and routing.
- **Separation of Concerns** – The electrical description (what the circuit does) is separated from the physical description (how it is laid out).
- **Constraint Hierarchy** – A priority system allows designers to express preferences while ensuring critical requirements are met.
- **Block Scoping** – Variables declared inside code blocks (`if`, `for`, `while`, `do-while`) are local to that block. Functions create their own local scope.

Chapter 3

Lexical Structure

3.1 Character Set

EDADSL uses ASCII for identifiers, keywords, operators, and grammar symbols. Unicode characters are permitted within string literals and comments.

3.2 Whitespace

Whitespace is defined using the following Regular Expression:

RegEx:
Whitespace = [\t\n\r]+

3.3 Comments

Comments are defined using the following Regular Expressions:

RegEx:
Line_Comments = "#c#" [^\n]*
Multiline_Comments = "#c#", "{", [any characters], "}#c#"

Multiline comments start with `#c#{` and end with `}c#`. Nested multiline comments are not supported.

Examples:

```
#c# This is a line comment
$x [int]= 5 #c# Inline comment

#c#{
This is a
multiline comment
}#c#

$x [int]= 5 #c#{ inline }#c# + 3
```

3.4 Identifiers

Identifiers are defined using the following Regular Expressions:

```
Regex:
id      = [_a-zA-Z][_a-zA-Z0-9]*
```

3.5 Keywords

The language uses the following Keywords defined using the following Regular Expressions:

```
Generic_Keywords:  start halt if else otherwise for while do break continue
                   gotomarker gotojump
                   writetolog make.pcb make.schematic export
                   echo return let letop skip function L::
```

```
Physical_Keywords: phys\[a-z]+
                   Known Physical Keywords:
                   phys.prox phys.creep phys.layer
                   phys.shrinknet phys.mkplane
                   phys.viafill phys.connectioncurrent
                   phys.align phys.connmatch phys.board
                   phys.mktext
```

```
Electrical_Keywords: elec\[a-z]+
                   Known Electrical Keywords:
                   elec.part elec.connect elec.net
                   elec.tag elec.untag
```

```
System_Keywords:  syst\[a-z]+
                   Known System Keywords:
                   syst.flag\[a-z]+
```

Keywords are reserved identifiers and may not be used as variable names, function names, or constant names. Keywords are case-sensitive: `if` is a keyword, but `If` or `IF` is an identifier.

3.6 Literals

For the Lexical Structure of Literals see: Section 5.1 (Types).

3.7 Operators

For the Lexical Structure of built in Operators see: Section 8 (Operators).
User-defined Operators are defined using the following Regular Expressions:

```
User_Operators    = o\.u\[a-zA-Z0-9]+
```


The `o.u.` prefix is distinct from dot-access chains (`obj.property`): `o.u.` is always followed by an operator name and used in infix position, while dot-access is always `identifier.identifier` and denotes property access.

3.8 Delimiters

EDADSL uses the following Delimiters:

`() [] {} . , : ; @ ~ | %N`

The `%N` token is a statement separator, not a comment. It allows multiple statements on a single line. The `::` token denotes a code block opener. The semicolon `;` appears in modifier syntax (`;M=, ;P=, ;T=`) and property separators.

3.9 Tokenization Rules

The Tokenization follows the following Rules:

The Lexer uses the Longest Match Rule: in Cases where multiple valid tokenizations are possible the longest token is chosen.

If Lexeme matches both a Keyword and an Identifier it is classified as a Keyword.

Whitespace and Comments are discarded.

3.10 Statement Separators

Statements within a code block are separated by implicit newlines. The `%N` token acts as an explicit statement separator, allowing multiple statements on a single line:

```
#c# These are equivalent:
::{
    f.print("a")
    f.print("b")
}

::{
    f.print("a") %N f.print("b")
}
```

The `%N` token is primarily useful inside inline code blocks where writing multiple lines is impractical.

3.11 End of File

Source files must end with a newline character. A line comment at the end of the file without a trailing newline is permitted but discouraged.

Chapter 4

Formal Grammar

The Formal Grammar of EDADSL is defined using Extended Backus–Naur form, while Primitives are given using Regular Expressions.

4.1 Primitives

The Primitives are defined using Regular Expressions. For more Information on Lexical structure see Part I Chapter 2.

```
idchar  = [_a-zA-Z0-9]
id      = [_a-zA-Z] [_a-zA-Z0-9]*
letter  = [a-zA-Z]
```

4.2 General Grammar

```
program                                = "start", statement,
                                       { ("%N" | newline), statement }

statement                             = assignment
                                       | flow_control
                                       | function
                                       | electrical_system_statement
                                       | compound_assignment
                                       | postfix_expression
                                       | echo_statement
                                       | writetolog
                                       | letop

assignment                            = variable_assignment
                                       | constant_assignment

function                              = function_call_built_in
                                       | function_call_ud
                                       | function_definition
```

```

flow_control          = conditional
                      | iterator
                      | while_loop
                      | do_while
                      | break_stmt
                      | continue_stmt
                      | jump
                      | halt
                      | noop

conditional            = "if", boolean_expression, ":", code_block,
                      { "otherwise", boolean_expression, ":", code_block },
                      ["else", ":", code_block]

iterator              = "for", variable, "E",
                      (list | set), ":", code_block
## Note: "E" in the iterator is disambiguated from the set comparison "E"
## by parser context | after "for $var" it is the iterator keyword.

while_loop            = "while", boolean_expression, ":", code_block,
                      ["else", ":", code_block]

do_while              = "do", ":", code_block,
                      "while", boolean_data_type

break_stmt            = "break"

continue_stmt         = "continue"

jump                  = ( "gotomarker" | "gotojump" ), id

echo_statement        = "echo", variable

writetolog            = "writetolog", string_expression

letop                 = "letop", "o.u.", id, assigner, data

function_definition = "function", "f.u.", id,
                      "(", [ { variable, "," }, variable ],
                      ")", ":", "{", f_statement,
                      {f_statement}, "}"

f_statement = statement | "return", [data]

function_call_ud      = "f.u.", id, "(",

```

```

                                [{ data, "," }, data] ,")"

function_call_built_in      = "f.", id, "(",
                                [{ data, "," }, data] ,")"

variable_assignment        = variable,
                                (typed_assign | inferred_assign),
                                [";", "M", "=", boolean_data_type]

typed_assign               = assigner
inferred_assign            = "t", "=", data

assigner                   = "[bool]=",    boolean_type
                                | "[int]=",    int_type
                                | "[real]=",   real_type
                                | "[list]=",   list_type
                                | "[string]=", string_type
                                | "[set]=",    set_type
                                | electrical_system_assigner

constant_assignment = "let", "c.u.", id, assigner,
                                [";", "P", "=", boolean_data_type]

halt                       = "halt"
noop                       = "skip"

list                       = "[", [{ data, "," }, data], "]"

variable                   = "$", id

code_block                 = "{", { ("%N" | newline), statement }, "}"

boolean_data_type          = expression (any expression evaluating to bool)

data                       = variable
                                | literal
                                | constant
                                | function_call_built_in
                                | function_call_ud
                                | expression (see Chapter 4 for expression grammar)

```

4.3 Grammar relating to the electrical System

```
electrical_system_statement  = make_statement
                              | elec_statement
                              | phys_statement
                              | syst_statement

electrical_system_assigner    = "[part]=", part_type
                              | "[connection]=", connection_type
                              | "[pin]=", pin_type
                              | "[net]=", net_type
```

4.3.1 Make Statement

```
make_statement = "make.", ("pcb" | "schematic")
```

4.3.2 Electrical Statements

```
elec_statement = "elec.",
  ( "part", part_reference, ":", model_name,
    "(", [model_params], ") ",
    [";", "des", "=", string_type],
    [";", "fp", "=", string_type]
  | "connect", (net_type | pin_type),
    pin_type, { pin_type }
  | "net", string_type, { string_type }
  | "tag", net_type, string_type,
    { string_type }, ["-remove"]
  | "untag", net_type, string_type,
    { string_type }
  | "makemodel", model_name, ":",
    "{", pin_entries, [metadata], "}"
  | "makemodule", id, "(", [variable, {"", variable}], ")",
    "{", module_body, "}"
  )

part_reference = string_type
               | "[", { string_type, ",", string_type, "]"

model_name     = id, ["/", id]
model_params   = [{ id, "=", data, ",", id, "=", data}]

pin_entries    = { pin_entry, ";" }
pin_entry      = id, "->", integer, ";", "T", "=", pin_type
pin_type       = "input" | "power" | "output"
metadata       = des_entry | dts_entry | fp_entry
des_entry      = "des", "=", string_type
```

```

dts_entry      = "dts", "=", string_type
fp_entry       = "fp", "=", string_type

module_body    = { statement }

```

4.3.3 Physical Constraints

```

phys_statement = phys_prox | phys_creep | phys_layer
                | phys_board | phys_shrinknet
                | phys_mkplane | phys_viafill
                | phys_connectioncurrent | phys_align
                | phys_connmatch | phys_mktext
                | phys_unite

phys_prox      = "phys.prox", set,
                ( set, "min", "=", real, "max", "=", real
                  | "minim" ), priority

phys_creep     = "phys.creep", set, set,
                "d", "=", real, priority

phys_layer     = "phys.layer", set, layer_ref, priority

phys_board     = "phys.board",
                "N=", int,
                "T=", list,
                "W=", list,
                ["M=", list]

phys_shrinknet = "phys.shrinknet", net, priority

phys_mkplane   = "phys.mkplane", layer_ref, net

phys_viafill   = "phys.viafill", pad,
                "v", "=", real, "d", "=", real,
                ["p", "=", pattern],
                ["l", "=", int]

phys_connectioncurrent = "phys.connectioncurrent",
                (connection | set), real, priority

phys_align     = "phys.align", list,
                "axis", "=", ("x" | "y"),
                "d", "=", real, priority

phys_connmatch = "phys.connmatch",
                match_property, set, priority

```

```

phys_mktext = "phys.mktext",
              "txt", string,
              ["font", string],
              ["size", real],
              ["material", string],
              ["layer", layer_ref]

phys_unite   = "phys.unite", "{", part, {"", part}, "}"
part         = id

```

4.3.4 System Statements

```

syst_statement = syst_flag | syst_cm | syst_alg

syst_flag = "syst.flag.", flag_name, (real | string_type)

flag_name = id

syst_cm    = "syst.cm.calc",
             ("approx" | "calculus" | "hybrid")

syst_alg   = "syst.alg", id

```

4.3.5 Additional Grammar Rules

```

set_builder    = "{", variable, "E", ( set | list ),
                ":", expression, [ "~", expression ], "}"

set_query      = "{", set_expr, "@~", "[",
                expression, "]",
                [ "~[", expression, "]" ],
                "~>", [ variable ], "}"

lambda         = "L::", "(", [variable, {"", variable}], ")",
                "->", "{", { statement }, expression, "}"

conditional_expr = expression, "?->",
                    "(", expression, ")",
                    "!", "(", expression, ")"

pipe_syntax     = elec_connect, { "->", elec_connect }

```

4.3.6 Common Rules

```

layer_ref      = int | "T" | "B"
priority       = ["-d" | "-nth" | "-imp" | "-vimp"]

```

```
        | "-oeo" | "-eo" | "-ne" | "-nonneg"]
pattern    = "grid" | "hex" | "stagger"
            | "radial" | "random"
match_property = "l" | "res" | "L" | "cap"
            | "imp" | "t"
net         = id
pad         = id | int
```


Chapter 5

Types

This Section defines Type Literals using EBNF and RegEx.

5.1 Generic Types

5.1.1 Boolean

Boolean Data Type Definition

The Boolean data type represents logical truth values. It has exactly two possible values: **True** and **False**. Booleans are used in conditional expressions, logical operations, and control flow decisions.

Boolean Data Type Literals

```
boolean_literal = "True" | "False"
```

Boolean literals are case-sensitive: only **True** and **False** (capitalized) are valid. Lowercase **true/false** are not recognized as boolean literals.

Boolean Data Type Casting and Coercion

Boolean values can be coerced from other types:

- Int and real values coerce to **False** if zero, **True** otherwise
- Strings coerce to **False** if empty, **True** otherwise
- Lists and Sets coerce to **False** if empty, **True** otherwise
- **None** coerces to **False**

Explicit conversion is not supported; boolean evaluation is implicit in conditional contexts.

5.1.2 Integer

Integer Data Type Definition

The Integer data type represents whole numbers (elements of $\mathbb{Z}$). Integers support standard arithmetic operations and can express values from very large negative to very large positive using SI decimal prefixes (kilo and above).

Integer Data Type Literals

Integer literals can be expressed in multiple formats:

EBNF:

```
int_literal = decimal_int | binary_literal
             | octal_literal | hex_literal
```

```
decimal_int = ["+" | "-"], digit, {digit}
```

```
binary_literal = "0b", bindigit, {bindigit}
```

```
octal_literal = "0o", octdigit, {octdigit}
```

```
hex_literal = "0x", hexdigit, {hexdigit}
```

RegEx:

```
digit = [0-9]
```

```
bindigit = [01]
```

```
octdigit = [0-7]
```

```
hexdigit = [0-9a-fA-F]
```

SI prefixes μ = k and scientific notation produce integers when the result is numerically equal to an integer:

```
6.2k    ## = 6200 (int)
```

```
1M      ## = 1000000 (int)
```

```
-5e9    ## = -5000000000 (int, exact)
```

```
5e-1    ## = 0.5 (real, not exact)
```

```
4Q      ## = 4e30 (int)
```

Binary, octal, and hex literals always produce integers:

```
0x10    ## = 16
```

```
0b1010  ## = 10
```

```
0o17    ## = 15
```

Integer Data Type Casting and Coercion

Integers can be coerced from other types:

- Booleans coerce to 0 (False) or 1 (True)
- Numeric strings with no decimal point are parsed as integers (e.g., "42" becomes 42)
- real values are truncated (e.g., `f.int(3.7)` becomes 3)

5.1.3 Real

Real Data Type Definition

The Real data type represents floating-point numbers (elements of $\mathbb{R}$). Reals support standard arithmetic operations and can express values from very small (quecto-scale, 10^{-30}) to very large (quetta-scale, 10^{30}) using SI decimal prefixes.

Real Data Type Literals

Real literals can be expressed in multiple formats:

EBNF:

```
real_literal = decimal_real | scientific_real
```

```
decimal_real  = ["+" | "-"], digit, {digit}, ".", {digit}  
               | ["+" | "-"], ".", digit, {digit}
```

```
scientific_real = ["+" | "-"], digit, {digit},  
                  [".", {digit}], ("e" | "E"),  
                  ["+" | "-"], digit, {digit}
```

A literal with a decimal point or scientific notation produces a real, even if the fractional part is zero:

```
3.14      ## = 3.14 (real)  
42.0      ## = 42.0 (real)  
42e0      ## = 42.0 (real)  
.4        ## = 0.4 (real)  
2e-6      ## = 0.000002 (real)
```

SI prefixes below k produce reals:

```
100n      ## = 0.0000001 (real)  
1m        ## = 0.001 (real)  
5c        ## = 0.05 (real)
```

The full SI prefix table:

SI_Prefixes:

q	1e-30	quecto	Q	1e+30	quetta
r	1e-27	ronto	R	1e+27	ronna
y	1e-24	yocto	Y	1e+24	yotta
z	1e-21	zepto	Z	1e+21	zetta
a	1e-18	atto	E	1e+18	exa
f	1e-15	femto	P	1e+15	peta
p	1e-12	pico	T	1e+12	tera
n	1e-9	nano	G	1e+9	giga
u	1e-6	micro	M	1e+6	mega
m	1e-3	milli	k	1e+3	kilo
c	1e-2	centi			
d	1e-1	deci			

Prefixes μ –k produce integers. Prefixes j–k produce reals.

Real Data Type Casting and Coercion

Reals can be coerced from other types:

- Integers coerce to real (e.g., `real(5)` becomes 5.0)

- Booleans coerce to 0.0 (False) or 1.0 (True)
- Numeric strings with a decimal point are parsed as reals (e.g., "3.14" becomes 3.14)
- Non-numeric strings raise a type error

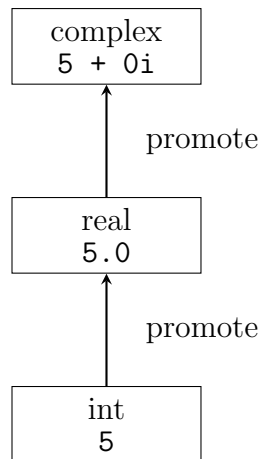
Lists of ints or reals can be passed to mathematical functions, which operate element-wise.

5.1.4 Complex

Complex Data Type Definition

The Complex data type represents complex numbers (elements of $\mathbb{C}$). A complex number has a real part and an imaginary part, and can be expressed in rectangular or polar form.

Type Hierarchy



Auto-promotion is implicit and lossless: an `int` can be used wherever a `real` is expected, and both can be used wherever a `complex` is expected.

Complex Data Type Literals

EBNF:

```
complex_literal = rectangular_literal | polar_literal
```

```
rectangular_literal = ["+" | "-"], number, ["+", number, "i"]
                    | ["+" | "-"], number, "-", number, "i"
                    | ["+" | "-"], number, "i"
```

```
polar_literal = number, "<|", number
```

The `i` suffix on a number creates a complex literal. The `<|` operator creates a polar literal (magnitude |— angle in radians).

Rectangular Form

```
3 + 4i      #c# real=3, imag=4
-2 - 7i     #c# real=-2, imag=-7
5i          #c# real=0, imag=5
3           #c# real=3, imag=0 (int promoted to complex)
3.14 + 2.71i #c# real=3.14, imag=2.71
```

Note: `3 + 4i` is a single complex literal in rectangular form. The `+` is part of the literal, not the addition operator.

Polar Form

```
2 <| 1.5708  #c# magnitude=2, angle=pi/2 (radians)
1 <| 0       #c# magnitude=1, angle=0 (real number 1)
1 <| 3.14159 #c# magnitude=1, angle=pi (real number -1)
```

Complex Data Type Casting and Coercion

- `int` promotes to `complex`: `5` becomes `5 + 0i`
- `real` promotes to `complex`: `3.14` becomes `3.14 + 0i`
- Mixed operations (`int + complex`, `real + complex`) produce `complex`
- `complex` cannot be coerced to `real` or `int` (use `real()` or `int()` to extract parts)
- Booleans coerce to `complex`: `True` becomes `1 + 0i`
- Lists of `complex` can be passed to mathematical functions, which operate element-wise

Complex Components

The real and imaginary parts of a complex number are accessed via functions:

```
real($z)  #c# Returns the real part (real)
imag($z)  #c# Returns the imaginary part (real)
```

See Section 11.1.1 for full documentation.

5.1.5 Set

Set Data Type Definition

A Set is an unordered collection of unique elements. Sets can contain any data type:

- Integers and reals (numbers)
- Strings
- Booleans
- Lists
- Parts

- Connections
- Pins
- Nets

Sets containing mixed types are allowed. Duplicate elements are automatically removed. Sets support standard set algebra operations (union, intersection, difference, subset, element-of).

Set Data Type Literals

EBNF:

```
set_literal = "{" , [ { data , "," } , data ] , "}"
            | "{" , range_spec , "}"
```

Examples:

```
$numbers [set]= { 1, 2, 3, 4, 5 }
$mixed [set]= { 1, "hello", True, Q1 }
$parts [set]= { Q1, Q2, R3 }
```

Sets also support range generation using the same syntax as lists:

```
{start:increment:end}    ## explicit increment
{start:end}              ## increment defaults to 1
{start|end|count}        ## evenly spaced, count elements
```

Examples:

```
$evens [set]= {0:2:10}    ## {0, 2, 4, 6, 8, 10}
$range [set]= {0:5}       ## {0, 1, 2, 3, 4, 5}
$lin [set]= {0|1|5}       ## {0, 0.25, 0.5, 0.75, 1}
```

Note: since sets are unordered and unique, duplicates within a range are silently ignored. For example, `{0:0.3:1}` produces `{0, 0.3, 0.6, 0.9}` with no duplicate 0 entries.

Set Data Type Casting and Coercion

Sets can be created from Lists using the `f.set()` function. When converting a List to a Set, duplicate elements are removed and order is lost.

Example:

```
$my_list [list]= [ 1, 2, 2, 3, 3, 3 ]
$my_set [set]= f.set( $my_list )    ## { 1, 2, 3 }
```

Sets can be converted to Lists using the `f.list()` function. The order of elements in the resulting List is determined by the order they appear in the global set `*e`.

5.1.6 String

String Data Type Definition

The String data type represents textual data. Strings can be delimited by single or double quotes, and support escape sequences. Long strings (multi-line) are delimited by triple quotes.

String Data Type Literals

EBNF:

```
string_literal = long_string | short_string
short_string   = '"', {short_string_item_I}, '"' |
                "' ", {short_string_item_II}, "'"
long_string    = '""', {long_string_item_I}, '""' |
                '""', {long_string_item_II}, '""'
```

RegEx:

```
short_string_item_I = [^"\\n)]|(\\. )
short_string_item_II = [^'\\n)]|(\\. )
long_string_item_I   = [^"\\]|(\\. )
long_string_item_II  = [^'\\]|(\\. )
```

String Data Type Casting and Coercion

Strings can be created from other types using the `f.str()` function, which concatenates all arguments into a single string.

Example:

```
$msg [string]= f.str( "The value is ", 42,
    " and color is ", "red" )
## $msg = "The value is 42 and color is red"
```

Additional string functions include `f.upperc()` (uppercase), `f.lowerc()` (lowercase), and `f.format()` (formatted output).

5.1.7 List

List Data Type Definition

The List data type represents an ordered, indexed collection of objects. Lists support 0-based indexing, slicing, and can contain elements of different types. Lists are mutable and can be dynamically resized. Assignment creates a copy (not a reference).

List Data Type Literals

EBNF:

```
list_literal = "[", [{ data, "," }, data], "]"
              | "[", range_spec, "]"

set_literal = "{", [ { data, "," }, data ], "}"
             | "{", range_spec, "}"

range_spec = expression, ":", expression, [":", expression]
            | expression, ":", expression
            | expression, "|", expression, "|", expression
```

```
slice      = [expression], ":", [expression], [":", expression]

reshape    = ".", "reshape", "(", int, {"", int}, ")"
```

List Data Type Casting and Coercion

Lists can be created from Sets using the `f.list()` function. The order is determined by the element order in `*e`.

Lists support range generation using the syntax:

```
[start:increment:end]    #c# explicit increment
[start:end]              #c# increment defaults to 1
[start|end|count]        #c# evenly spaced, count elements
```

Examples:

```
$r1 [list]= [0:2:10]      #c# [0, 2, 4, 6, 8, 10]
$r2 [list]= [0:5]        #c# [0, 1, 2, 3, 4, 5]
$r3 [list]= [0|1|5]      #c# [0, 0.25, 0.5, 0.75, 1]
```

Ndarray (N-dimensional Array)

An ndarray is a list that is **rectangular** (all sublists have the same length) and contains only **numeric** values (`int` or `real`). Ndarray status is determined automatically – no type annotation is needed. A list that is rectangular and all-numeric gains ndarray operations; all other lists remain standard lists.

Detection Rules A list is an ndarray if and only if it satisfies all of the following:

1. All elements are `int` or `real` (no strings, bools, or other types).
2. All sublists (if any) have identical length (rectangular).
3. Nesting is consistent: if element i is a list, all elements $j \neq i$ must also be lists of the same length.

Detection Algorithm

```
is_ndarray(list):
    if list is empty: return True  (1D ndarray)
    if all elements are int or real: return True  (1D ndarray)
    if any element is not a list: return False
    lengths = [len(x) for x in list if x is list]
    if not all equal(lengths): return False
    return all(is_ndarray(x) for x in list if x is list)
```


Examples

- `[1, 2, 3]` — ndarray (1D, shape (3))
- `[[1,2], [3,4], [5,6]]` — ndarray (2D, shape (3,2))
- `[[1,2], [3,4,5]]` — NOT ndarray (different lengths)
- `[1, "hello", True]` — NOT ndarray (non-numeric)
- `[]` — ndarray (1D, shape (0))

Shape The shape of an ndarray is described by its dimensions:

- 1D: `[1, 2, 3]` has shape (3)
- 2D: `[[1,2], [3,4], [5,6]]` has shape (3, 2) (3 rows, 2 columns)

Broadcasting When an ndarray is combined with a scalar, the scalar is broadcast to match the ndarray's shape:

```
[1, 2, 3] + 5          #c# = [6, 7, 8]
[1, 2, 3] * 2          #c# = [2, 4, 6]
[[1,2],[3,4]] + 10     #c# = [[11,12],[13,14]]
```

Element-wise Operations When two ndarrays of the same shape are combined with a standard operator (+, -, *, /, %, ^), the operation is applied element-wise:

```
[1, 2, 3] + [4, 5, 6]   #c# = [5, 7, 9]
[1, 2, 3] * [4, 5, 6]   #c# = [4, 10, 18]
[10, 20, 30] - [1, 2, 3] #c# = [9, 18, 27]
[10, 20, 30] / [2, 4, 5] #c# = [5.0, 5.0, 6.0]
```

Slicing Ndarrays (and all lists) support Python-style slicing with the syntax `list[start:stop:step]:`

- `list[start:stop]` – elements from `start` up to (not including) `stop`
- `list[start:]` – from `start` to end
- `list[:stop]` – from beginning to `stop`
- `list[:]` – full copy
- `list[start:stop:step]` – with step size
- Negative indices count from the end: `list[-1]` is the last element
- Negative zero is equivalent to zero: `list[-0]` is the same as `list[0]`

```
$a [list]= [0, 1, 2, 3, 4, 5]
$a[1:4]      #c# = [1, 2, 3]
$a[2:]       #c# = [2, 3, 4, 5]
$a[:3]       #c# = [0, 1, 2]
$a[::2]      #c# = [0, 2, 4]   (every 2nd element)
$a[-2:]      #c# = [4, 5]     (last 2 elements)
$a[::-1]     #c# = [5, 4, 3, 2, 1, 0] (reversed)
```

Reshape Ndarrays support reshaping via the `.reshape()` method. The total number of elements must remain constant.

```
$b [list]= [[1,2,3],[4,5,6]]
$b.reshape(3, 2)   #c# = [[1,2],[3,4],[5,6]]
$b.reshape(6)      #c# = [1, 2, 3, 4, 5, 6] (flatten)
$b.reshape(1, 6)   #c# = [[1,2,3,4,5,6]] (row vector)
```

5.2 Electrical Types

5.2.1 Part

Part Data Type Definition

A Part represents a physical electronic component in the circuit. Each Part has a reference designator, a model type, a description, and a footprint. Parts contain Pins which can be connected to Nets or other Pins.

Part Data Type Literals

Parts are created using the `elec.part` statement:

EBNF:

```
part_statement = "elec.part", part_reference,
  ":", model_name, "(", [model_params], ")",
  [";", "des", "=", string_type],
  [";", "fp", "=", string_type]
```

```
part_reference = string_type |
  "[", { string_type, "," }, string_type, "]"
model_name      = id
model_params    = [ { id, "=", data, "," },
  id, "=", data ]
```

Examples:

```
elec.part "R1": r( R = 100, P = 0.25,
  type = "metal film" )
; des = "100 ohm resistor"
; fp = "Resistor_SMD:R_0805_2012Metric"
```

```
elec.part ["R2", "R3"]: r( R = 220, P = 0.25 )
; des = "220 ohm resistors"; fp = "Resistor_SMD:R_0805_2012Metric"
```

Part Data Type Casting and Coercion

Parts cannot be cast from other types. Parts are created via `elec.part` statements and accessed by their reference designator. Parts support the following properties:

- Model parameters (accessed via model-specific syntax)

- Pin references (using `part_ref[pin_ref]` syntax)
- Description (set via `des` parameter)
- Footprint (set via `fp` parameter)

5.2.2 Pin

Pin Data Type Definition

A Pin represents a connection point on a Part. Pins are referenced using the syntax `part_ref[pin_ref]`, where `pin_ref` is typically a numeric index or a named terminal (e.g., `R1[1]`, `Q1[d]`, `U1[v+]`).

Pin Data Type Literals

Pins are created implicitly when Parts are instantiated. Pin references are formed using the bracket notation:

```
pin_literal = part_ref, "[", pin_ref, "]"
```

Examples:

```
R1[1]      #c# Pin 1 of resistor R1
Q1[d]      #c# Drain pin of MOSFET Q1
U1[v+]     #c# Non-inverting input of op-amp U1
J1[3]      #c# Pin 3 of connector J1
```

The available pin references depend on the Part's footprint and model definition.

Pin Data Type Casting and Coercion

Pins cannot be cast from other types. Pins are accessed via Part references and support the following operations:

- Connection to other Pins or Nets via `elec.connect`
- Membership testing in Sets
- Iteration over Pins of a Part

5.2.3 Connection

Connection Data Type Definition

A Connection represents an electrical connection between two Pins. Connections are binary (exactly two Pins) and are created using the pipe syntax or the `elec.connect` statement.

Connection Data Type Literals

Connections can be created using the pipe syntax:

```
connection_literal = "(", pin_literal, "|", pin_literal, ")"
```

Example:

```
( R2[1] | J4[7] )  #c# R2 pin 1 to J4 pin 7
```

Connections are also created implicitly by `elec.connect` statements.

Connection Data Type Casting and Coercion

Connections cannot be cast from other types. Connections support:

- Membership testing in Sets
- Property queries (length, resistance, inductance, capacitance, impedance)

5.2.4 Net

Net Data Type Definition

A Net represents a named electrical node that connects multiple Pins and Connections. Nets are the fundamental grouping mechanism in the electrical system, allowing multiple components to share a common electrical connection.

Net Data Type Literals

Nets are created using the `elec.net` statement:

EBNF:

```
net_statement = "elec.net", string_type, { string_type }
```

Example:

```
elec.net "Gnd"
```

```
elec.net "hvbus" "hv" "high_i" "noisy"
```

The first argument is the net name; subsequent arguments are optional tags.

Net Data Type Casting and Coercion

Nets cannot be cast from other types. Nets support:

- Tagging/untagging via `elec.tag` and `elec.untag`
- Membership testing in Sets
- Connection to Pins via `elec.connect`
- Querying connected objects using quantifier syntax (e.g., `I@Gnd`, `P@Vin`)

Chapter 6

Variables

Variables in EDADSL are prefixed with the `$` character and are assigned values using type-specific assignment operators.

6.1 Variable Declaration and Assignment

Variables are declared and assigned simultaneously using the following syntax:

EBNF:

```
variable           = "$", id
variable_assignment = variable, (typed_assign | inferred_assign),
                        [";", "M", "=", boolean_data_type]

typed_assign       = "[", type, "]", "=", data
inferred_assign    = "t", "=", data

type               = "bool" | "int" | "real" | "complex"
                   | "list" | "string" | "set"
                   | "part" | "connection" | "pin"
                   | "net"
```

6.1.1 Type-Inferred Assignment

As syntax sugar, `$var t= val` is equivalent to `$var [type(val)]= val`. The type is inferred from the value at assignment time.

```
$a t= 3           #c# Equivalent to $a [int]= 3
$name t= "EDADSL" #c# Equivalent to $name [string]= "EDADSL"
$flag t= True     #c# Equivalent to $flag [bool]= True
$colors t= [1, 2, 3] #c# Equivalent to $colors [list]= [1, 2, 3]
```

6.1.2 Variable Redclaration

Variables can be redeclared with a different type. The new declaration overwrites the previous value and type:

```
$x [int]= 5          ## $x is int
$x [real]= 3.14      ## $x is now real
$x [string]= "hi"    ## $x is now string
```

The type determines the variable's data type:

- `bool` – Boolean
- `int` – Integer
- `real` – Real
- `complex` – Complex number
- `list` – List
- `string` – String
- `set` – Set
- `part` – Part
- `connection` – Connection
- `pin` – Pin
- `net` – Net

6.2 Mutability

By default, variables are mutable (can be reassigned). To make a variable immutable, append `; M = False` after the assignment:

```
$immutable [int]= 42; M = False
$mutable [int]= 100      ## Can be reassigned
```

6.3 Scope

Variables declared at the top level of the program have **global scope** and are visible from their declaration to the end of the program. Variables declared inside a code block (`if`, `for`, `while`, `do-while`) are **local to that block** and are destroyed when the block exits. Variables declared inside a function are local to that function (see Section 11.2.4). A variable may be re-declared at the same scope level; the new declaration silently replaces the previous binding. Variables must be declared before they are accessed; reading an uninitialized variable raises a **Runtime Error**.

6.4 Examples

```
$a [int]= 3
$b [int]= 5
$result [int]= $a + $b          #c# Scalar assignment

$name [string]= "EDADSL"        #c# String assignment
$flag [bool]= True              #c# Boolean assignment

$colors [list]= ["red", "green", "blue"] #c# List
$hwsset [set]= { Q1, Q2, R5 }      #c# Set
```

Chapter 7

Expressions

Expressions in EDADSL evaluate to values and can be composed of literals, variables, function calls, and operators.

7.1 Expression Grammar

EBNF:

```
expression          = arithmetic_expression
                    | boolean_expression
                    | comparison_expression
                    | set_expression
                    | postfix_expression
                    | function_call
                    | lambda
                    | "(" expression ")"
```

```
arithmetic_expression = expression,
    ("+" | "-" | "*" | "/" | "%" | "^" | "." | "x"
    | ".*" | ".^"),
    expression
    | "-" expression
    | expression, ".T"
```

```
boolean_expression = expression,
    ("&&" | "||" | "^^"), expression
    | "!", expression
```

```
comparison_expression = expression,
   ("<" | "<=" | "==" | "!=" | "=n=" | "><" | ">=" | ">"
    | "~" | "~*"), expression
    | expression, "~+", "(", expression, "~", expression
    | expression, "~*", "(", expression, "~", expression
```

```
set_expression = expression, ("u" | "n" | "/"), expression
```



```

| "||", expression
| expression, ("c" | "E" | "!E"), expression

postfix_expression = variable, ("++" | "--")

compound_assignment = variable,
    #c# arithmetic
    ("+=" | "-=" | "*=" | "/=" | "%=" | "^="
     | "+r=" | "-r=" | "/r=" | "%r=" | "^r=")
    #c# set
| variable, ("u=" | "n=")
    #c# vector
| variable, (".=" | "x=" | "xr=")
    #c# tensor
| variable, (".*=" | ".^=" | ".*r=" | ".^r=")
| variable, "~.T"
    #c# bitwise / shift
| variable, ("<<=" | ">>=" | "<<r=" | ">>r=")
| variable, ("<<<[" int_literal "]"=
    | ">>>[" int_literal "]"=
    | "<<<[" int_literal "]"r="
    | ">>>[" int_literal "]"r=")
    #c# boolean
| variable, ("&&=" | "||=" | "^=" | "!&=" | "!=" | "!^=")
    #c# imply / nimply
| variable, ("->=" | "!-->=" | "->r=" | "!-->r="), expression

```

7.2 Operator Precedence

Operators are evaluated in the following order of precedence (highest to lowest):

1. Exponentiation (^) – right-associative
2. Boolean negation (!)
3. Postfix increment/decrement (\$var++, \$var--), transpose (.T)
4. Set cardinality (||a)
5. Multiplication (*), Division (/), Floor Division (//), Modulo (%), Dot Product (.), Cross Product (x), Matrix Multiply (.*), Matrix Power (.^)
6. Left Shift (<<), Right Shift (>>), Left Rotate (<<<[w]), Right Rotate (>>>[w])
7. Addition (+), Subtraction (-)
8. Comparison operators (<, <=, ==, !=, =n=, ><, >=, >, ~, ~*, ~+()~, ~*()~)
9. Set comparison (c, E, !E)
10. Logical AND (&&), NAND (!&)

11. Logical OR (`||`), NOR (`!||`)
12. Logical XOR (`^^`), XNOR (`!^`)
13. IMPLY (`->`), NIMPLY (`!->`)
14. Conditional expression (`?->...!...`)
15. Lambda expression (`L::(...) -> {...}`)
16. User-defined operators (`o.u.*`)

Note: `||` has two distinct meanings depending on position. As a **prefix** operator (`||expr`), it denotes cardinality/length and has high precedence (level 4). As an **infix** operator (`expr || expr`), it denotes logical OR and has low precedence (level 10). Parentheses override precedence. See Section 8 for detailed operator descriptions.

7.3 Evaluation Order

Expressions are evaluated left-to-right for operators of equal precedence. Parentheses can be used to override precedence. Function arguments are evaluated before the function is called.

Chapter 8

Operators

8.1 Built-in Operators

8.1.1 Arithmetic

Arithmetic operators perform mathematical operations. Most operators support multiple type combinations.

```
n + m  Addition / Concatenation
n - m  Subtraction
n * m  Multiplication
n / m  Division (always returns real)
n // m Floor Division (returns int)
n % m  Modulo
n ^ m  Exponentiation
n << m Left Shift (returns int)
n >> m Right Shift (returns int)
n <<<[w] m Left Rotate (returns int, w-bit word)
n >>>[w] m Right Rotate (returns int, w-bit word)
```

Type Overloading

- `+: int + int → int`
- `+: int + real → real`
- `+: real + int → real`
- `+: real + real → real`
- `+: string + string → string (concatenation)`
- `+: list + list → list (append)`
- `-: int - int → int`
- `-: int - real → real`
- `-: real - int → real`

- -: real - real $\rightarrow$ real
- *: int * int $\rightarrow$ int
- *: int * real $\rightarrow$ real
- *: real * int $\rightarrow$ real
- *: real * real $\rightarrow$ real
- *: int * string $\rightarrow$ string (repetition)
- *: int * list $\rightarrow$ list (replication)
- /: int / int $\rightarrow$ real (true division)
- /: real / int $\rightarrow$ real
- /: int / real $\rightarrow$ real
- /: real / real $\rightarrow$ real
- //: int // int $\rightarrow$ int (floor division)
- %: int % int $\rightarrow$ int
- %: real % int $\rightarrow$ real
- %: int % real $\rightarrow$ real
- %: real % real $\rightarrow$ real
- ^: int ^ int $\rightarrow$ int (non-negative exponents) or real (negative exponents)
- ^: int ^ real $\rightarrow$ real
- ^: real ^ int $\rightarrow$ real
- ^: real ^ real $\rightarrow$ real
- <<: int << int $\rightarrow$ int (left shift)
- >>: int >> int $\rightarrow$ int (right shift)
- <<<[w]: int <<<[w] int $\rightarrow$ int (left rotate)
- >>>[w]: int >>>[w] int $\rightarrow$ int (right rotate)
- +: int + complex $\rightarrow$ complex
- +: real + complex $\rightarrow$ complex
- +: complex + int $\rightarrow$ complex
- +: complex + real $\rightarrow$ complex
- +: complex + complex $\rightarrow$ complex
- -: int - complex $\rightarrow$ complex

- `-: real - complex → complex`
- `-: complex - int → complex`
- `-: complex - real → complex`
- `-: complex - complex → complex`
- `*: int * complex → complex`
- `*: real * complex → complex`
- `*: complex * int → complex`
- `*: complex * real → complex`
- `*: complex * complex → complex`
- `/: int / complex → complex`
- `/: real / complex → complex`
- `/: complex / int → complex`
- `/: complex / real → complex`
- `/: complex / complex → complex`
- `^: int ^ complex → complex`
- `^: real ^ complex → complex`
- `^: complex ^ int → complex`
- `^: complex ^ real → complex`
- `^: complex ^ complex → complex`

Mixed-type operations promote to `real`. If any operand is `complex`, the result is `complex`. Floor division (`//`) truncates toward negative infinity. Modulo uses truncated division (`sign(result) = sign(dividend)`). Exponentiation is right-associative: `-2^3 = -8`.

Edge Cases

- Division by zero (`/`, `//`, `%`) raises a `Division by Zero Error`.
- Negative exponent on `int` base auto-promotes to `real`: `2 ^ -1` returns `0.5`.
- Shift operators (`<<`, `>>`) require `int` operands; a `Type Error` is raised otherwise. The right operand must be non-negative; a `Value Error` is raised for negative shift amounts.
- Rotation operators (`<<<[w]`, `>>>[w]`) require `int` operands; a `Type Error` is raised otherwise. The word width `w` must be a positive integer literal. The rotation amount must be non-negative; a `Value Error` is raised otherwise.
- A `Type Error` is raised when operands are incompatible (e.g., `string + int`).

8.1.2 Increment / Decrement

Postfix increment and decrement operators add 1 or subtract 1 from an int or real variable. The variable must already be declared.

```
$var++    Increment $var by 1 (returns old value)
$var--    Decrement $var by 1 (returns old value)
```

Semantics `$var++` increments `$var` by 1 and returns the original value before the increment. `$var--` decrements `$var` by 1 and returns the original value before the decrement.

Examples:

```
$counter [int]= 0
$counter++
## $counter = 1
```

```
$counter++
## $counter = 2
```

```
$counter--
## $counter = 1
```

```
## Can be used in expressions
$val [int]= $counter++    ## $val = 1, $counter = 2
```

8.1.3 Compound Assignment

Compound assignment operators combine an operation with assignment. They are available for all operators.

Arithmetic

```
## forward
$var += expr    $var = $var + expr
$var -= expr    $var = $var - expr
$var *= expr    $var = $var * expr
$var /= expr    $var = $var / expr
$var %= expr    $var = $var % expr
$var ^= expr    $var = $var ^ expr
```

```
## reverse (reversed operands)
$var +r= expr    $var = expr + $var
$var -r= expr    $var = expr - $var
$var /r= expr    $var = expr / $var
$var %r= expr    $var = expr % $var
$var ^r= expr    $var = expr ^ $var
```

Set

```
$var u= expr    $var = $var u expr  
$var n= expr    $var = $var n expr
```

Vector

```
$var .= expr    $var = $var . expr    (dot product)  
$var x= expr    $var = $var x expr    (cross product)  
$var xr= expr   $var = expr x $var    (reverse cross product)
```

Tensor (Nddarray)

```
$var .*= expr   $var = $var .* expr   (matrix multiply)  
$var .^= expr   $var = $var .^ expr   (matrix power)  
$var .*r= expr  $var = expr .* $var   (reverse matrix multiply)  
$var .^r= expr  $var = expr .^ $var   (reverse matrix power)  
$var ~.T       $var = $var .T        (transpose in place)
```

Bitwise / Shift

```
$var <<= expr    $var = $var << expr  
$var >>= expr    $var = $var >> expr  
$var <<r= expr    $var = expr << $var  
$var >>r= expr    $var = expr >> $var  
$var <<<[w]= expr $var = $var <<<[w] expr  
$var >>>[w]= expr $var = $var >>>[w] expr  
$var <<<[w]r= expr $var = expr <<<[w] $var  
$var >>>[w]r= expr $var = expr >>>[w] $var
```

Boolean

```
$var &&= expr    $var = $var && expr  
$var ||= expr    $var = $var || expr  
$var ^= expr     $var = $var ^ expr  
$var !&= expr    $var = !($var && expr)  
$var !|= expr    $var = !($var || expr)  
$var !^= expr    $var = $var == expr
```

IMPLY / NIMPLY

```
$var ->= expr    $var = !$var || expr  
$var !->= expr    $var = $var && !expr  
$var ->r= expr    $var = !expr || $var  
$var !->r= expr    $var = expr && !$var
```

Examples:

```

$x [int]= 5
$x += 3      #c# $x = 8
$x *= 2      #c# $x = 16
$x -= 4      #c# $x = 12
$x /= 3      #c# $x = 4
$x %= 3      #c# $x = 1
$x ^= 3      #c# $x = 1

$str [string]= "hello"
$str += " " + "world"  #c# $str = "hello world"
$str += "cd"           #c# $str = "cdhello"

$list [list]= [1, 2, 3]
$list += [4, 5]        #c# $list = [1, 2, 3, 4, 5]

Reverse compound assignment operators reverse the operand order:

$x [int]= 5
$x -= 3      #c# $x = 3 - 5 = -2
$x /= 2      #c# $x = 2 / 5 = 0.4
$x %= 2      #c# $x = 2 % 5 = 2

$str [string]= "hello"
$str += "cd"  #c# $str = "cdhello"

$v [list]= [1, 2, 3]
$v += [0, 0, 1]  #c# $v = [0, 0, 1] x [1, 2, 3]

$x [int]= 5
$x .= 3      #c# $x = $x . 3 = 15 (in-place dot product)
$x x= [1, 0, 0]  #c# $x = $x x [1, 0, 0] (in-place cross product)

$b [bool]= True
$b != False  #c# $b = !(True && False) = True (NAND)
$b != False  #c# $b = !(True || False) = False (NOR)
$b != False  #c# $b = True == False = False (XNOR)
$b ->= False  #c# $b = !True || False = False (IMPLY)
$b !->= False  #c# $b = True && !False = True (NIMPLY)
$b ->r= True   #c# $b = !True || True = True (reverse IMPLY)
$b !->r= True   #c# $b = True && !True = False (reverse NIMPLY)

$x [int]= 5
$x <<= 2      #c# $x = 5 << 2 = 20
$x >>= 1      #c# $x = 20 >> 1 = 10
$x <<r= 3      #c# $x = 3 << $x = 3 << 10 = 3072
$x >>r= 2      #c# $x = 2 >> $x = 2 >> 3072 = 0

$x [int]= 0b1101

```



```

$x <<<[4]= 1  #c# $x = 0b1101 <<<[4] 1 = 0b1011  (left rotate by 1)
$x >>>[4]= 1  #c# $x = 0b1011 >>>[4] 1 = 0b1101  (right rotate by 1)
$x <<<[4]r= 2  #c# $x = 2 <<<[4] $x = 2 <<<[4]
                    #c# 1101 = 0111 (reverse left rotate)
$x >>>[4]r= 1  #c# $x = 1 >>>[4] $x = 1 >>>[4]
                    #c# 0111 = 1011 (reverse right rotate)

$m [list]= [[1,2],[3,4]]
$m .*= [[5,6],[7,8]]  #c# $m = [[19,22],[43,50]]  (matrix multiply)
$m .^= 2              #c# $m = $m .^ 2           (matrix power)

$n [list]= [[1,2],[3,4]]
$n ~.T
    #c# $n = [[1,3],[2,4]]          (transpose in place)

$p [list]= [[1,2],[3,4]]
$p .*r= [[5,7],[6,8]]
    #c# $p = [[5,7],[6,8]] .* [[1,2],[3,4]]  (reverse)
$p .^r= 2
    #c# $p = 2 .^ $p                (reverse matrix power)

```

Examples:

```

#c# Scalar arithmetic
3 + 4      #c# = 7
10 - 3     #c# = 7
4 * 5      #c# = 20
15 / 4     #c# = 3.75
17 % 5     #c# = 2
2 ^ 10     #c# = 1024
5 << 3     #c# = 40 (5 * 2^3)
40 >> 3    #c# = 5  (40 / 2^3)
0b1101 <<<[4] 1  #c# = 0b1011 (left rotate 4-bit word by 1)
0b1101 >>>[4] 1  #c# = 0b1110 (right rotate 4-bit word by 1)

```

```

#c# String concatenation
"hello" + " " + "world"  #c# = "hello world"

```

```

#c# String repetition
"ab" * 3          #c# = "ababab"

```

```

#c# List operations
[1,2] + [3,4]      #c# = [1,2,3,4]
[0] * 5            #c# = [0,0,0,0,0]

```

8.1.4 Comparison

Comparison operators evaluate to True or False.

```

n <  m Less Than
n <= m Less or Equal
b == c Equal
b != c Not Equal
a =n= b Numeric Equal (cross-type)
a >< b Three-way Numeric Compare
n >= m Greater or Equal
n >  m Greater Than
a ~  b Approximately Equal (uses syst.flag.approx_eps)
a ~* b Approximately Equal (uses syst.flag.approx_eps_mult)
a ~+( c )~ b Approximately Equal with additive tolerance c
a ~*( c )~ b Approximately Equal with multiplicative tolerance c

```

Type Overloading

- `<, <=, >=, >`: `int` or `real` $\rightarrow$ `bool` (ordered)
- `<, <=, >=, >`: `string` $\rightarrow$ `bool` (lexicographic)
- `==, !=`: `any` $\rightarrow$ `bool` (strict equality)
- `=n=`: `int` or `real` $\rightarrow$ `bool` (numeric equality)
- `><`: `int` or `real` $\rightarrow$ `int` (three-way numeric compare)
- `~, ~*, ~+()~, ~*()~`: `int` or `real` $\rightarrow$ `bool` (approximate equality)

Ordered comparisons (`<`, `<=`, `>=`, `>`) only work on ints, reals, and strings. Using them on complex numbers raises a **Type Error**. Equality (`==`, `!=`) works on any type including lists, sets, parts, and connections. `=n=` works only on numeric types.

Numeric Equality `=n=` compares values numerically across types. Unlike `==` which is type-strict, `=n=` returns `True` if the numeric values are equal regardless of whether the operands are `int` or `real`.

- `5 == 5.0` $\rightarrow$ `False` (strict: `int` vs `real`)
- `5 =n= 5.0` $\rightarrow$ `True` (numeric: same value)

Note: `==` is strict type equality. `5 == 5.0` is `False` because the left operand is `int` and the right is `real`. Use `=n=` when you want to compare values regardless of numeric type.

Three-way Numeric Compare `a >< b` performs a three-way numeric comparison using `=n=` semantics.

- Returns 1 if $a > b$
- Returns 0 if $a =n= b$
- Returns -1 if $a < b$

Raises a **Type Error** if either operand is not numeric.

Approximate Equality $a \sim b$ returns True if $|a - b| \leq \text{syst.flag.approx_eps}$. $a \sim * b$ returns True if $|a - b| \leq \text{syst.flag.approx_eps_mult} \cdot \max(|a|, |b|)$. $a \sim +(c) \sim b$ returns True if $|a - b| \leq c$. $a \sim *(c) \sim b$ returns True if $|a - b| \leq c \cdot \max(|a|, |b|)$. The default value of `syst.flag.approx_eps` is 1.

Examples:

```
3 < 5                #c# = True
"a" < "b"           #c# = True (lexicographic)
[1,2] == [1,2]       #c# = True (list equality)
Q1 == Q1             #c# = True (part identity)

#c# Numeric equality (cross-type)
5 == 5.0             #c# = False (strict: int vs real)
5 =n= 5.0            #c# = True  (numeric: same value)
5 =n= 5.1            #c# = False (different values)
3.14 =n= 3.14        #c# = True  (real vs real works too)

#c# Three-way numeric compare
5 >< 3                #c# = 1  (5 > 3)
5 >< 5                #c# = 0  (5 =n= 5)
5 >< 7                #c# = -1 (5 < 7)
5 >< 5.0              #c# = 0  (numeric equality)
3.14 >< 3.14          #c# = 0  (real vs real)

#c# Approximate equality
syst.flag.approx_eps 0.01
3.0 ~ 3.005          #c# = True (within 0.01)
3.0 ~ 3.1            #c# = False (not within 0.01)
3.0 ~+(0.2)~ 3.1     #c# = True (within 0.2)
3.0 ~+(0.05)~ 3.1    #c# = False (not within 0.05)

#c# Execution mode
syst.flag.exec_mode "strict" #c# warnings become errors

#c# Multiplicative approximate equality (relative tolerance)
100 ~*(0.01)~ 100.5 #c# = True (within 1%)
100 ~*(0.001)~ 100.5 #c# = False (not within 0.1%)

#c# Multiplicative approximate equality with system default
syst.flag.approx_eps_mult 0.01
100 ~* 100.5        #c# = True (within 1%)
```

8.1.5 Boolean Logic

Boolean operators combine or negate boolean values.

```
b && c Logical And
b || c Logical Or
```

```

b ^^ c Logical Xor
b !& c Nand (Not And)
b !| c Nor (Not Or)
b !^ c Xnor (Not Xor)
b -> c Imply (a implies b)
b !-> c Nimp (a does not imply b)
!b      Negation

```

- Operands: bool, bool
- Result: bool
- Short-circuit evaluation: `a && b` evaluates `b` only if `a` is True. `a || b` evaluates `b` only if `a` is False.

Truth tables:

a	b	a!&b	a! b	a!^b	a->b	a!->b
T	T	F	F	T	T	F
T	F	T	F	F	F	T
F	T	T	F	F	T	F
F	F	T	T	T	T	F

Semantics:

- `!&` (NAND): `!(a && b)`
- `!|` (NOR): `!(a || b)`
- `!^` (XNOR): `a == b` (logical equality)
- `->` (IMPLY): `!a || b`. Not commutative: `a -> b ≠ b -> a`
- `!->` (NIMPLY): `a && !b`. Not commutative: `a !-> b ≠ b !-> a`

Examples:

```

True && False   #c# = False
True || False   #c# = True
True ^^ False   #c# = True
True ^^ True    #c# = False
True !& False    #c# = True  (NAND)
True !| False    #c# = False (NOR)
True !^ False    #c# = True  (XNOR)
True -> False    #c# = False (IMPLY)
True !-> False    #c# = True  (NIMPLY)
!True           #c# = False

```

8.1.6 Collection Operators

Collection operators manipulate sets, lists, and strings.

```

a u b  Union (sets only)
a n b  Intersection (sets only)
a / b  Difference (sets) / Division (numeric)
||a    Cardinality / Length

```

Type Overloading

- `u: set + set → set` (union)
- `n: set + set → set` (intersection)
- `/: set / set → set` (difference)
- `/: int / int → real` (division)
- `||: ||set → int` (cardinality)
- `||: ||list → int` (length)
- `||: ||string → int` (length)

Set operations (`u`, `n`) only work on sets. The `/` operator is overloaded: on sets it returns difference, on ints or reals it returns division. Cardinality (`||`) returns the element count for sets, lists, and strings.

Edge Cases

- A `Type Error` is raised for `u` or `n` if either operand is not a set.
- A `Type Error` is raised for `||` if the operand is not a set, list, or string.
- A `Type Error` is raised for `/` if the operands are a set and a non-set type (e.g., `set / int`).

Examples:

```
#c# Set operations
{1,2} u {2,3}      #c# = {1,2,3} (union)
{1,2} n {2,3}      #c# = {2} (intersection)
{1,2,3} / {2}      #c# = {1,3} (difference)

#c# Cardinality
||{1,2,3}          #c# = 3 (set size)
|[1,2,3,4]         #c# = 4 (list length)
|"hello"           #c# = 5 (string length)
```

8.1.7 List Operators

List operators perform vector and matrix operations on lists.

```
a . b  Dot product (real result)
a x b  Cross product (list result)
```

Type Overloading

- `.: list . list → real` (dot product, n-dimensional)
- `x: list x list → ndarray` (cross product, 3D only)

The dot product computes $\sum_i a_i \cdot b_i$ for n-dimensional vectors (any length). The cross product computes the vector cross product (both lists must have length 3).

Edge Cases

- Dot product: A **Value Error** is raised if the vectors have different lengths.
- Dot product: A **Type Error** is raised if any element is not numeric (**int** or **real**).
- Dot product: Two empty vectors return 0.
- Cross product: A **Value Error** is raised if either vector does not have length 3.
- Cross product: A **Type Error** is raised if any element is not numeric.

Examples:

```
#c# Dot product (n-dimensional)
[1, 2, 3] . [4, 5, 6]      #c# = 32 (1*4 + 2*5 + 3*6)
[1, 2] . [3, 4]           #c# = 11 (2D)
[1, 2, 3, 4] . [5, 6, 7, 8] #c# = 70 (4D)
```

```
#c# Cross product (3D only)
[1, 0, 0] x [0, 1, 0]      #c# = [0, 0, 1]
[1, 2, 3] x [4, 5, 6]     #c# = [-3, 6, -3]
```

8.1.8 Tensor (Nddarray) Operators

The `.` prefix on operators enables matrix/tensor operations on ndarrays. Without the prefix, operations are element-wise (see Section above).

Matrix Multiply

`a .* b` Matrix multiply (dot-product for 1D, matrix multiply for 2D)

- Operands: ndarray, ndarray
- Result: ndarray or real
- Shape rules:
 - 1D `.*` 1D $\rightarrow$ scalar (dot product, same as `.`)
 - 2D($m \times k$) `.*` 2D($k \times n$) $\rightarrow$ 2D($m \times n$)
 - 2D($m \times k$) `.*` 1D(k) $\rightarrow$ 1D(m)

Edge Cases

- A **Value Error** is raised if inner dimensions do not match (e.g., 2D(2×3) `.*` 2D(2×2)).
- A **Value Error** is raised if 2D `.*` 1D dimension is incompatible (e.g., 2D(2×3) `.*` 1D(2)).
- A **Type Error** is raised if any element is not numeric.

Examples:

```
#c# 1D .* 1D = scalar (dot product)
[1, 2, 3] .* [4, 5, 6]      #c# = 32

#c# 2D .* 2D = matrix multiply
[[1,2],[3,4]] .* [[5,6],[7,8]] #c# = [[19,22],[43,50]]

#c# 2D .* 1D = matrix-vector multiply
[[1,2],[3,4]] .* [5, 6]      #c# = [17, 39]
```

Transpose

```
a .T   Transpose (postfix operator)
a ~.T  Transpose in place (no expression needed)
```

- Operands: ndarray
- Result: ndarray
- Swaps rows and columns: shape $(m \times n) \rightarrow (n \times m)$
- 1D arrays are treated as row vectors and transposed to column vectors.
- `~.T` is shorthand for `[list]= $var.T` — transposes in place without repeating the variable name.

Examples:

```
[1, 2, 3] .T      #c# = [[1],[2],[3]] (1D -> column)
[[1,2,3]] .T      #c# = [[1],[2],[3]] (row -> column)
[[1,2],[3,4]] .T  #c# = [[1,3],[2,4]]
```

```
$m [list]= [[1,2],[3,4]]
$m ~.T      #c# $m is now [[1,3],[2,4]]
```

Matrix Power

```
a .^ n  Matrix power (repeated matrix multiply, n must be int >= 0)
```

- Operands: ndarray, int
- Result: ndarray
- `A .^ 0` returns the identity matrix (square matrices only)
- `A .^ n = A .* A .* ... .* A` (n times)

Edge Cases

- A `Value Error` is raised if the matrix is not square.
- A `Value Error` is raised if the exponent is negative.
- A `Type Error` is raised if the exponent is not `int`.

Examples:

```
[[1,2],[3,4]] .^ 0      #c# = [[1,0],[0,1]] (identity)
[[1,2],[3,4]] .^ 1      #c# = [[1,2],[3,4]] (unchanged)
[[1,2],[3,4]] .^ 2      #c# = [[7,10],[15,22]]
```

8.1.9 Set Comparison

Set comparison operators test membership and subset relationships.

```
a c b  Subset of
a E b  Element of
a !E b Not element of
```

- Operands: **set**, **set** (subset)
- Operands: any, **set** (membership)
- Result: **bool**
- `a c b` is true if every element of `a` is in `b`
- `a E b` is true if `a` is an element of set `b`

Examples:

```
{1,2} c {1,2,3}      #c# = True (subset)
1 E {1,2,3}          #c# = True (membership)
Q1 E *p              #c# = True (part in all parts)
4 !E {1,2,3}         #c# = True (not element)
```

8.1.10 Conditional Expression

A ternary-like operator for inline conditional values.

```
(condition)?->(value_if_true)!(value_if_false)
```

- Operands: **bool**, any, any
- Result: type of the selected branch
- Evaluates condition, returns first value if **True**, second if **False**
- Both branches must be of the same type (or coercible per the type coercion hierarchy:
`int < real < complex`)

Examples:

```
$x [int]= 5
$result [int]= ($x > 3)?->("big")!("small")
#c# $result = "big"

$val [int]= 10
$abs [int]= ($val >= 0)?->($val)!($val * -1)
```



```
#c# $abs = 10

$a [int]= 1
$b [int]= 2
$max [int]= ($a > $b)?->($a)!($b)
#c# $max = 2
```

8.1.11 Lambda Expressions

Lambda expressions create anonymous functions that can be passed as arguments to higher-order functions. They are defined with the `L::` prefix (consistent with the `f.u.` prefix for user-defined functions).

Syntax

EBNF:

```
lambda = "L::", "(", [ parameter, { ",", parameter } ], ")",
        "->", "{", { statement }, expression, "}"
parameter = "$", id, "[", type, "]"
```

Parameters must always have explicit types. The last expression in the body is the implicit return value. Multi-line bodies are allowed (multiple statements in `{}`).

Type Overloading

- Parameters may be any scalar type: `int`, `real`, `bool`, `string`
- Parameters may also be `list` when operating on lists of lists
- Return type is inferred from the body expression
- All parameter types must match the elements passed by the higher-order function

Scope Rules

- Lambdas have **read-only** access to variables from the enclosing scope (closures can read but not write)
- Lambdas **can** declare local variables inside their body (block-scoped to the lambda)
- Lambdas **cannot** modify variables from the enclosing scope
- Attempting to write to an enclosing variable raises a Compile Error

Return Value

The last expression in the body is the implicit return value. There is no explicit `return` keyword inside lambdas. If the body contains only statements with no trailing expression, the lambda returns nothing (useful for side effects only, e.g., with `f.print`).

Examples

```
#c# Basic lambda (passed to map)
map([1, 2, 3], L::($x [int])->{ $x * 2 })
#c# Returns [2, 4, 6]

#c# Lambda with multiple parameters
L::($x [int], $y [int])->{ $x + $y }
#c# Returns a function object (used inline)

#c# Multi-line lambda
map([1, 2, 3], L::($x [int])->{
    $y [int]= $x * 10
    $y + 1
})
#c# Returns [11, 21, 31]

#c# Closure (read-only access to outer variable)
$factor [int]= 10
map([1, 2, 3], L::($x [int])->{ $x * $factor })
#c# Returns [10, 20, 30]

#c# Lambda with real types
map([1.5, 2.5, 3.5], L::($x [real])->{ $x * $x })
#c# Returns [2.25, 6.25, 12.25]

#c# Lambda with nested list (2D ndarray)
map([[1,2],[3,4]], L::($row [list])->{ $row })
#c# Returns [[1,2],[3,4]] (identity, demonstrates list parameter)

#c# Lambda with bool parameter
filter([True, False, True], L::($x [bool])->{ !$x })
#c# Returns [False]
```

Error Conditions

- Compile Error: parameter type is missing (e.g., `L::($x)->{ ... }` is invalid)
- Compile Error: attempting to write to an outer variable inside a lambda
- Runtime Error: type mismatch between parameter type and actual element type

8.2 Operator Precedence

Operators are evaluated in the following order (highest to lowest):

1. `^` – Exponentiation (right-associative)
2. `!` – Boolean negation

3. `$var++`, `$var--`, `.T` – Postfix increment / decrement, transpose
4. `||a` – Cardinality / length
5. `*`, `/`, `//`, `%`, `.`, `x`, `.*`, `.^` – Multiplication, division, floor division, modulo, dot product, cross product, matrix multiply, matrix power
6. `<<`, `>>`, `<<<`, `>>>` – Left shift, right shift, left rotate, right rotate
7. `+`, `-` – Addition, subtraction
8. `<`, `<=`, `==`, `!=`, `=n=`, `><`, `>=`, `>`, `~`, `~*`, `~+()`, `~*()` – Comparison
9. `c`, `E`, `!E` – Set comparison
10. `&&`, `!&` – Logical AND, NAND
11. `||`, `!|` – Logical OR, NOR
12. `^^`, `!^` – Logical XOR, XNOR
13. `->`, `!->` – IMPLY, NIMPLY
14. `?->...!...` – Conditional expression
15. `L::(...)->{...}` – Lambda expression
16. User-defined operators (`o.u.*`)

Note: `||` has two distinct meanings depending on position. As a **prefix** operator (`||expr`), it denotes cardinality/length and has high precedence (level 3). As an **infix** operator (`expr || expr`), it denotes logical OR and has low precedence (level 9). Parentheses override precedence. Function arguments are evaluated before the function is called.

8.3 User-defined Operators

User-defined operators extend the language with custom infix notation.

8.3.1 Definition

EBNF:

```
"letop", "o.u.", id, assigner, data
```

The operator name must follow the pattern `o.u.[_a-zA-Z0-9]+`.

8.3.2 Examples

```
letop o.u.mod_minus_1 [int]=
    a % b - 1

f.print( 4 o.u.mod_minus_1 3 )  #c# = 0

letop o.u.parallel [int]=
```

$(a * b) / (a + b)$

```
f.print( 100 o.u.parallel 100 )  ## = 50
```

8.3.3 Precedence

User-defined operators have the lowest precedence among all operators. Use parentheses to ensure correct evaluation order.

Chapter 9

Constants

Constants are immutable values that cannot be reassigned after declaration.

9.1 Built-in Constants

9.1.1 Proper Constants

Proper constants are fixed values provided by the language.

Mathematical Constants

- `c.pi` – The mathematical constant $\pi \approx 3.14159265\dots$
- `c.e` – Euler’s number $e \approx 2.71828182\dots$
- `c.i` – Imaginary unit $\sqrt{-1}$ (complex type)
- `c.iinf` – Integer positive infinity
- `c.rinf` – Real positive infinity
- `c.inf` – Unsigned infinity (numeric)
- `c.egamma` – Euler-Mascheroni constant $\gamma \approx 0.5772156649\dots$
- `c.sqrt2` – $\sqrt{2} \approx 1.4142135623\dots$ (RMS conversion: $V_{\text{rms}} = V_{\text{pk}}/\sqrt{2}$)
- `c.sqrt2_inv` – $1/\sqrt{2} \approx 0.7071067811\dots$ (the -3dB point)
- `c.ln2` – $\ln(2) \approx 0.6931471805\dots$ (half-life, RC time constants)

Physical Constants

- `c.c` – Speed of light in vacuum $c \approx 2.998 \times 10^8$ m/s
- `c.h` – Planck’s constant $h \approx 6.626 \times 10^{-34}$ J·s
- `c.hbar` – Reduced Planck’s constant $\hbar \approx 1.055 \times 10^{-34}$ J·s
- `c.q` – Elementary charge $e \approx 1.602 \times 10^{-19}$ C

- `c.kb` – Boltzmann constant $k_B \approx 1.381 \times 10^{-23}$ J/K
- `c.me` – Electron mass $m_e \approx 9.109 \times 10^{-31}$ kg
- `c.mp` – Proton mass $m_p \approx 1.673 \times 10^{-27}$ kg
- `c.g` – Standard gravitational acceleration $g \approx 9.80665$ m/s²
- `c.atm` – Standard atmosphere 1 atm = 101325 Pa
- `c.sigma` – Stefan-Boltzmann constant $\sigma \approx 5.670 \times 10^{-8}$ W/(m²· K⁴)
- `c.k` – Coulomb (electrostatic) constant $k_e \approx 8.988 \times 10^9$ N·m²/C²
- `c.mu0` – Vacuum permeability $\mu_0 \approx 4\pi \times 10^{-7}$ H/m
- `c.eps0` – Vacuum permittivity $\epsilon_0 \approx 8.854 \times 10^{-12}$ F/m
- `c.kT300` – Thermal voltage at 300K $kT/q \approx 0.02585$ V (semiconductor bias point)
- `c.G0` – Conductance quantum $G_0 = 2e^2/h \approx 7.748 \times 10^{-5}$ S
- `c.phi0` – Magnetic flux quantum $\Phi_0 = h/(2e) \approx 2.068 \times 10^{-15}$ Wb

9.1.2 Pseudo-Constants

Pseudo-constants are system-defined sets that provide access to electrical objects.

Set Pseudo-Constants

- `*n` – Empty set
- `*e` – All parts, connections, pins, holes, and nets
- `*i` – All pins and connections
- `*u` – All parts, connections, and pins on the upper side of the board
- `*b` – All parts, connections, and pins on the bottom side of the board
- `*h` – All holes
- `*c` – All connections
- `*p` – All parts
- `*t` – All pins

Example:

```
$my_set = *n    #c# Initialize with empty set
$my_set = $my_set u { R1, R2 }    #c# Add elements
```

9.2 User-defined Constants

User-defined constants are created using the `let` keyword with the `c.u.` prefix:

EBNF:

```
constant_assignment = "let", "c.u.", id, assigner,  
                      [";", "P", "=", boolean_data_type]
```

Examples:

```
let c.u.$max_current [int]= 30  
let c.u.$board_name [string]= "PowerSupply"
```

Constants are immutable by definition and cannot be reassigned.

Chapter 10

Control Flow

EDADSL provides several control flow constructs for conditional execution, iteration, and program flow control.

10.1 Conditional Execution

Conditional execution uses the `if` keyword followed by a boolean expression and a code block. Additional conditions can be tested with `otherwise`. A final `else` clause provides a fallback code block executed when no conditions match:

EBNF:

```
conditional = "if", boolean_expression,  
             ":", code_block,  
             { "otherwise", boolean_expression,  
               ":", code_block },  
             ["else", ":", code_block]
```

Example:

```
$flag [bool]= True  
if $flag ::{  
    f.print("Flag is set")  
} otherwise $other ::{  
    f.print("Other condition")  
} else ::{  
    f.print("Flag is not set")  
}
```

Basic if without else:

Example:

```
$flag [bool]= True  
if $flag ::{  
    f.print("Flag is set")  
}
```


10.2 For Loops

For loops iterate over a list or set:

EBNF:

```
iterator = "for", variable, "E",  
          (list | set), ":", code_block
```

Example:

```
$colors [list]= ["red", "green", "blue"]  
for $c E $colors ::{  
    f.print($c)  
}
```

The loop variable (`$c`) takes on each value in the list/set in sequence.

Variable Scoping The loop variable is block-scoped: it is only visible inside the loop body and is destroyed when the loop exits. If the loop iterates over an empty list, the loop body never executes and the variable is never initialized; accessing it after the loop raises a **Runtime Error**.

10.3 While Loops

While loops repeat a code block as long as a boolean condition evaluates to **True**.

10.3.1 Basic While Loop

EBNF:

```
while_loop = "while", boolean_expression,  
            ":", code_block
```

Example:

```
$i [int]= 0  
while $i < 5 ::{  
    f.print($i)  
    $i = $i + 1  
}
```

Variable Scoping While loops do not introduce any implicit variables. Any variables declared inside the loop body are block-scoped and destroyed when the loop exits. Variables used in the loop condition must be declared before the loop.

10.3.2 While with Else Clause

An optional **else** clause executes when the loop condition becomes **False**. If the loop exits via **break**, the **else** clause is **not** executed:

EBNF:
while_loop = "while", boolean_expression,
 "::", code_block,
 ["else", "::", code_block]

Example:
\$i [int]= 0
while \$i < 5 ::{
 \$i = \$i + 1
} else ::{
 f.print("Loop completed normally")
}

Example with break:
\$i [int]= 0
while \$i < 10 ::{
 if \$i == 3 ::{
 break
 }
 \$i = \$i + 1
} else ::{
 f.print("This will NOT be printed")
}

10.3.3 Do-While Loops

Do-while loops execute the code block **at least once** before evaluating the condition:

EBNF:
do_while = "do", "::", code_block,
 "while", boolean_expression

Example:
\$val [real]= f.random()
do ::{
 \$val = f.random()
} while \$val < 0.5

The code block executes first, then the condition is checked. If the condition is **True**, the loop repeats. The same block-scoping rules as while loops apply: variables declared inside the code block are destroyed when the block exits. If the code block raises an error, the loop terminates immediately and the condition is not evaluated. **break** and **continue** work as expected inside do-while loops.

10.3.4 Break and Continue

break exits the innermost loop immediately. **continue** skips the remainder of the current iteration and proceeds to the next evaluation of the loop condition:

```
EBNF:
break_stmt    = "break"
continue_stmt = "continue"
```

```
Example:
for $x E $values ::{
    if $x < 0 ::{
        continue
    }
    if $x > 100 ::{
        break
    }
    f.print($x)
}
```

```
Example with while:
$i [int]= 0
while $i < 10 ::{
    $i = $i + 1
    if $i % 2 == 0 ::{
        continue
    }
    f.print($i)
}
```

`break` and `continue` apply only to the innermost enclosing loop.

10.3.5 Nested While Loops

While loops may be nested to arbitrary depth:

```
Example:
$rows [int]= 3
$cols [int]= 3
$i [int]= 0
while $i < $rows ::{
    $j [int]= 0
    while $j < $cols ::{
        f.print("Cell (" + f.str($i) + "," + f.str($j) + ")")
        $j = $j + 1
    }
    $i = $i + 1
}
```

10.3.6 Safety: Maximum Iteration Limit

To prevent infinite loops, the interpreter enforces a maximum iteration count per loop. The limit is controlled by `syst.flag.max_loop_iter` (default: 100000). If a loop exceeds this

limit, a `Value Error` is raised:

Example:

```
syst.flag.max_loop_iter 500
$i [int]= 0
while $i < 1000 ::{
    $i = $i + 1
}
f.print($i) # never reached | Value Error at 501 iterations
```

The limit applies independently to each loop. Nested loops each have their own counter.

10.3.7 Infinite Loops

An infinite loop can be written with `True` as the condition. Use `break` to exit:

Example:

```
$count [int]= 0
while True ::{
    $count = $count + 1
    if $count == 10 ::{
        break
    }
}
f.print($count) #c# Prints: 10
```

10.4 Goto Statements

Goto statements provide unconditional jumps to labeled positions:

EBNF:

```
gotomarker = "gotomarker", id
gotojump   = "gotojump", id
```

Example:

```
gotojump skip_section
f.print("This will not be printed")
gotomarker skip_section
f.print("Jumped here")
```

Warning – Deprecated Usage Goto statements are **strongly discouraged**. They make code difficult to read, debug, and maintain. Prefer structured control flow (`if`, `for`, `while`, `break`, `continue`) for all new code. Gotos are retained for backward compatibility only. Gotos may cross loop boundaries (e.g., jump from outside a loop into the loop body) but may not cross function boundaries.

10.5 End of Program Statements

EBNF:

```
halt = "halt"
```

```
noop = "skip"
```

- `halt` – Terminates program execution and exports all created files
- `skip` – No operation; program continues to next statement

10.6 Output and Logging

EBNF:

```
echo_statement = "echo", variable
```

```
writetolog      = "writetolog", string_expression
```

Example:

```
f.print("Hello, EDADSL!")
```

```
writetolog "Board design completed successfully"
```

- `echo` – Keyword that prints a variable reference to the console. Only accepts a variable, not an arbitrary expression. Use `f.print()` for expression output.
- `writetolog` – Keyword that appends a message to the log file

Chapter 11

Functions

11.1 Built-in Functions

Important: All built-in function calls require the `f.` prefix. This is defined in the formal grammar (Chapter 3):

```
function_call_built_in = "f.", id, "(",  
                        [{ data, ",", }], data, ")"
```

Correct usage: `f.abs(-3.2)`, `f.sin($x)`, `f.map($list, $func)`

Incorrect: `abs(-3.2)`, `sin($x)`, `map($list, $func)`

`f.print()` takes a **single** argument. Use `f.str()` for string concatenation:

```
f.print("value = " + $x)           # OK  
f.print(f.str("value = ", $x))     # OK  
f.print("value = ", $x)           # WRONG - two arguments
```

11.1.1 Mathematical Functions

Absolute Value Function

The Absolute Value Function (denoted as `abs`) returns the Absolute Value of the Argument or the Magnitude of a Vector.

Morphology of the Absolute Value Function:

EBNF:

```
"f.abs", "(", argument, [ ",", "vec", "=",  
                        boolean_data_type ], ")"
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.abs(-3.2)
```

This Example would return a value of `3.2`.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.abs([-3, 0.2, 0, -7])
```

This Example would return `[3, 0.2, 0, 7]`.

If `vec = True` is passed a List shall be passed as the Argument and the Function will return Magnitude of the List as an n-dimensional Vector, where n is the Length of the List.

Example:

```
f.abs( [ 3, -4 ], vec = True )
```

This Example would return `5.0`.

There are no Domain-Restrictions for the function inside $\mathbb{R}^n$.

Complex Overload When the Argument is a Complex Number $z = a + bi$, `abs` returns the Modulus (Magnitude) of the Complex Number, defined as:

$$|a + bi| = \sqrt{a^2 + b^2}$$

The Result is always a Real Number, even if the Argument is Complex.

Domain: All Complex Numbers ($\mathbb{C}$).

Example:

```
f.abs(3 + 4i)    #c# Returns 5.0
f.abs(1 + 1i)    #c# Returns approximately 1.41421
f.abs(0 - 5i)    #c# Returns 5.0
```

Parallel Equivalent Function

The Parallel Equivalent Function (denoted as `parallel`) computes the parallel combination of a list of values. This is the standard formula for parallel resistances, conductances, or any set of admittances.

Morphology of the Parallel Equivalent Function:

EBNF:

```
"f.parallel", "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a List of Numeric Values) a Value Error is raised.

The Argument must be a List of Numeric Values (`int`, `real`, or `complex`). The function computes:

$$\text{parallel}([v_1, v_2, \dots, v_n]) = \frac{1}{\frac{1}{v_1} + \frac{1}{v_2} + \dots + \frac{1}{v_n}}$$

If any Element in the List is Zero, the Function immediately Returns Zero.

If any Element is Infinite, it is Ignored (does not contribute to the Result).

The Return Type is determined by the Numerical Value of the Result:

- If the Result is Numerically Equal to an Integer, Return `int`
- Otherwise, if the Result is Numerically Equal to a Real, Return `real`
- Otherwise, Return `complex`

Examples:

```
f.parallel([100, 100])           #c# Returns 50 (int)
f.parallel([100, 200, 300])      #c# Returns 54 (int, softint)
f.parallel([1e3, 2.2e3])         #c# Returns 687.5 (real)
f.parallel([100, 0])             #c# Returns 0 (any zero)
f.parallel([100, 200, 0, 300])   #c# Returns 0 (any zero)
f.parallel([100, inf])           #c# Returns 100 (inf ignored)
f.parallel([100, 200, inf])      #c# Returns 66.666... (inf ignored)
```

Complex Overload When the List contains Complex Numbers, the function computes the parallel combination using Complex Arithmetic. If the Result has a Zero Imaginary Part, it is Returned as `real`.

Example:

```
f.parallel([100 + 0i, 200 + 0i])
    #c# Returns 66.666... (real, imag=0)
f.parallel([3 + 4i, 1 - 2i])      #c# Returns complex result
```

Domain: All Lists of Numeric Values. Empty Lists raise a Value Error. Lists with a single Element return that Element unchanged (after `softint(softreal())`).

Signum Function

The Signum Function (denoted as `sign`) returns the Sign of the Argument.

Morphology of the Signum Function:

EBNF:

```
"f.sign", "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.sign(-3.2)
```

This Example would return a value of `-1`.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.sign([-3, 0.2, 0, -7])
```

This Example would return `[-1, 1, 0, -1]`.

There are no Domain-Restrictions for the function inside $\mathbb{R}^n$.

Regular Trigonometric Functions

Important: Angles are interpreted as Radians by Default.

The Regular Trigonometric Functions (Sine, Cosine and Tangent) (denoted as `sin`, `cos`, `tan`) return the Sine, Cosine, or Tangent of the Argument.

Edge Cases – Trigonometric Functions

- A **Type Error** is raised if the argument is not `int`, `real`, or `complex`.

Morphology of the Trigonometric Functions:

EBNF:

```
( "f.sin" | "f.cos" | "f.tan" ), "(", argument,  
  [ ",", "unit", "=", unit ], ")"
```

```
unit = ( "'", ("rad" | "deg" ), "'" )  
       | ( '""', ("rad" | "deg" ), '""' )
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.sin(1.570796327)
```

This Example would return a value of around 1.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.sin([0, 30, 90], unit = "deg" )
```

This Example would return [0, 0.5, 1].

For Sine and Cosine there are no Domain-Restrictions for the functions inside $\mathbb{R}^n$.

For Tangent there are Domain-Restrictions inside $\mathbb{R}^n$.

It is only defined for $\{x \in \mathbb{R} : \cos x \neq 0\}^n$

Complex Overload All three functions (`sin`, `cos`, `tan`) are also defined for Complex Arguments via Analytic Continuation. For complex $z = a + bi$, the functions are computed using Euler's Formula:

$$\sin(a + bi) = \sin(a) \cosh(b) + i \cos(a) \sinh(b)$$

$$\cos(a + bi) = \cos(a) \cosh(b) - i \sin(a) \sinh(b)$$

$$\tan(a + bi) = \frac{\sin(a + bi)}{\cos(a + bi)}$$

Domain: All Complex Numbers ($\mathbb{C}$) for `sin` and `cos`. For `tan`, all Complex Numbers where $\cos z \neq 0$.

Examples:

```
f.sin(1 + 2i)      #c# Returns approximately 3.1658 - 1.9596i  
f.cos(1 + 2i)      #c# Returns approximately 2.0327 + 3.0519i  
f.tan(0 + 1i)      #c# Returns approximately 0.0 + 0.7616i
```

Regular Inverse Trigonometric Functions

Important: Angles are interpreted as Radians by Default.

The Regular Inverse Trigonometric Functions (Arcsine, Arccosine and Arctangent) (denoted as `asin`, `acos`, `atan`, `arsin`, `arcsin`, `arccos`, `arctan`) return the Inverse for the Sine, Cosine, or Tangent of the Argument.

Morphology of the Trigonometric Functions:

EBNF:

```
inverses_trig_functions, "(", argument,
  [ ",", "unit", "=", unit ], ")"
```

```
inverses_trig_functions =
  "f.asin" | "f.acos" | "f.atan"
  | "f.arsin" | "f.arcsin" | "f.arccos" | "f.artan"
  | "f.arcsin" | "f.arccos" | "f.arctan"
```

```
unit = ( "'", ("rad" | "deg" ), "'" )
       | ( "'", ("rad" | "deg" ), "'", "'" )
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.asin(1)
```

This Example would return a value of around 1.570796327.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.arcsin([0, 0.5, 1], unit = "deg" )
```

This Example would return [0, 30, 90].

For Arctangent there is no Domain-Restrictions for the functions inside $\mathbb{R}^n$.

For Arcsine and Arccosine the Domain is Restricted to $[-1, 1]^n$.

Inverse Tangent of Complex Argument

Hyperbolic Functions

The Hyperbolic Functions (Hyperbolic Sine, Hyperbolic Cosine and Hyperbolic Tangent) (denoted as `sinh`, `cosh`, `tanh`) return the Hyperbolic Sine, Hyperbolic Cosine, or Hyperbolic Tangent of the Argument.

Morphology of the Hyperbolic Functions:

EBNF:

```
( "f.sinh" | "f.cosh" | "f.tanh" ), "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.sinh(1.443635475)
```

This Example would return a value of around 2.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.sinh([0, 1, 2] )
```

This Example would return around [0, 1.175, 3.627].

There are no Domain-Restrictions for the functions inside $\mathbb{R}^n$.

Complex Overload All three functions (`sinh`, `cosh`, `tanh`) are also defined for Complex Arguments. For complex $z = a + bi$, the functions are computed as:

$$\sinh(a + bi) = \sinh(a) \cos(b) + i \cosh(a) \sin(b)$$

$$\cosh(a + bi) = \cosh(a) \cos(b) + i \sinh(a) \sin(b)$$

$$\tanh(a + bi) = \frac{\sinh(a + bi)}{\cosh(a + bi)}$$

Equivalently, the identities $\sinh(z) = -i \sin(iz)$, $\cosh(z) = \cos(iz)$, and $\tanh(z) = \sinh(z)/\cosh(z)$ hold for Complex Arguments.

Domain: All Complex Numbers ($\mathbb{C}$) for `sinh` and `cosh`. For `tanh`, all Complex Numbers where $\cosh z \neq 0$.

Examples:

```
f.sinh(1 + 2i)    #c# Returns approximately -0.4891 + 0.9889i
f.cosh(1 + 2i)    #c# Returns approximately -0.4891 + 0.9889i
f.tanh(0 + 1i)    #c# Returns approximately 0.0 + 0.5524i
```

Hyperbolic Inverse Functions

The Hyperbolic Inverse Functions (Hyperbolic Arcsine, Hyperbolic Arccosine and Hyperbolic Arctangent) (denoted as `asinh`, `acosh`, `atanh`, `arsinh`, `arcosh`, `artanh` or `arcsinh`, `arccosh`, `arctanh`) return the Inverse for the Hyperbolic Sine, Hyperbolic Cosine, or Hyperbolic Tangent of the Argument.

Morphology of the Hyperbolic Functions:

EBNF:

```
inverses_hypb_functions, "(", argument, ")"
```

```
inverses_hypb_functions =
    "f.asinh" | "f.acosh" | "f.atanh"
    | "f.arsinh" | "f.arcosh" | "f.artanh"
    | "f.arcsinh" | "f.arccosh" | "f.arctanh"
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.asinh(1)
```

This Example would return a value of around 0.881373587.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.arcsinh( [0, 0.5, 1] )
```

This Example would return [0, 0.481, 0.881].

For the Hyperbolic Arcsine there is no Domain-Restrictions for the function inside $\mathbb{R}^n$.

For Hyperbolic Arctangent and Hyperbolic Arccosine the Domain is Restricted to $(-1, 1)^n$ and $[1, \infty)^n$ respectively.

Complex Overload All three functions (`asinh`, `acosh`, `atanh`) are also defined for Complex Arguments using the Complex Logarithm. On the Complex Plane, the Real-Domain Restrictions no longer apply:

$$\operatorname{asinh}(z) = \ln\left(z + \sqrt{z^2 + 1}\right)$$

$$\operatorname{acosh}(z) = \ln\left(z + \sqrt{z^2 - 1}\right)$$

$$\operatorname{atanh}(z) = \frac{1}{2} \ln\left(\frac{1+z}{1-z}\right)$$

Domain: All Complex Numbers ($\mathbb{C}$).

Examples:

```
f.asinh(1 + 2i)    ## Returns approximately 1.4694 + 1.0987i
```

```
f.acosh(1 + 2i)    ## Returns approximately 1.5286 + 1.1437i
```

```
f.atanh(0.5 + 0i) ## Returns approximately 0.5493 + 0.0i
```

Exponential Functions and Logarithms

Exponential Function The `exp` function returns e raised to the power of the argument.
Morphology:

EBNF:

```
"f.exp", "(", argument, ")"
```

Example:

```
f.exp(0)    ## Returns 1.0
```

```
f.exp(1)    ## Returns approximately 2.71828
```

Complex Overload When the Argument is a Complex Number $z = a + bi$, the function is computed using Euler's Formula:

$$e^{a+bi} = e^a (\cos b + i \sin b)$$

In particular, $e^{i\pi} = -1$ (Euler's Identity) and $|e^{a+bi}| = e^a$.

Domain: All Complex Numbers ($\mathbb{C}$).

Examples:

```
f.exp(0)           #c# Returns 1.0
f.exp(1 + 3.14159i) #c# Returns approximately -2.7183 + 0.0i
f.exp(0 + 3.14159i)
    #c# Returns approximately -1 + 0.0i (Euler's Identity)
f.exp(1 + 1i)       #c# Returns approximately 1.4687 + 2.2874i
```

Square Root Function The `sqrt` function returns the square root of the argument.

Morphology:

EBNF:

```
"f.sqrt", "(", argument, ")"
```

Example:

```
f.sqrt(9)    #c# Returns 3.0
f.sqrt(2)    #c# Returns approximately 1.41421
```

Complex Overload Domain for Real Arguments: $[0, \infty)$. When the Argument is a Negative Real or Complex Number, the Principal Square Root is returned, defined such that:

$$-\pi < \arg(\sqrt{z}) \leq \pi$$

For $z = re^{i\theta}$, the Principal Square Root is $\sqrt{z} = \sqrt{r} e^{i\theta/2}$.

Domain: All Complex Numbers ($\mathbb{C}$), using the Principal Branch.

Examples:

```
f.sqrt(-1)       #c# Returns 0 + 1i
f.sqrt(-4)       #c# Returns 0 + 2i
f.sqrt(4i)        #c# Returns approximately 1.41421 + 1.41421i
f.sqrt(3 + 4i)    #c# Returns approximately 2 + 1i
f.sqrt(0)         #c# Returns 0.0
```

Power and Modular Exponentiation Function

The `pow` function computes exponentiation and optional modular exponentiation.

Morphology:

EBNF:

```
"f.pow", "(", argument, ",", argument,
    [ ",", argument ], ")"
```

With two arguments, `pow(a, b)` returns a^b . With three arguments, `pow(a, b, c)` returns $a^b \bmod c$ (modular exponentiation). Modular exponentiation is efficient for large exponents and is used in cryptographic and hardware applications.

Return type: returns `int` when both arguments are `int` and the result is exact; returns `real` otherwise (e.g., negative exponents, fractional exponents). Examples:

```
f.pow(2, 3)          ## Returns 8 (int, both args int, result exact)
f.pow(9, 0.5)        ## Returns 3.0
f.pow(2, 10)         ## Returns 1024
f.pow(3, 4, 7)       ## Returns 4 (81 mod 7)
f.pow(2, 32, 100)    ## Returns 96 (4294967296 mod 100)
f.pow(3, -1, 7)      ## Returns 5 (modular inverse: 3*5=15==1 mod 7)
f.pow(3, -2, 7)      ## Returns 4 (5^2=25==4 mod 7, i.e. 3^(-2) mod 7)
f.pow(3, -3, 7)      ## Returns 6 (5^3=125==6 mod 7, i.e. 3^(-3) mod 7)
f.pow(5, -1, 13)     ## Returns 8 (5*8=40==1 mod 13)
```

Domain restrictions: When `c` is provided, `a` and `c` must be `int` and `c` must be positive. Without `c`, `b` must be non-negative (negative exponents on `int` would produce fractions). With `c`, negative `b` computes $a^b \bmod c$ via the modular multiplicative inverse: first find $a^{-1} \bmod c$ (which exists when $\gcd(a, c) = 1$), then raise to $|b|$.

Complex Overload The function also accepts Complex Arguments. For complex z_1 and z_2 , the result is computed via:

$$z_1^{z_2} = e^{z_2 \ln z_1}$$

where $\ln$ is the Principal Branch of the Complex Logarithm.

Domain: All Complex Numbers ($\mathbb{C} \times \mathbb{C}$), using the Principal Branch of $\ln$. Note that 0^z is only defined for $\operatorname{Re}(z) > 0$.

Examples:

```
f.pow(2, 1 + 1i)     ## Returns approximately 1.538 + 0.999i
f.pow(1 + 1i, 2)     ## Returns 0 + 2i
f.pow(0 + 1i, 0+1i)  ## Returns approximately 0.2079 + 0.0i (i^i = e^{-pi/2})
```

Natural Logarithm The `ln` function returns the natural logarithm (base e) of the argument. Morphology:

EBNF:

```
"f.ln", "(", argument, ")"
```

Example:

```
f.ln(1)             ## Returns 0.0
f.ln(e)             ## Returns 1.0
```

Complex Overload Domain for Real Arguments: $(0, \infty)$. When the Argument is a Complex Number, the Complex Logarithm (Principal Branch) is returned:

$$\ln(a + bi) = \ln|a + bi| + i \arg(a + bi)$$

where $-\pi < \arg(a + bi) \leq \pi$.

Domain: All Complex Numbers except 0 ($\mathbb{C} \setminus \{0\}$).

Examples:

```
f.ln(1)          #c# Returns 0.0 + 0.0i
f.ln(e)          #c# Returns 1.0 + 0.0i
f.ln(-1)         #c# Returns approximately 0 + 3.1416i
f.ln(1 + 1i)     #c# Returns approximately 0.3466 + 0.7854i
f.ln(0 + 1i)     #c# Returns approximately 0 + 1.5708i (i*pi/2)
```

Base-10 Logarithm The log function returns the base-10 logarithm of the argument.

Morphology:

EBNF:

"f.log", "(", argument, ")"

Example:

```
f.log(100)      #c# Returns 2.0
f.log(1)        #c# Returns 0.0
```

Complex Overload Domain for Real Arguments: $(0, \infty)$. When the Argument is a Complex Number, the Complex Logarithm to base 10 is returned:

$$\log(z) = \frac{\ln(z)}{\ln(10)}$$

where ln is the Principal Branch of the Complex Logarithm.

Domain: All Complex Numbers except 0 ($\mathbb{C} \setminus \{0\}$).

Examples:

```
f.log(100)      #c# Returns 2.0 + 0.0i
f.log(1)        #c# Returns 0.0 + 0.0i
f.log(-100)     #c# Returns approximately 2 + 1.3644i
f.log(10i)      #c# Returns approximately 1 + 1.3644i
```

Complex Functions

Functions for creating and decomposing complex numbers.

Real Part The real function returns the real part of a complex number.

EBNF:

"f.real", "(", complex, ")"

```
f.real(3 + 4i)  #c# Returns 3.0
f.real(-2i)     #c# Returns 0.0
f.real(5)       #c# Returns 5.0 (int promoted)
```

Imaginary Part The `imag` function returns the imaginary part of a complex number (as a real).

EBNF:

```
"f.imag", "(", complex, ")"
```

```
f.imag(3 + 4i)    #c# Returns 4.0
f.imag(-2i)       #c# Returns -2.0
f.imag(5)         #c# Returns 0.0 (real has no imaginary part)
```

Angle The `angle` function returns the angle (argument) of a complex number in radians.

EBNF:

```
"f.angle", "(", complex, ")"
```

```
f.angle(1 + 0i)    #c# Returns 0.0
f.angle(0 + 1i)    #c# Returns 1.5708 (pi/2)
f.angle(-1 + 0i)   #c# Returns 3.14159 (pi)
f.angle(1 + 1i)    #c# Returns 0.7854 (pi/4)
```

Complex Conjugate The `conj` function returns the complex conjugate: $\text{conj}(a + bi) = a - bi$.

EBNF:

```
"f.conj", "(", complex, ")"
```

```
f.conj(3 + 4i)    #c# Returns 3 - 4i
f.conj(-2i)       #c# Returns 2i
f.conj(5)         #c# Returns 5 (real numbers are self-conjugate)
```

Polar Conversion The `polar` function converts a complex number to polar form, returning a list [magnitude, angle].

EBNF:

```
"f.polar", "(", complex, ")"
```

```
f.polar(3 + 4i)    #c# Returns [5.0, 0.9273]
f.polar(1 + 0i)    #c# Returns [1.0, 0.0]
f.polar(0 + 1i)    #c# Returns [1.0, 1.5708]
```

Rectangular Conversion The `rect` function creates a complex number from polar form (magnitude and angle in radians).

EBNF:

```
"f.rect", "(", real, ",", real, ")"
```

```
f.rect(5, 0.9273)    #c# Returns approximately 3 + 4i
f.rect(1, 0)         #c# Returns 1 + 0i
f.rect(1, 1.5708)    #c# Returns approximately 0 + 1i
```


Radians to Degrees The `rad2deg` function converts an angle from radians to degrees.

Edge Cases – Angle Conversion

- A `Type Error` is raised if the argument is not `int` or `real`.

EBNF:

```
"f.rad2deg", "(", real, ")"
```

```
f.rad2deg(3.14159)    ## Returns 180.0
f.rad2deg(1.5708)     ## Returns 90.0
f.rad2deg(0)          ## Returns 0.0
```

Degrees to Radians The `deg2rad` function converts an angle from degrees to radians.

EBNF:

```
"f.deg2rad", "(", real, ")"
```

```
f.deg2rad(180)    ## Returns 3.14159
f.deg2rad(90)     ## Returns 1.5708
f.deg2rad(0)      ## Returns 0.0
```

Rounding Functions

Edge Cases – Rounding Functions

- A `Type Error` is raised if the argument to `f.floor`, `f.ceil`, or `f.round` is not `int` or `real`.

Floor Function The `floor` function rounds the argument down to the nearest integer.

Morphology:

EBNF:

```
"f.floor", "(", argument, ")"
```

Example:

```
f.floor(2.7)    ## Returns 2
f.floor(-1.3)   ## Returns -2
```

Ceiling Function The `ceil` function rounds the argument up to the nearest integer.

Morphology:

EBNF:

```
"f.ceil", "(", argument, ")"
```

Example:

```
f.ceil(2.3)     ## Returns 3
f.ceil(-2.7)    ## Returns -2
```

Round Function The `round` function rounds the argument to the nearest value. Without `digits`, rounds to the nearest integer. With `digits`, rounds to the specified number of decimal places.

Morphology:

EBNF:

```
"f.round", "(", argument, [ ",", "digits", "=", int ], ")"
```

Examples:

```
f.round(2.5)           #c# Returns 3
f.round(2.4)           #c# Returns 2
f.round(3.14159, digits=2) #c# Returns 3.14
f.round(3.14159, digits=0) #c# Returns 3
f.round(3.14159, digits=-1) #c# Returns 0
f.round(155, digits=-2)   #c# Returns 200
```

Negative `digits` rounds to the nearest 10^n : `digits=-1` rounds to nearest 10, `digits=-2` to nearest 100, etc.

E-Series Rounding The `eseries` function rounds a value to the nearest standard E-series value. The E-series are logarithmically spaced value series used for resistors, capacitors, and inductors.

Morphology:

EBNF:

```
"f.eseries", "(", argument, ",", string, [ ",", "rm", "=", string ], ")"
```

The first argument is the value to round (must be `int` or `real`). The second argument is the series name: "E3", "E6", "E12", "E24", "E48", "E96", or "E192" (case-insensitive: "e24" is equivalent to "E24"). The optional `rm` parameter specifies the rounding mode: "nearest" (default), "floor", or "ceil". Returns the value in the selected series, scaled to the correct decade.

Edge Cases

- A **Type Error** is raised if the first argument is not `int` or `real`.
- Zero returns 0 directly (0Ω resistors are valid components).
- A **Value Error** is raised if the value is negative.
- A **Value Error** is raised if the series name is not one of the valid E-series (E3, E6, E12, E24, E48, E96, E192).

Examples:

```
f.eseries(330, "E24")           #c# Returns 330
f.eseries(350, "E24")           #c# Returns 330
f.eseries(350, "E12")           #c# Returns 330
f.eseries(350, "E6")            #c# Returns 330
```

```

f.eseries(390, "E24")           ## Returns 390
f.eseries(4.7e3, "E12")         ## Returns 4700
f.eseries(1.5e3, "E3")          ## Returns 2200
f.eseries(330, "e24")           ## Returns 330 (case-insensitive)
f.eseries(0, "E24")             ## Returns 0
f.eseries(330, "E5")            ## Value Error
f.eseries(-100, "E24")          ## Value Error
f.eseries("abc", "E24")         ## Type Error
f.eseries(330, "E24", rm="floor") ## Returns 330
f.eseries(350, "E24", rm="ceil")  ## Returns 390

```

E-Series Reference Tables All values below are the base values within one decade (1.0 to 10). Multiply by 10^n for other decades.

E3: 1.0, 2.2, 4.7

E6: 1.0, 1.5, 2.2, 3.3, 4.7, 6.8

E12	E24	E48	E96		
1.0	1.0	1.00	1.00	1.21	1.43
1.2	1.1	1.10	1.10	1.33	1.58
1.5	1.2	1.21	1.21	1.47	1.74
1.8	1.3	1.33	1.33	1.62	1.91
2.2	1.5	1.47	1.47	1.78	2.10
2.7	1.6	1.62	1.62	1.96	2.31
3.3	1.8	1.78	1.78	2.15	2.54
3.9	2.0	1.96	1.96	2.37	2.80
4.7	2.2	2.15	2.15	2.61	3.09
5.6	2.4	2.37	2.37	2.87	3.40
6.8	2.7	2.61	2.61	3.16	3.74
8.2	3.0	2.87	2.87	3.48	4.12
	3.3	3.16	3.16	3.83	4.53
	3.6	3.48	3.48	4.22	4.99
	3.9	3.83	3.83	4.64	5.49
	4.3	4.22	4.22	5.11	6.04
	4.7	4.64	4.64	5.62	6.65
	5.1	5.11	5.11	6.19	7.32
	5.6	5.62	5.62	6.81	8.06
	6.2	6.19	6.19	7.50	8.87
	6.8	6.81	6.81	8.25	9.76
	7.5	7.50	7.50		
	8.2	8.25	8.25		
	9.1	9.09	9.09		

E192 contains 192 values per decade (0.1% tolerance). The base values are: 1.00, 1.02, 1.05, 1.07, 1.10, 1.13, 1.15, 1.18, 1.21, 1.24, 1.27, 1.30, 1.33, 1.37, 1.40, 1.43, 1.47, 1.50, 1.54, 1.58, 1.62, 1.65, 1.69, 1.74, 1.78, 1.82, 1.87, 1.91, 1.96, 2.00, 2.05, 2.10, 2.15, 2.21, 2.26, 2.32, 2.37, 2.43, 2.49, 2.55, 2.61, 2.67, 2.74, 2.80, 2.87, 2.94, 3.01, 3.09, 3.16, 3.24, 3.32, 3.40, 3.48, 3.57, 3.65, 3.74, 3.83, 3.92, 4.02, 4.12, 4.22, 4.32, 4.42, 4.53, 4.64, 4.75, 4.87, 4.99, 5.11, 5.23,

5.36, 5.49, 5.62, 5.76, 5.90, 6.04, 6.19, 6.34, 6.49, 6.65, 6.81, 6.98, 7.15, 7.32, 7.50, 7.68, 7.87, 8.06, 8.25, 8.45, 8.66, 8.87, 9.09, 9.31, 9.53, 9.76.

Extrema Functions

Edge Cases – Extrema Functions

- A **Value Error** is raised if the list is empty.
- A **Type Error** is raised if the argument is not a list.

Minimum Function The `min` function returns the minimum value in a list. For `ndarrays`, an optional `axis` parameter reduces along that axis.

Morphology:

EBNF:

```
"f.min", "(", list, [ ",", "axis", "=", int ], ")"
```

Examples:

```
f.min([3, 1, 4, 1, 5])           #c# Returns 1
f.min([[3,1],[4,2]], axis=0)      #c# Returns [3, 1]
f.min([[3,1],[4,2]], axis=1)      #c# Returns [1, 2]
```

Maximum Function The `max` function returns the maximum value in a list. For `ndarrays`, an optional `axis` parameter reduces along that axis.

Morphology:

EBNF:

```
"f.max", "(", list, [ ",", "axis", "=", int ], ")"
```

Examples:

```
f.max([3, 1, 4, 1, 5])           #c# Returns 5
f.max([[3,1],[4,2]], axis=0)      #c# Returns [4, 2]
f.max([[3,1],[4,2]], axis=1)      #c# Returns [3, 4]
```

Inverse Tangent Function based on a Vector Argument

The `atan2` function computes the arctangent of y/x using the signs of both arguments to determine the correct quadrant.

Morphology:

EBNF:

```
"f.atan2", "(", argument, ",", argument, ")"
```

Example:

```
f.atan2(1, 0)    #c# Returns approximately 1.5708 (pi/2)
f.atan2(0, 1)    #c# Returns 0.0
```

Norm and Normalization Functions

EDADSL provides functions for computing vector norms and normalizing vectors.

Norm Function The `norm` function computes the p-norm of a vector (list of ints or reals).
Morphology of the Norm Function:

EBNF:

```
"f.norm", "(", argument, [ ",", "p", "=",  
    real_data_type ], ")"
```

Parameters:

- **argument** – A list of int or real values (the vector)
- **p** – (Optional) The order of the norm. Defaults to 2 (Euclidean norm).

Supported norms:

- $p = 1$ – L1 norm (Manhattan distance): $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$
- $p = 2$ – L2 norm (Euclidean distance): $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- $p > 2$ – Lp norm: $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$
- $p = 0$ – L0 "norm" (count of non-zero elements): $\|\mathbf{x}\|_0 = |\{i : x_i \neq 0\}|$
- $p = -1$ – L-infinity norm (Chebyshev distance): $\|\mathbf{x}\|_\infty = \max_i |x_i|$

Examples:

```
f.norm([3, 4])           #c# Returns 5.0 (L2 norm)  
f.norm([3, 4], p=1)      #c# Returns 7 (L1 norm)  
f.norm([3, 4], p=0)      #c# Returns 2 (L0 norm)  
f.norm([-3, 4, -2], p=-1) #c# Returns 4 (L-inf norm)
```

If the argument is not a list of ints or reals, a Value Error is raised.

Normalize Function The `normalize` function returns a unit vector in the direction of the input vector.

Morphology of the Normalize Function:

EBNF:

```
"f.normalize", "(", argument, [ ",", "p", "=",  
    real_data_type ], ")"
```

Parameters:

- **argument** – A list of int or real values (the vector to normalize)
- **p** – (Optional) The order of the norm to use for normalization. Defaults to 2.

The function computes: $\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_p}$

Examples:

```
f.normalize([3, 4])      ## Returns [0.6, 0.8] (L2)
f.normalize([3, 4], p=1)  ## Returns [0.43, 0.57] (L1)
f.normalize([6, 8], p=2)  ## Returns [0.6, 0.8] (L2)
```

If the norm of the vector is zero, a Division by Zero Error is raised.

Relationship between `abs(vec=True)` and `norm` The function `abs(v, vec=True)` is equivalent to `norm(v)` (L2 norm by default).

11.1.2 String Functions

String Conversion

The `str` function converts its arguments to a string.
Morphology:

EBNF:

```
"f.str", "(", { argument }, ")"
```

Example:

```
f.str("Hello")      ## Returns "Hello"
f.str(42)            ## Returns "42"
f.str("Value: ", 3.14)  ## Returns "Value: 3.14"
f.str()              ## Returns "" (empty string)
```

With zero arguments, `f.str()` returns an empty string.

String Formatting

The `format` function formats a value using printf-style format specifiers.
Morphology:

EBNF:

```
"f.format", "(", argument, ",", string_type, ")"
```

Format specifiers:

- `%d` – Integer decimal
- `%f` – Fixed-point real (default 6 decimal places)
- `%e` – Scientific notation
- `%g` – Shortest of `%f` or `%e`
- `%s` – String
- `%x` – Hexadecimal (integers)
- `%o` – Octal (integers)
- `%b` – Binary (integers)

Width and precision: %<width>f, %<width>.<precision>f

Examples:

```
f.format(3.14159, "%.2f")    #c# Returns "3.14"
f.format(42, "%d")          #c# Returns "42"
f.format(42, "%5d")         #c# Returns "   42"
f.format(0xff, "%x")        #c# Returns "ff"
f.format(3.14159, "%4.1f")  #c# Returns " 3.1"
f.format(1234567, "%e")     #c# Returns "1.234567e+06"
```

String Case Functions

Uppercase The `upperc` function converts a string to uppercase.

Morphology:

EBNF:

```
"f.upperc", "(", string, ")"
```

Example:

```
f.upperc("hello")    #c# Returns "HELLO"
```

Lowercase The `lowerc` function converts a string to lowercase.

Morphology:

EBNF:

```
"f.lowerc", "(", string, ")"
```

Example:

```
f.lowerc("HELLO")    #c# Returns "hello"
```

11.1.3 List and Set Functions

Concatenation

The `concat` function combines all arguments into a single list.

Morphology:

EBNF:

```
"f.concat", "(", { argument }, ")"
```

Example:

```
f.concat([1, 2], [3, 4])    #c# Returns [1, 2, 3, 4]
f.concat("a", "b", "c")    #c# Returns ["a", "b", "c"]
```

Type Conversion

List to Set The `set` function converts a list to a set. All unique elements are preserved; duplicate elements are removed and order is lost.

Morphism:

EBNF:

```
"f.set", "(", list, ")"
```

Example:

```
f.set([Q1, Q2, R5])    #c# Returns {Q1, Q2, R5}
```

Set to List The `list` function converts a set to a list. The order is determined by the lexicographic order of the canonical string representations of the elements.

Morphology:

EBNF:

```
"f.list", "(", set, ")"
```

Example:

```
f.list({Q1, Q2})    #c# Returns [Q1, Q2] (order from *e)
```

11.1.4 Nddarray Functions

Functions for operating on ndarrays (and lists). These functions extend the built-in math and list functions with array-aware behavior.

Edge Cases – Reduction Functions

- `f.sum([])` returns 0 (additive identity).
- `f.prod([])` returns 1 (multiplicative identity).
- `f.mean([])`, `f.std([])`, `f.var([])`, `f.median([])` raise a `Value Error` on empty lists.
- A `Type Error` is raised if the argument is not a list.

Reduction Functions

Reduction functions collapse an ndarray along an axis. When `axis` is omitted, the operation is applied to the flattened array (all elements). When `axis = 0`, the operation is applied row-wise (collapses rows). When `axis = 1`, the operation is applied column-wise (collapses columns).

Sum

EBNF:

```
"f.sum", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.sum([1, 2, 3])           #c# Returns 6
f.sum([[1,2],[3,4]])       #c# Returns 10 (flattened)
f.sum([[1,2],[3,4]], axis=0) #c# Returns [4, 6] (row-wise)
f.sum([[1,2],[3,4]], axis=1) #c# Returns [3, 7] (column-wise)
```

Product

EBNF:

```
"f.prod", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.prod([2, 3, 4])          #c# Returns 24
f.prod([[2,3],[4,5]], axis=0) #c# Returns [8, 15]
f.prod([[2,3],[4,5]], axis=1) #c# Returns [6, 20]
```

Mean

EBNF:

```
"f.mean", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.mean([1, 2, 3, 4])       #c# Returns 2.5
f.mean([[1,2],[3,4]], axis=0) #c# Returns [2.0, 3.0]
f.mean([[1,2],[3,4]], axis=1) #c# Returns [1.5, 3.5]
```

Standard Deviation

EBNF:

```
"f.std", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.std([1, 2, 3, 4, 5])     #c# Returns ~1.4142
f.std([1, 1, 1, 1])        #c# Returns 0.0
```

Variance Computes the population variance (divides by N , not $N - 1$).

EBNF:

```
"f.var", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.var([1, 2, 3, 4, 5])     #c# Returns 2.0
f.var([1, 1, 1, 1])        #c# Returns 0.0
```

Median

EBNF:

```
"f.median", "(", argument, ")"
```

```
f.median([3, 1, 4, 1, 5])  #c# Returns 3
f.median([1, 2, 3, 4])     #c# Returns 2.5 (average of middle two)
```

Percentile

EBNF:

```
"f.percentile", "(", argument, ",", real, ")"
```

The second argument is a percentage (0–100).

Edge Cases – Percentile

- A Value Error is raised if the list is empty.
- A Value Error is raised if the percentage is outside 0–100.

```
f.percentile([10, 20, 30, 40, 50], 50)  ## Returns 30
f.percentile([10, 20, 30, 40, 50], 0)   ## Returns 10
f.percentile([10, 20, 30, 40, 50], 100) ## Returns 50
```

Root Mean Square

EBNF:

```
"f.rms", "(", argument, ")"
```

Computes $\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$. Useful for AC signal analysis and power calculations.

```
f.rms([1, -1, 1, -1])      ## Returns 1.0
f.rms([3, 4])              ## Returns 3.5355...
f.rms([0, 0, 0])           ## Returns 0.0
```

Peak

EBNF:

```
"f.peak", "(", argument, [ ",", "mode", "=", string ], ")"
```

Modes:

- "p2p" (default) – Peak-to-peak: $\max(x) - \min(x)$
- "abs" – Maximum absolute value: $\max(|x|)$
- "pos" – Maximum positive value: $\max(x)$
- "neg" – Minimum negative value: $\min(x)$

```
f.peak([1, -3, 2, -5, 4])      ## Returns 9 (p2p: 4 - (-5))
f.peak([1, -3, 2, -5, 4], mode="abs")  ## Returns 5
f.peak([1, -3, 2, -5, 4], mode="pos")  ## Returns 4
f.peak([1, -3, 2, -5, 4], mode="neg")  ## Returns -5
```

Edge Cases – Peak

- A Value Error is raised if the list is empty.
- A Value Error is raised if an invalid mode is specified.

Search Functions

Edge Cases – Search Functions

- A Value Error is raised if the list is empty.

Index of Minimum

EBNF:

```
"f.argmin", "(", argument, ")"
```

```
f.argmin([5, 3, 1, 4])    ## Returns 2 (index of 1)
```

Index of Maximum

EBNF:

```
"f.argmax", "(", argument, ")"
```

```
f.argmax([5, 3, 1, 4])    ## Returns 0 (index of 5)
```

Shape Functions

Flatten

EBNF:

```
"f.flatten", "(", argument, ")"
```

```
f.flatten([[1,2],[3,4],[5,6]])    ## Returns [1, 2, 3, 4, 5, 6]
```

```
f.flatten([[1,2,3]])              ## Returns [1, 2, 3]
```

Sort

EBNF:

```
"f.sort", "(", argument, [ ",", lambda ], ")"
```

Returns a new sorted list (does not modify the original). An optional lambda comparator can be provided for custom ordering (see Higher-Order Functions section).

```
f.sort([5, 3, 1, 4])            ## Returns [1, 3, 4, 5]
```

```
f.sort([3.1, 2.7, 1.4])         ## Returns [1.4, 2.7, 3.1]
```

```
f.sort([5, 1, 3], L::($a [int], $b [int])->{ $b - $a })
```

```
## Returns [5, 3, 1] (descending)
```

Reverse

EBNF:

```
"f.reverse", "(", argument, ")"
```

Returns a new reversed list (does not modify the original).

```
f.reverse([1, 2, 3])            ## Returns [3, 2, 1]
```

```
f.reverse([[1,2],[3,4]])        ## Returns [[3,4],[1,2]]
```

Unique

EBNF:

```
"f.unique", "(", argument, ")"
```

```
f.unique([1, 2, 2, 3, 3, 3])    ## Returns [1, 2, 3]
```

Linear Algebra Functions

Edge Cases – Linear Algebra Functions

- A `Type Error` is raised if the argument is not an `ndarray` (matrix).
- A `Value Error` is raised for `f.det`, `f.inv`, `f.eig`, `f.eigvec`, `f.lu`, `f.qr`, `f.trace`, `f.cond` if the matrix is not square.
- A `Value Error` is raised for `f.cond` if the matrix is singular.
- A `Value Error` is raised for `f.normalize` if the vector has zero magnitude.

Determinant

EBNF:

```
"f.det", "(", argument, ")"
```

```
f.det([[1,2],[3,4]])    ## Returns -2.0
```

```
f.det([[2,0],[0,3]])    ## Returns 6.0
```

Matrix Inverse

EBNF:

```
"f.inv", "(", argument, ")"
```

Returns the inverse matrix as a list of lists. A `Division by Zero Error` is raised if the matrix is singular.

```
f.inv([[1,2],[3,4]])    ## Returns [[-2.0, 1.0],[1.5, -0.5]]
```

Eigenvalues

EBNF:

```
"f.eig", "(", argument, ")"
```

Returns a list of eigenvalues.

```
f.eig([[2,0],[0,3]])    ## Returns [2, 3]
```

```
f.eig([[1,2],[2,1]])    ## Returns [3, -1]
```

Eigenvectors

EBNF:

```
"f.eigvec", "(", argument, ")"
```

Returns a list of eigenvectors (one per eigenvalue, as rows).

```
f.eigvec([[2,0],[0,3]])    ## Returns [[1,0],[0,1]]
```

Singular Value Decomposition

EBNF:

```
"f.svd", "(", argument, ")"
```

Returns a list of 3 lists: [U, S, V] where U and V are orthogonal matrices and S is the list of singular values.

```
f.svd([[1,0],[0,2]])  
#c# Returns [[[1,0],[0,1]], [1, 2], [[1,0],[0,1]]]
```

LU Decomposition

EBNF:

```
"f.lu", "(", argument, ")"
```

Returns a list of 2 lists: [L, U] where L is lower triangular and U is upper triangular.

```
f.lu([[2,1],[4,3]]) #c# Returns L and U such that L .* U = original
```

QR Decomposition

EBNF:

```
"f.qr", "(", argument, ")"
```

Returns a list of 2 lists: [Q, R] where Q is orthogonal and R is upper triangular.

```
f.qr([[1,1],[1,-1]]) #c# Returns Q and R such that Q .* R = original
```

Cholesky Decomposition

EBNF:

```
"f.chol", "(", argument, ")"
```

Returns the lower triangular matrix L such that L .* L.T = original.

Edge Cases

- A **Type Error** is raised if the argument is not a list of lists (ndarray).
- A **Value Error** is raised if the matrix is not square.
- A **Value Error** is raised if the matrix is not symmetric.
- A **Value Error** is raised if the matrix is not positive definite.

```
f.chol([[4,2],[2,3]]) #c# Returns [[2,0],[1,1.7321]]  
f.chol([[1,2],[3,4]]) #c# Value Error (not symmetric)  
f.chol([[1,2,3]]) #c# Value Error (not square)
```

Trace

EBNF:

```
"f.trace", "(", argument, ")"
```

```
f.trace([[1,2],[3,4]])    ## Returns 5
```

```
f.trace([[2,0],[0,3]])    ## Returns 5
```

Matrix Rank

EBNF:

```
"f.rank", "(", argument, ")"
```

```
f.rank([[1,2],[3,4]])    ## Returns 2
```

```
f.rank([[1,2],[2,4]])    ## Returns 1 (rows are linearly dependent)
```

Condition Number

EBNF:

```
"f.cond", "(", argument, ")"
```

Returns the ratio of the largest to smallest singular value. A high condition number indicates numerical instability.

```
f.cond([[1,0],[0,1]])    ## Returns 1.0 (well-conditioned)
```

```
f.cond([[1,0],[0,1e-10]]) ## Returns 1e10 (ill-conditioned)
```

Pseudo-Inverse

EBNF:

```
"f.pinv", "(", argument, ")"
```

Returns the Moore–Penrose pseudo-inverse. Works for non-square or singular matrices.

```
f.pinv([[1,2],[3,4]])    ## Returns [[-2, 1],[1.5, -0.5]]
```

Statistical Functions

Histogram

EBNF:

```
"f.histogram", "(", argument, ",", int, ")"
```

Returns a list of 2 lists: [edges, counts] where `edges` has `n + 1` values and `counts` has `n` values.

Edge Cases – Histogram

- A `Type Error` is raised if the first argument is not a list.
- A `Value Error` is raised if `n <= 0`.

```
f.histogram([1, 2, 3, 4, 5], 3)
```

```
## Returns [[1.0, 2.333, 3.667, 5.0], [2, 1, 2]]
```

11.1.5 Higher-Order Functions

Higher-order functions take lambda expressions as arguments. They operate on lists and ndarrays. See Section 8.1.10 for lambda syntax.

Edge Cases – Higher-Order Functions

- A **Type Error** is raised if the second argument is not a lambda.
- A **Type Error** is raised if the first argument is not a list.
- `f.reduce` raises a **Value Error** if the list is empty and no initial value is provided (third argument).

Map

The `map` function applies a lambda to each element of a list and returns a new list with the results. The input list is not modified.

Morphology

EBNF:

```
"f.map", "(", list, ",", lambda, ")"
```

Type Overloading

- `map(list[T], L::($x [T])->{...}) → list[U]` (T and U can be different types)
- Works on any list: flat, nested, mixed types
- The lambda parameter type must match the element type

Examples

```
f.map([1, 2, 3], L::($x [int])->{ $x * 2 })  
## Returns [2, 4, 6]
```

```
f.map([1, 2, 3], L::($x [int])->{ $x > 2 })  
## Returns [False, False, True] (type change: int -> bool)
```

```
f.map(["a", "bb", "ccc"], L::($x [string])->{ f.len($x) })  
## Returns [1, 2, 3] (type change: string -> int)
```

```
f.map([1, 2, 3], L::($x [int])->{  
    $y [int]= $x * 10  
    $y + 1  
})  
## Returns [11, 21, 31] (multi-line lambda)
```

```
f.map([], L::($x [int])->{ $x * 2 })  
## Returns [] (empty list)
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error

Filter

The `filter` function returns a new list containing only elements for which the lambda returns `True`. The input list is not modified.

Morphology

EBNF:

```
"f.filter", "(", list, ",", lambda, ")"
```

Type Overloading

- `filter(list[T], L::($x [T])->{bool}) → list[T]` (return type is the same as input)
- The lambda must return `bool`

Examples

```
f.filter([1, 2, 3, 4, 5], L::($x [int])->{ $x % 2 == 0 })  
## Returns [2, 4]
```

```
f.filter([1, 2, 3, 4, 5], L::($x [int])->{ $x > 10 })  
## Returns [] (no matches)
```

```
f.filter(["a", "bb", "ccc", "dd"], L::($x [string])->{ f.len($x) > 2 })  
## Returns ["ccc"]
```

```
f.filter([-1, -2, 3, -4, 5], L::($x [int])->{ $x > 0 })  
## Returns [3, 5]
```

```
f.filter([1, 2, 3], L::($x [int])->{ $x > 5 })  
## Returns [] (empty result)
```

```
f.filter([], L::($x [int])->{ $x > 0 })  
## Returns [] (empty input)
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return `bool` raises a Type Error

Reduce

The `reduce` function folds a list to a single value by applying a binary lambda cumulatively. The lambda receives an accumulator and the current element. The `init` value is always required.

Morphology

EBNF:

```
"f.reduce", "(", list, ",", lambda, ",", expression, ")"
```

Type Overloading

- `reduce(list[T], L::($acc [U], $x [T])->{U}, init[U]) → U` (accumulator type U can differ from element type T)
- The `init` value type must match the accumulator type

Examples

```
#c# Sum integers
```

```
f.reduce([1, 2, 3, 4], L::($acc [int], $x [int])->{ $acc + $x }, 0)
```

```
#c# Returns 10
```

```
#c# Product integers
```

```
f.reduce([2, 3, 4], L::($acc [int], $x [int])->{ $acc * $x }, 1)
```

```
#c# Returns 24
```

```
#c# Find maximum
```

```
f.reduce([3, 1, 4, 1, 5], L::($acc [int], $x [int])->{  
    ($acc > $x)?->($acc)!($x)  
}, 0)
```

```
#c# Returns 5
```

```
#c# Reduce with string concatenation
```

```
f.reduce(["a", "b", "c"],  
    L::($acc [string], $x [string])->{ $acc + $x }, "")
```

```
#c# Returns "abc"
```

```
#c# Reduce on empty list returns init
```

```
f.reduce([], L::($acc [int], $x [int])->{ $acc + $x }, 42)
```

```
#c# Returns 42
```

```
#c# Flatten with reduce (using concat)
```

```
f.reduce([[1,2],[3,4]],  
    L::($acc [list], $x [list])->{ f.concat($acc, $x) }, [])
```

```
#c# Returns [1, 2, 3, 4]
```

Error Conditions

- Type mismatch: lambda parameter types do not match accumulator/element types raises a Runtime Error
- Lambda does not return a type coercible to the accumulator type raises a Type Error

Where

The **where** function returns a list of **indices** (ints) for which the lambda returns **True**. Indices are 0-based.

Morphology

EBNF:

```
"f.where", "(", list, ",", lambda, ")"
```

Type Overloading

- `where(list[T], L::($x [T])->{bool}) → list[int]`
- The lambda must return `bool`

Examples

```
f.where([-1, 3, -2, 5], L::($x [int])->{ $x > 0 })  
## Returns [1, 3]
```

```
f.where([10, 20, 30], L::($x [int])->{ $x >= 20 })  
## Returns [1, 2]
```

```
f.where([1, 2, 3], L::($x [int])->{ $x > 10 })  
## Returns [] (no matches)
```

```
f.where(["a", "bb", "ccc"], L::($x [string])->{ f.len($x) >= 2 })  
## Returns [1, 2]
```

```
f.where([], L::($x [int])->{ $x > 0 })  
## Returns [] (empty input)
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return `bool` raises a Type Error

Count

The **count** function returns the number of elements for which the lambda returns **True**.

Morphology

EBNF:

```
"f.count", "(", list, ",", lambda, ")"
```

Type Overloading

- `count(list[T], L::($x [T])->{bool}) → int`
- The lambda must return bool

Examples

```
f.count([1, 2, 3, 4, 5], L::($x [int])->{ $x > 3 })  
## Returns 2
```

```
f.count([True, False, True, True], L::($x [bool])->{ $x })  
## Returns 3
```

```
f.count([1, 2, 3], L::($x [int])->{ $x > 10 })  
## Returns 0
```

```
f.count([], L::($x [int])->{ $x > 0 })  
## Returns 0
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return bool raises a Type Error

Find

The `find` function returns the first element for which the lambda returns `True`. If no element matches, a Value Error is raised.

Morphology

EBNF:

```
"f.find", "(", list, ",", lambda, ")"
```

Type Overloading

- `find(list[T], L::($x [T])->{bool}) → T` (return type is the same as element type)
- The lambda must return bool

Examples

```
f.find([1, 2, 3, 4, 5], L::($x [int])->{ $x > 2 })  
## Returns 3
```

```
f.find([1, 2, 3, 4, 5], L::($x [int])->{ $x == 4 })  
## Returns 4
```

```
f.find(["a", "bb", "ccc"], L::($x [string])->{ f.len($x) > 1 })  
## Returns "bb"
```

```
f.find([1, 2, 3, 4, 5], L::($x [int])->{ $x > 10 })  
## Value Error: no matching element
```

```
f.find([], L::($x [int])->{ $x > 0 })  
## Value Error: no matching element
```

Error Conditions

- Value Error: no element matches the lambda condition
- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return bool raises a Type Error

Sort with Comparator

The `sort` function returns a new sorted list. Without a comparator, default sort order is ascending. An optional lambda comparator can be provided for custom ordering (see Sort paragraph above for EBNF).

Comparator Lambda The comparator lambda receives two elements (`$a` and `$b`) and must return a numeric value:

- **Negative** value (e.g., `$a - $b`): `$a` comes before `$b`
- **Zero**: order unchanged (stable sort)
- **Positive** value (e.g., `$b - $a`): `$b` comes before `$a`

This is equivalent to the `cmp` convention used in many languages. The simplest ascending comparator is `$a - $b`, and descending is `$b - $a`.

Type Overloading

- `sort(list[T]) → list[T]` (default ascending)
- `sort(list[T], L::($a [T], $b [T])->{int}) → list[T]` (custom comparator, must return int or real)

Examples

```
#c# Default sort (ascending)
f.sort([5, 1, 3])
#c# Returns [1, 3, 5]

#c# Ascending with comparator
f.sort([5, 1, 3], L::($a [int], $b [int])->{ $a - $b })
#c# Returns [1, 3, 5]

#c# Descending with comparator
f.sort([5, 1, 3], L::($a [int], $b [int])->{ $b - $a })
#c# Returns [5, 3, 1]

#c# Sort strings by length
f.sort(["bb", "a", "ccc"],
      L::($a [string], $b [string])->{ f.len($a) - f.len($b) })
#c# Returns ["a", "bb", "ccc"]

#c# Sort reals ascending
f.sort([3.14, 1.41, 2.72], L::($a [real], $b [real])->{ $a - $b })
#c# Returns [1.41, 2.72, 3.14]

#c# Sort with nested sort key (sort by absolute value)
f.sort([-5, 3, -1, 4], L::($a [int], $b [int])->{ f.abs($a) - f.abs($b) })
#c# Returns [-1, 3, 4, -5]

#c# Sort preserves original (returns new list)
$list [list]= [5, 3, 1]
$sorted [list]= f.sort($list)
#c# $list is still [5, 3, 1], $sorted is [1, 3, 5]
```

Error Conditions

- Type mismatch: comparator parameter types do not match element types raises a Runtime Error
- Comparator does not return a numeric type raises a Type Error

11.1.6 Signal Processing Functions

Functions for frequency-domain analysis of signals.

Fast Fourier Transform

The `fft` function computes the Discrete Fourier Transform of a real-valued signal.

EBNF:

```
"f.fft", "(", list, ")"
```

Type Overloading

- `fft(list[int]) → list[complex]`
- `fft(list[real]) → list[complex]`
- Output length equals input length
- The input signal is zero-padded to the next power of 2 if needed

Examples

```
#c# FFT of a simple signal
f.fft([1, 0, 1, 0])    #c# Returns [2+0i, 0+0i, 2+0i, 0+0i]

#c# FFT of a single impulse
f.fft([1, 0, 0, 0])    #c# Returns [1+0i, 1+0i, 1+0i, 1+0i]

#c# FFT of a DC signal (constant value)
f.fft([5, 5, 5, 5])    #c# Returns [20+0i, 0+0i, 0+0i, 0+0i]

#c# Extract magnitude spectrum
$freq [list]= f.fft([1, 0, 1, 0])
$mags [list]= f.map($freq, L::($z [complex])->{ f.abs($z) })
#c# $mags = [2.0, 0.0, 2.0, 0.0]
```

Error Conditions

- Empty list raises a Value Error

Inverse Fast Fourier Transform

The `ifft` function computes the Inverse Discrete Fourier Transform, reconstructing a real signal from complex frequency components.

EBNF:

```
"f.ifft", "(", list, ")"
```

Type Overloading

- `ifft(list[complex]) → list[real]`
- Output length equals input length
- Imaginary parts are discarded (reconstructed signal is real)

Examples

```
#c# Round-trip: fft then ifft
$signal [list]= [1, 0, 1, 0]
$freq [list]= f.fft($signal)
$reconstructed [list]= f.ifft($freq)
#c# $reconstructed is approximately [1, 0, 1, 0]

#c# ifft of DC component
f.ifft([4+0i, 0+0i, 0+0i, 0+0i])    #c# Returns [1, 1, 1, 1]

#c# ifft of impulse in frequency domain
f.ifft([1+0i, 1+0i, 1+0i, 1+0i])    #c# Returns [1, 0, 0, 0]
```

Error Conditions

- Empty list raises a Value Error

11.1.7 Shared Functions

Length/Size Function

The `len` function returns the length of a list or string, or the size of a set.
Morphology:

EBNF:

```
"f.len", "(", ( list | string | set ), ")"
```

Edge Cases – Collection Utilities

- A `Type Error` is raised if the argument to `f.len` is not a list, string, or set.
- A `Type Error` is raised if the arguments to `f.concat` are not all lists or all strings.
- A `Type Error` is raised if the argument to `f.set` is not a list.
- A `Type Error` is raised if the argument to `f.list` is not a set.

Examples:

```
f.len([1, 2, 3])    #c# Returns 3
f.len("hello")      #c# Returns 5
f.len({Q1, Q2, R5}) #c# Returns 3
```

11.1.8 Special Functions

Print Function

The `f.print()` function outputs its argument to the console. Unlike `echo`, it can evaluate expressions and function calls directly.
Morphology:

EBNF:

```
"f.print", "(", argument, ")"
```

Example:

```
f.print("Hello, World!")  
f.print(3 + 2 * 5)    #c# Outputs: 13
```

Type Function

The **type** function returns the type of an object as a string.

Morphology:

EBNF:

```
"f.type", "(", argument, ")"
```

Example:

```
f.type(42)           #c# Returns "int"  
f.type(3.14)         #c# Returns "real"  
f.type("hello")      #c# Returns "string"  
f.type(Q1)           #c# Returns "part"
```

Reference Function

The **ref** function converts a string containing a part reference into the referenced part.

Morphology:

EBNF:

```
"f.ref", "(", string, ")"
```

Example:

```
f.ref("R1")          #c# Returns the part referenced as R1
```

No-Operation Function

The **noop** function is the identity function. It returns its argument unchanged.

Morphology:

EBNF:

```
"f.noop", "(", argument, ")"
```

Example:

```
f.noop(42)           #c# Returns 42  
f.noop("hello")      #c# Returns "hello"  
f.noop(Q1)           #c# Returns Q1
```

Use **noop** as a placeholder in queries where no transformation is needed.

Soft Integer Conversion

The `softint` function converts its argument to an integer if it is either already an integer or is a real number numerically equal to an integer. Otherwise it returns the argument unchanged.

Morphology:

EBNF:

```
"f.softint", "(", argument, ")"
```

Examples:

```
f.softint(42)          #c# Returns 42 (already int)
f.softint(5.0)         #c# Returns 5 (real equal to int)
f.softint(3.7)         #c# Returns 3.7 (not equal to int, unchanged)
f.softint("hello")    #c# Returns "hello" (not numeric, unchanged)
```

Soft Real Conversion

The `softreal` function converts its argument to a real if it is either already a real, an integer, or a complex number with zero imaginary part. Otherwise it returns the argument unchanged.

Morphology:

EBNF:

```
"f.softreal", "(", argument, ")"
```

Examples:

```
f.softreal(42)         #c# Returns 42.0 (int promoted to real)
f.softreal(3.14)       #c# Returns 3.14 (already real)
f.softreal(5 + 0i)     #c# Returns 5.0 (complex with zero imag part)
f.softreal(3 + 4i)
    #c# Returns 3 + 4i (complex with non-zero imag, unchanged)
f.softreal("hello")   #c# Returns "hello" (not numeric, unchanged)
```

11.2 User-defined Functions

User-defined functions allow creation of reusable code blocks with local scoping.

11.2.1 Function Definition

Functions are defined using the `function` keyword with the `f.u.` prefix:

EBNF:

```
function_definition = "function", "f.u.", id,
    "(", [ { id, ",", id }, id ], ")", ":", "{",
    f_statement, {f_statement}, "}"
```

```
f_statement = statement | "return", [data]
```

Example:

```
function f.u.add( $a, $b )::{  
    $retval [int]= $a + $b  
}
```

Function parameters are local variables scoped to the function body. Parameters may optionally include type annotations using the `[type]` syntax, consistent with lambda parameters. To return a value, set the `$retval` variable.

11.2.2 Function Calls

User-defined functions are called using the `f.u.` prefix:

EBNF:

```
function_call_ud = "f.u.", id, "(",  
                    [{ data, "," }, data], ")"
```

Example:

```
$result [int]= f.u.add( 3, 5 )    #c# $result = 8
```

11.2.3 Return Values

Functions return values by setting the `$retval` variable. A bare `return` without a value, or a function that never sets `$retval`, returns `None`.

```
function f.u.square( $x )::{  
    $retval [int]= $x ^ 2  
}
```

```
function f.u.greet( $name )::{  
    f.print( f.str( "Hello, ", $name, "!" ) )  
    #c# No return value  
}
```

```
$sq [int]= f.u.square( 4 )    #c# $sq = 16  
f.u.greet( "World" )         #c# Prints: Hello, World!
```

11.2.4 Local Scoping

Functions have local scoping. Variables declared inside a function are only accessible within that function. Parameters are also local variables.

Scope Rules

- Parameters (`$a`, `$b`, etc.) are local to the function body
- Variables declared inside the function with `$var [type]= val` are local
- Local variables shadow global variables of the same name

- The `$retval` variable is always local to the function
- Functions cannot access local variables from other function invocations

Return Values Functions can return values using two equivalent mechanisms:

- Assign to `$retval` and let execution fall through the closing `}`
- Use the `return` keyword (see Chapter 3, `f_statement` rule), which is syntactic sugar that assigns its argument to `$retval` and immediately exits the function

Both `return $value` and `$retval [type]= $value` followed by natural function exit are valid and equivalent. If both are used, `return` takes precedence.

Examples

```
$x [int]= 10      #c# Global variable

function f.u.doublen()::{
    $x [int]= $x * 2  #c# Local $x shadows global
    f.print($x)      #c# Prints: 20
}

f.u.doublen()
f.print($x)         #c# Prints: 10 (global unchanged)

function f.u.addLocal( $a, $b )::{
    $sum [int]= $a + $b  #c# $sum is local
    $retval [int]= $sum
}

f.u.addLocal( 3, 4 )  #c# Returns 7
#c# $sum is not accessible here
```

Recursion Local scoping enables recursion since each call has its own copy of parameters:

```
function f.u.factorial( $n )::{
    if $n <= 1 ::{
        $retval [int]= 1
    } else ::{
        $retval [int]= $n *
            f.u.factorial( $n - 1 )
    }
}

$result [int]= f.u.factorial( 5 )  #c# = 120
```

Part II

Formal Semantics

Chapter 12

Evaluation Model

Expressions in EDADSL are evaluated according to standard mathematical precedence rules. The evaluation model is eager (strict), meaning all sub-expressions are evaluated before the operator is applied, with the following exceptions:

- `&&` and `||`: Short-circuit evaluation. `a && b` evaluates `b` only if `a` is `True`. `a || b` evaluates `b` only if `a` is `False`.
- `?->`: Conditional expressions only evaluate the taken branch.
- `*`: If either operand evaluates to 0 (int or real), the result is 0 and the other operand is not evaluated.

12.1 Evaluation Order

Within an expression, operands are evaluated left-to-right. Function arguments are evaluated left-to-right before the function is called.

12.2 Operator Precedence

The complete operator precedence is defined in Section 8.

12.3 Type Coercion

Type coercion follows the hierarchy: `int` < `real` < `complex`.

- `int` coerces to `real`: 5 becomes 5.0
- `real` coerces to `complex`: 3.14 becomes 3.14 + 0i
- `int` coerces to `complex`: 5 becomes 5 + 0i
- `bool` coerces to `int`: `True` becomes 1
- `bool` coerces to `real`: `True` becomes 1.0
- `bool` coerces to `complex`: `True` becomes 1 + 0i

- `bool` coerces to `string`: `True` becomes `"true"`, `False` becomes `"false"`
- Mixed numeric operations promote to the wider type: `int + real → real`, `real + complex → complex`
- Coercion is never lossy in the upward direction
- Downward coercion (`complex → real`, `real → int`) raises a `Type Error` unless done explicitly via a casting function (e.g., `f.int()`, `f.real()`)

Chapter 13

Execution Order

13.1 Sequential Execution

Program statements are executed sequentially from the entry point (**start**). Electrical statements (**elec.part**, **elec.connect**, **elec.net**) execute immediately and register components to the electrical graph. Physical constraints (**phys.***) execute immediately but their resolution is deferred until after program execution completes.

13.2 Expression Evaluation

Within a single expression, operands are evaluated left-to-right. Function arguments are evaluated left-to-right before the function is called. Short-circuit operators (**&&**, **||**) may skip evaluation of the right operand when the result is determined.

13.3 Declaration Order

Variables must be declared before they are accessed. Reading a variable before its declaration statement has been executed raises a **Runtime Error**. Declarations are processed in order during execution (there is no separate declaration phase). Variable types may change on redeclaration.

Chapter 14

Constraint Resolution

Physical constraints are resolved by the solver according to the priority system defined in Section 22.

14.1 Conflict Resolution

When constraints conflict, the solver applies the higher-priority constraint and optimizes the lower-priority one. See Section 23 for the complete resolution rules.

Chapter 15

Type System Rules

EDADSL uses a dynamic type system with distinct types for electrical and non-electrical objects.

15.1 Type Categories

- Generic types: Boolean, Integer, Real, Complex, String, List, Set
- Electrical types: Part, Pin, Connection, Net

15.2 Type Safety

EDADSL performs type checking at both compile time and runtime:

- **Compile-time:** Type mismatches in variable declarations (e.g., `$x [int]= "hello"`) are detected when the script is parsed.
- **Runtime:** Type mismatches in function arguments and operator usage are detected when the code is executed. A **Type Error** is raised for incompatible types.
- **Sets:** Sets can contain any data type. Mixed-type sets are allowed.
- **Coercion:** Numeric types coerce upward: `int` $\rightarrow$ `real` $\rightarrow$ `complex`. Other types do not coerce.

Part III

Core Formal Model

Chapter 16

Structure of the Program

16.1 Program Entry Point

Every EDADSL program must begin with the **start** keyword. The program executes sequentially from the entry point.

16.2 Global Scope

Variables and constants declared at the top level of the program have global scope and are visible from their declaration onward. Variables declared inside code blocks (**if**, **for**, **while**, **do-while**) are local to that block. Variables declared inside functions are local to that function (see Section 11.2.4).

16.3 Execution Model

EDADSL uses a single-pass execution model. There is no separate declaration phase — declarations are processed in order during execution (see Appendix for execution-phase diagram). Variables must be declared before they are accessed.

Electrical statements (**elec.part**, **elec.connect**, **elec.net**) are executed immediately and register components and nets to the electrical graph. Physical constraints (**phys.***) are executed immediately but their resolution is deferred until after program execution completes.

The solver interacts with program execution as follows:

1. Program executes sequentially, registering all components, nets, and constraints
2. After program execution completes, the constraint solver runs
3. The solver uses CSP propagation and simulated annealing to determine optimal placement and routing
4. Output files are generated from the solved state

Chapter 17

Structure of the Electrical System

17.1 Electrical Graph

The electrical system is modeled as an undirected graph where:

- Vertices represent Nets
- Edges represent Connections
- Parts attach to the graph via their Pins

17.2 Object Lifecycle

Electrical objects (parts, connections, pins, nets) are committed to the graph immediately when their declaration statement is executed. The graph is modified incrementally as the program runs.

- `elec.part` creates a Part node and attaches Pins
- `elec.net` creates a Net node
- `elec.connect` creates Connection edges between Pins and Nets
- `elec.tag` / `elec.untag` modify Net metadata

There are no undo/redo semantics. Once an object is committed to the graph, it cannot be removed. The graph state at the end of program execution is the final state passed to the constraint solver.

Chapter 18

Structure of the Solver

18.1 Constraint Representation

Physical constraints are represented as optimization objectives with priorities.

18.2 Resolution Algorithm

The constraint solver uses a two-phase approach:

1. **CSP Propagation:** Hard constraints (type matching, connectivity) are enforced first. This reduces the search space by eliminating invalid configurations.
2. **Simulated Annealing:** Soft constraints (proximity, alignment, length matching) are optimized using simulated annealing. Constraints are weighted by priority level:
 - `-d` (default) — weight 1.0
 - `-nth` — weight 0.5
 - `-imp` — weight 2.0
 - `-vimp` — weight 4.0
 - `-oeo` — weight 3.0 (override exact order)
 - `-eo` — weight 1.5 (exact order)
 - `-ne` — weight 0.3 (non-exact)

Convergence criteria: The solver terminates when (a) all hard constraints are satisfied and no soft constraint has improved by more than 0.1% over the last 1000 iterations, or (b) the maximum iteration count (default 10000) is reached.

Part IV

Hardware Model

Chapter 19

Circuit Model

The circuit model represents the electrical system as a graph of interconnected objects.

19.1 Graph Structure

The circuit is represented as a graph where:

- Nodes are Nets (electrical connection points)
- Edges are Connections (physical wires/traces between pins)
- Parts are components that connect to the graph via Pins

19.2 Hierarchy

The model supports hierarchical grouping:

- Parts contain Pins
- Pins connect to Nets via Connections
- Nets can be grouped by Tags

Chapter 20

Electrical Objects

The electrical system consists of five primary object types.

20.1 Nets

A Net is a named electrical node that connects multiple Pins and Connections. Nets are the fundamental grouping mechanism for shared electrical connections.

20.1.1 Creation

Nets are created using the `elec.net` statement:

```
elec.net <name> [<tag1> ... <tag n>]
```

Example:

```
elec.net "Gnd"
```

```
elec.net "hvbus" "hv" "high_i" "noisy"
```

If two `elec.net` statements use the same name, they are merged into a single net. Tags from both declarations are combined, and a warning is issued.

20.1.2 Properties

- Name (unique identifier)
- Connected objects (Pins and Connections)
- Tags (user-defined labels)

20.2 Parts

A Part represents a physical electronic component in the circuit.

20.2.1 Creation

Parts are created using the `elec.part` statement:

```
elec.part <reference>: <model>(<args>);  
    des="<description>"; fp="<footprint>"
```

Example:

```
elec.part "R3": R(R=220, P=0.25, type="carbon");  
    des="220 ohm resistor";  
    fp="Resistor_THT:R_Axial_DIN0309"
```

20.2.2 PCB Text Visibility Flags

Per-component PCB text visibility can be overridden in the property list:

```
elec.part "R3": R(R=220); des="220R"; fp="R_Axial_DIN0309";  
    show_ref=True; show_value=False; show_tol=True; show_mpn=False
```

Available flags: `show_ref`, `show_value`, `show_tol`, `show_mpn`. These override the global `syst.flag.show_pcb_part_*` flags (see Chapter 24).

20.2.3 Properties

- Reference designator (e.g., R3, Q1, U1)
- Model and parameters
- Description
- Footprint
- Pins (derived from model/footprint)

20.3 Pins

A Pin is a connection point on a Part. Pins are accessed using the bracket notation.

20.3.1 Creation

Pins are created implicitly when Parts are instantiated.

20.3.2 Reference Syntax

`<part_ref>[<pin_ref>]`

Examples:

```
R1[1]    #c# Pin 1 of resistor R1  
Q1[d]    #c# Drain pin of MOSFET Q1  
U1[v+]   #c# Non-inverting input of op-amp U1
```

20.4 Connections

A Connection is a binary electrical connection between exactly two Pins.

20.4.1 Creation

Connections can be created using the pipe syntax or `elec.connect`:

```
#c# Pipe syntax (creates a Connection object):  
( R2[1] | J4[7] )
```

```
#c# Connect statement (joins pins to a Net):  
elec.connect <net> <pin1> <pin2> ... <pin n>
```

Example:

```
elec.connect Gnd Q2[s] C6[-] R3[2]
```

20.5 Models

Models define the electrical characteristics and available pins of a Part type.

20.5.1 Model Declaration Syntax

Custom models are defined using `elec.makemodel`:

EBNF:

```
model_declaration = "elec.makemodel", model_name, ":",  
                    "{", pin_entries, [metadata], "}"
```

```
model_name        = id, ["/", id]
```

```
pin_entry          = pin_name, "->", pin_id, ";",  
                    "T", "=", pin_type
```

```
pin_name           = id
```

```
pin_id             = integer
```

```
pin_type           = "input" | "power" | "output"
```

```
metadata           = des_entry | dts_entry | fp_entry
```

```
des_entry          = "des", "=", string_type
```

```
dts_entry          = "dts", "=", string_type
```

```
fp_entry           = "fp", "=", string_type
```

The `model_name` consists of a category and variant separated by / (e.g., `mosfet/C3M0016120K1`).

Pin types classify the electrical role:

- `input` – Signal input (gate, base, control)

- **power** – Power connection (source, drain, emitter, collector)
- **output** – Signal output (drain for sense, kelvin connections)

20.5.2 Built-in Models

- **R** – Resistor. Params: **R** (resistance), **P** (power rating), **tol** (tolerance), **type** (carbon, metal film, etc.)
- **C** – Capacitor. Params: **C** (capacitance), **V** (voltage rating), **type** (ceramic, electrolytic, etc.)
- **L** – Inductor. Params: **L** (inductance), **core** (core material)
- **opamp** – Operational Amplifier. Params: **type** (generic, LM741, etc.), **GBP** (gain-bandwidth product), **slew** (slew rate), **Vos** (offset voltage)
- **MOSFET** – MOSFET transistor. Params: **type** (N/P), **Vds** (drain-source voltage), **Id** (drain current), **Rds** (on-resistance), **Vgs** (gate threshold)
- **LED** – Light Emitting Diode. Params: **color** (red, green, blue, etc.), **Vf** (forward voltage), **If** (forward current)
- **D** – Diode. Params: **type** (signal, rectifier, zener), **Vf** (forward voltage), **If** (forward current), **Vr** (reverse voltage)
- **Q** – BJT Transistor. Params: **type** (NPN/PNP), **Vce** (collector-emitter voltage), **Ic** (collector current), **hfe** (DC gain)
- **X** – Crystal/Oscillator. Params: **freq** (frequency), **load** (load capacitance), **ESR** (series resistance)
- **J** – Connector. Params: **pins** (pin count), **type** (header, barrel, etc.), **pitch** (pin spacing)
- **F** – Fuse. Params: **I** (current rating), **V** (voltage rating), **type** (glass, blade, etc.)
- **Y** – Transformer. Params: **ratio** (turns ratio), **Vpri** (primary voltage), **Vsec** (secondary voltage), **power** (VA rating)

20.5.3 Custom Model Examples

MOSFET Model

```
elec.makemodel MOSFET/IRLZ44N:{
    g -> 1; T = input
    S -> 2; T = power
    d -> 3; T = power
}
```

Non-Polarized Capacitor

```
elec.makemodel nonpol_C:{  
    1 -> 1; T = power  
    2 -> 2; T = power  
}
```

SiC MOSFET with Metadata

```
elec.makemodel mosfet/C3M0016120K1:{  
pins:  
    g      -> 4; T = input  
    s      -> 2; T = power  
    d      -> 1; T = power  
    kelvin_s -> 3; T = output  
  
des = "1200 V, 125A, 16 mOhm, TO-247-4"  
  
dts = "https://assets.wolfspeed.com/..."  
  
fp  = "Package_TO_SOT_THT/TO-247-4_Vert"  
}
```

The `des` (description), `dts` (datasheet URL), and `fp` (footprint path) are optional metadata entries.

20.6 Electrical Modules

Electrical modules encapsulate reusable subcircuits with configurable parameters. Modules can contain parts, connections, nets, physical constraints, and nested modules.

20.6.1 Definition

Modules are defined using `elec.makemodule`:

```
elec.makemodule <name>(<$param1>, <$param2>, ...) {  
    pin <name> <direction>  
  
    <full program syntax>  
}
```

- Parameters are declared after the module name as dollar-prefix variables.
- Exposed pins are declared with `pin <name> <direction>` (input, output, power).
- The body uses full program syntax: `elec.part`, `elec.connect`, `elec.net`, `phys.*`, etc.
- Parameters are substituted into the body at instantiation time.

20.6.2 Instantiation

Modules are instantiated via `elec.part` using the module name as the model:

```
elec.part "<reference>": <module_name>(<params>)
```

Only the module name and parameter values are provided. Internal parts, connections, and constraints are defined inside the module.

20.6.3 Example

Definition:

```
elec.makemodule LowPassFilter($R, $C) {  
    pin in input  
    pin out output  
  
    elec.part "R1": R(R=$R); des="Resistor"  
    elec.part "C1": C(C=$C); des="Capacitor"  
  
    elec.connect in R1[1]  
    elec.connect R1[2] C1[1] out  
    elec.connect C1[2] Gnd  
}
```

Usage:

```
elec.part "F1": LowPassFilter(R=10k, C=100n)  
elec.part "F2": LowPassFilter(R=4.7k, C=1u)
```

20.7 Relationship between the Objects

- Parts contain Pins
- Pins are connected to Nets via Connections
- Nets can have Tags for grouping
- The global set `*e` contains all objects in creation order

Chapter 21

Querying

EDADSL provides a powerful querying system for selecting electrical objects based on their relationships.

21.1 Output and Input Quantifiers

The quantifier syntax selects objects connected to a net:

```
<quantifier>@<net_name>
```

21.2 Net based Querying

Quantifiers filter by object type:

```
P@<net>    #c# Parts connected to net
C@<net>    #c# Connections on net
T@<net>    #c# Pins connected to net
I@<net>    #c# Connections and Pins on net
H@<net>    #c# Holes on net
E@<net>    #c# Everything on net (or just @<net>)
```

Example:

```
$input [set]= I@Vin
```

21.3 Tagged Net based Querying

Query by tag instead of net name:

```
<quantifier>@^<tag>
```

This expands to the union of the quantifier across all nets with the given tag.

Example:

```
elec.tag Vin "sensitive"
elec.tag Vfb "sensitive"
I@^sensitive #c# All connections/pins on
              #c# nets tagged "sensitive"
```

21.4 Affiliation-based Querying

Affiliation-based querying selects objects affiliated with a specific base object, regardless of net:

`<primary>@<secondary>~<object>`

- `<primary>` – What to return (same quantifiers as net-based: C, P, T, N, I, H, E)
- `<secondary>` – Relationship type: p (part), c (connection), n (net), t (pin), or e (no type filter)
- `<object>` – The base object to query from

The result is a set of objects of type `<primary>` affiliated with `<object>` through the relationship classified by `<secondary>`.

Examples:

```
C@p~U2      #c# All connections affiliated with part U2
T@p~U2      #c# All pins of part U2
N@p~R1      #c# All nets connected to R1
P@c~N1      #c# All parts connected via net N1
E@p~U2      #c# Everything affiliated with U2
E@e~U2      #c# Same as above (no type filter)
```

21.5 General Querying

General querying provides two powerful forms for filtering and transforming sets: set builder notation and set query notation. Both support nesting.

21.5.1 Set Builder Notation

Set builder notation iterates over a set or list, filtering and optionally transforming elements.

EBNF:

```
set_builder = "{", variable, "E", ( set | list ),
              ":", filter_expr, [ "~", transform_expr ], "}"
```

```
filter_expr  = expression    #c# any expression evaluating to bool
transform_expr = function_call #c# applied to filtered elements
```

The filter function receives each element and returns `True` to keep it. The transform function is applied **per-element** to each item that passes the filter; the result set contains the transformed values. If transform is omitted, elements are returned as-is.

Examples:

```
#c# Filter: only active parts
{ $x E *p : f.u.is_active($x) ~ noop($x) }

#c# Filter and transform: get names of large caps
```

```

{ $x E *p : f.u.is_capacitor($x) &&
    f.u.value($x) > 100u ~
    f.u.name($x) }

## Nested: find all nets connected to parts
## in a set
{ $x E *p : f.u.is_mosfet($x) ~
    { $y E *n : f.u.connected($y, $x) ~
        noop($y) } }

```

21.5.2 Set Query Notation

Set query notation applies a filter and optional transform to an existing set, with optional output assignment.

EBNF:

```

set_query = "{", set_expr, "@~", "[",
    filter_expr, "]",
    [ "~[", transform_expr, "]" ],
    "~>", [ variable ], "}"

```

The @~ operator queries the set. The trailing ~> optionally writes the result to a variable; if omitted, the result is returned directly.

Examples:

```

## Query with filter only
{ *p @~[f.u.is_active] ~> }

## Query with filter and transform
{ *p @~[f.u.is_mosfet]
    ~[f.u.name] ~> }

## Store result in variable
{ *p @~[f.u.is_capacitor]
    ~[f.u.value] ~> $cap_values }

## Query connections on a net
{ C@Vin @~[f.u.is_high_current] ~> }

```

21.5.3 Nesting

Both forms can be nested to any depth. Each nested query creates a new scope for its loop variable.

Example:

```

## For each MOSFET, find all connected nets,
## then filter to high-voltage nets
{ $q E *p : f.u.is_mosfet($q) ~
    { C@~power @~[f.u.connected_to($q)]

```



```
      ~[f.u.voltage_rating] ~>  
$ratings } }
```

Part V

Physical System

Chapter 22

Physical Constraints

Physical constraints are specified using the `phys.` prefix and control the physical layout and manufacturing requirements of the PCB.

22.1 Units and SI Prefixes

Warning: All physical values use base SI units (meters, ohms, farads, etc.). A value like `min=15` means 15 **meters**, not 15 millimeters!

Always use SI prefixes for typical PCB dimensions:

- `min=15m` – 15 millimeters (15×10^{-3} m)
- `min=15u` – 15 micrometers (15×10^{-6} m)
- `d=0.25m` – 0.25 millimeters (250 μ m)

SI prefixes commonly used in PCB design: **n** (10^{-9}), **u** (10^{-6}), **m** (10^{-3}), **k** (10^3), **M** (10^6), **G** (10^9). All SI prefixes (quecto through quetta) are supported — see Section 5.1 for the full table.

22.2 Priority System

All physical constraints accept a priority parameter that determines how conflicts are resolved:

Priorities:

<code>(none)</code> or <code>-d</code>	Default
<code>-nth</code>	Nice to have
<code>-imp</code>	Important
<code>-vimp</code>	Very important
<code>-oeo</code>	Only explicit overrides
<code>-eo</code>	Explicit override
<code>-ne</code> = <code>-nonneg</code>	No exceptions

Priority Chain:

`-d` < `-nth` < `-imp` < `-vimp` < `-oeo` < `-eo` < `-ne` = `-nonneg`

22.3 Proximity Constraint

Controls the distance between sets of objects:

```
phys.prox <set_a> <set_b>  
    min=<min_distance> max=<max_distance>  
    <priority>  
phys.prox <set> minim <priority>
```

- **min** – Minimum distance between edges of set a and set b
- **max** – Maximum distance anything in set a can be from anything in set b
- **minim** – Tries to minimize distance between all objects in the set

Example:

```
phys.prox $hvsys $lvsys min=15m -ne  
phys.prox $noisy_loop minim -vimp
```

22.4 Creepage Constraint

Controls the surface distance between sets (affected by slots, vias, traces, and conductive material):

```
phys.creep <set_a> <set_b>  
    d=<min_creepage_distance> <priority>
```

Note: Unlike **phys.prox** (Euclidean distance), **phys.creep** measures surface distance. Conductive material contributes 0 distance.

Example:

```
phys.creep $hvsys $lvsys d=30m -nonneg
```

22.5 Layer Constraint

Assigns objects to specific board layers:

```
phys.layer <set> <layer_number> <priority>
```

Layer 0 is the top layer; the bottom layer is layer n (where n is the total number of layers).

Example:

```
phys.layer *p 0 -oeo
```

22.6 Board Definition

Defines the PCB stackup characteristics. Can only be invoked once; a **Runtime Error** is raised if called a second time.

```
phys.board N=<layers> T=[<thicknesses>]  
          W=[<weights>] M=[<materials>]
```

- N – Number of board layers
- T – List of thicknesses between layers in meters (length $n - 1$)
- W – List of copper weights in oz/ft² (length n)
- M – List: [dielectric, conductor, surface-plating]

Default: `phys.board N=2 T=[0.0016] W=[1, 1] M=["FR4", "copper", "HASL"]`

Example:

```
phys.board N=2 T=[0.0016] W=[2, 2]
          M=["FR4", "copper", "ENIG"]
```

22.7 Footprint Unification

Merges multiple parts into a single physical unit on the PCB. The parts share one combined footprint and reference designator, but retain independent electrical pins. In the schematic, united parts are grouped together.

```
phys.unite { <part>, <part>, ... }
```

- The argument is a set of part references.
- United parts share a single footprint on the PCB.
- Reference designators are combined (e.g., R1.R2.R3).
- Pins remain independently accessible for net connections.

Example:

```
phys.unite { R1, R2, R3 }
## R1, R2, R3 share one footprint on the PCB
## Schematic groups them together
## R1[1], R2[1], R3[1] are still individually accessible
```

22.8 Net Shrink

Minimizes the physical extent of a net:

```
phys.shrinknet <net> <priority>
```

A **Value Error** is raised if the referenced net does not exist.

Example:

```
elec.net "Vsw"
phys.shrinknet Vsw -vimp
```

22.9 Plane Creation

Creates a copper plane on a layer connected to a net:

```
phys.mkplane <layer_number> <net>
```

A `Value Error` is raised if the referenced net or layer does not exist.

Example:

```
elec.net "Gnd"  
phys.mkplane 1 Gnd
```

22.10 Text Annotation

Places text on the board as a physical object:

```
phys.mktext txt = <text>  
    font = <font_name>  
    size = <point_size>  
    material = <material>  
    layer = <layer>
```

- `txt` – Text string to display
- `font` – Font name (e.g., "Ubuntu Mono")
- `size` – Font size in points
- `material` – Material ("cu" for copper silkscreen)
- `layer` – Target layer (e.g., T for top silkscreen). Note: In `phys.mktext`, T refers to the top silkscreen layer, not the top copper layer as in `phys.layer`.

Example:

```
phys.mktext txt = "HV" font = "Ubuntu Mono"  
    size = 9 material = "cu" layer = T
```

22.11 Via Fill

Fills a pad with vias for thermal management:

```
phys.viafill <pad> v=<via_hole_diameter>  
    d=<via_spacing> [p=<pattern>] [l=<layers>]
```

- `v` – Via hole diameter (use m for mm)
- `d` – Minimum spacing between via centers
- `p` – Fill pattern (default: `grid`)
- `l` – Layer span (default: `all`). Can be `all` or a pair like 1-2

Edge Cases

- A **Value Error** is raised if `v` or `d` is ≤ 0 .
- A **Value Error** is raised if `d < v` (vias would overlap).
- A **Value Error** is raised if an invalid pattern is specified.
- A **Value Error** is raised if `l` references layers outside the board stackup.
- A **Type Error** is raised if the `pad` argument is not a valid pin reference.

22.11.1 Pattern Types

All patterns tile the pad area with vias at spacing `d`. Each via is placed at the vertices of the respective tiling.

grid Rectangular lattice. Vias at $(i \cdot d, j \cdot d)$ for integers i, j .

```
phys.viafill Q1[d] v=0.25m d=0.5m p=grid
```

hex Densest circle packing arrangement. Each via has 6 nearest neighbors at distance `d`. Approximately 15% denser than `grid`, best thermal performance due to maximized copper coverage.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=hex
```

stagger Offset rows forming the same spacing as `hex` but with alternating row offsets. Equivalent to `hex` for thermal purposes but creates straighter routing channels between rows.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=stagger
```

radial Concentric rings emanating from pad center. Ring radii at $r_n = d \cdot n$ for integer $n \geq 1$. Optimal for round thermal pads under ICs.

```
phys.viafill U1[pin5] v=0.3m d=0.8m p=radial
```

random Poisson-disk distributed vias with minimum spacing `d`. Avoids periodic structures that can cause resonance or EMI.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=random
```

22.11.2 Examples

```
#c# Standard grid fill for MOSFET drain
phys.viafill Q1[d] v=0.25m d=0.5m p=grid
```

```
#c# Dense hex fill for high-current pad
phys.viafill Q1[d] v=0.2m d=0.4m p=hex
```

```
#c# Radial fill for IC thermal pad
phys.viafill U1[pin5] v=0.3m d=0.8m p=radial
```

```
#c# Random fill to avoid EMI resonance
phys.viafill Q1[d] v=0.25m d=0.5m p=random
```

22.12 Connection Current

Sets the minimum RMS current a connection must carry:

```
phys.connectioncurrent <connection | set>
    <rms_current> <priority>
```

Example:

```
phys.connectioncurrent ( J1[2] | F1[1] ) 30 -oeo
```

22.13 Alignment

Aligns parts along an axis with specified spacing:

```
phys.align <list_of_parts>
    axis=<x|y> d=<edge_spacing> <priority>
```

Example:

```
phys.align [ Q1, Q2 ] axis=x d=5m -vimp
```

22.14 Connection Matching

Minimizes variance of a property across a set of connections:

```
phys.connmatch <property>
    <set_of_connections> <priority>
```

The set of connections is a **set** of connection objects (e.g., connections between parts and nets), or a variable holding such a set.

22.14.1 Properties

- **l** – Physical trace length
- **res** – DC resistance
- **L** – Self-inductance
- **cap** – Capacitance to reference plane
- **imp** – Characteristic impedance
- **t** – Signal propagation delay

22.14.2 Value Calculation Methods

Calculation Mode Selection

The calculation method can be selected using the `syst.cm.calc` flag:

```
syst.cm.calc <mode>
```

Available modes:

- **approx** – (Default) Uses simplified closed-form formulas for fast computation. Suitable for most designs.
- **calculus** – Uses numerical integration and differential equations for higher accuracy. Computationally more expensive.
- **hybrid** – Uses closed-form formulas for initial estimates, then refines with calculus-based methods for critical nets.

Example:

```
syst.cm.calc calculus  
phys.connmatch imp $high_speed_lines -vimp
```

Length (len)

The physical length of the trace path from source pin to destination pin, measured in millimeters. Calculated as the sum of all trace segments.

Resistance (res)

DC resistance is calculated using:

$$R = \frac{\rho \cdot l}{W \cdot T} \quad (22.1)$$

where:

- ρ – Resistivity of the conductor (copper: $1.72 \times 10^{-8} \Omega \cdot \text{m}$)
- l – Trace length (mm)
- W – Trace width (mm)
- T – Trace thickness (mm, derived from copper weight in oz/ft²)

Calculus Mode When `syst.cm.calc calculus` is active, skin effect is considered:

$$R_{ac} = R_{dc} \cdot \left(1 + \frac{1}{12} \cdot \left(\frac{\delta}{T} \right)^4 \right) \quad (22.2)$$

where $\delta = \sqrt{\frac{2\rho}{\omega\mu}}$ is the skin depth at frequency ω .

Inductance (ind)

Self-inductance is approximated using the loop inductance formula for a microstrip:

$$L \approx \frac{\mu_0 \cdot l_{trace}}{2\pi} \cdot \ln \left(\frac{2h}{w+t} \right) \quad (22.3)$$

where:

- μ_0 – Permeability of free space ($4\pi \times 10^{-7}$ H/m)
- l_{trace} – Trace length
- h – Height above reference plane
- w – Trace width
- t – Trace thickness

Calculus Mode When `syst.cm.calc calculus` is active, mutual inductance and frequency-dependent effects are computed using numerical integration:

$$L_{total} = \int_0^{l_{trace}} \frac{\mu_0}{2\pi} \cdot \ln \left(\frac{2h(x)}{w(x)+t} \right) dx \quad (22.4)$$

This accounts for trace width variations and non-uniform height above the reference plane.

Capacitance (cap)

Capacitance to the reference plane is calculated using the microstrip capacitance formula:

$$C \approx \frac{\varepsilon_0 \cdot \varepsilon_r \cdot w \cdot l_{trace}}{h} \quad (22.5)$$

where:

- ε_0 – Permittivity of free space (8.854×10^{-12} F/m)
- ε_r – Relative permittivity of the dielectric (FR4: ≈ 4.5)
- w – Trace width
- l_{trace} – Trace length
- h – Height above reference plane

Calculus Mode When `syst.cm.calc calculus` is active, fringing effects and dielectric variations are included:

$$C = \int_0^{l_{trace}} \frac{\varepsilon_0 \cdot \varepsilon_r(x) \cdot w_{eff}(x)}{h(x)} dx \quad (22.6)$$

where w_{eff} includes fringing capacitance contributions and $\varepsilon_r(x)$ accounts for material variations along the trace.

Impedance (imp)

Characteristic impedance for a microstrip is calculated as:

$$Z_0 \approx \frac{87}{\sqrt{\varepsilon_r + 1.41}} \cdot \ln \left(\frac{5.98h}{0.8w + t} \right) \quad (22.7)$$

For stripline:

$$Z_0 \approx \frac{60}{\sqrt{\varepsilon_r}} \cdot \ln \left(\frac{4b}{0.67(0.8w + t)} \right) \quad (22.8)$$

where b is the total dielectric thickness for stripline configurations.

Calculus Mode When `syst.cm.calc calculus` is active, the full telegrapher's equations are solved numerically:

$$Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \quad (22.9)$$

where R , L , G , and C are per-unit-length parameters computed from the trace geometry and material properties. This accounts for frequency-dependent losses and dispersion.

Propagation Time (t)

Signal propagation delay is calculated as:

$$t = \frac{l_{trace}}{v_p} = \frac{l_{trace} \cdot \sqrt{\varepsilon_{eff}}}{c} \quad (22.10)$$

where:

- v_p – Propagation velocity
- c – Speed of light in vacuum (3×10^8 m/s)
- ε_{eff} – Effective permittivity (depends on geometry and dielectric)

Calculus Mode When `syst.cm.calc calculus` is active, propagation delay is computed by integrating along the trace path:

$$t = \int_0^{l_{trace}} \frac{\sqrt{\varepsilon_{eff}(x)}}{c} dx \quad (22.11)$$

This accounts for varying dielectric thickness, material transitions, and impedance discontinuities along the trace.

22.14.3 Matching Objective

The solver minimizes the scaled variance of the selected property across all connections in the set:

$$\text{minimize} \quad \frac{\text{var}(P)}{||S||} \quad (22.12)$$

where P is the property value for each connection and $||S||$ is the cardinality of the connection set.

22.14.4 Example

```
phys.connmatch t $CPU_to_RAM_lines -vimp
```

This ensures all CPU-to-RAM signal lines have matched propagation delays, critical for parallel data buses.

Chapter 23

Solver Model

23.1 Overview

The EDADSL solver transforms declarative physical constraints into an optimized PCB layout. The process involves collecting constraints, selecting an optimization algorithm, and generating the final layout through placement and routing phases.

23.2 Algorithm Selection

The solver algorithm can be selected using the `syst.alg` flag:

```
syst.alg <algorithm_name>
```

23.2.1 Available Algorithms

Simulated Annealing (sa)

A probabilistic optimization technique inspired by the annealing process in metallurgy. The algorithm explores the solution space by accepting worse solutions with a probability that decreases over time.

Objective Function Let $\mathbf{x} \in \mathcal{X}$ represent the complete layout state (component positions, trace routes, via placements). The objective function $f : \mathcal{X} \rightarrow \mathbb{R}$ quantifies the total constraint violation cost:

$$f(\mathbf{x}) = \sum_{i=1}^n w_i \cdot v_i(\mathbf{x}) \quad (23.1)$$

where w_i is the priority weight of constraint i and $v_i(\mathbf{x})$ is the violation measure (0 if satisfied, positive otherwise).

Neighbourhood Structure A neighbour state $\mathbf{x}'$ is generated by applying a random perturbation δ to the current state:

$$\mathbf{x}' = \mathbf{x} + \delta, \quad \delta \sim \mathcal{U}(-\epsilon, \epsilon) \quad (23.2)$$

where ϵ is the maximum perturbation magnitude, adaptively scaled based on acceptance history.

Acceptance Probability The Metropolis acceptance criterion determines whether to move to a neighbour state:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } f(\mathbf{x}') \leq f(\mathbf{x}) \\ \exp\left(-\frac{f(\mathbf{x}') - f(\mathbf{x})}{T}\right) & \text{if } f(\mathbf{x}') > f(\mathbf{x}) \end{cases} \quad (23.3)$$

where $T > 0$ is the current temperature parameter.

Cooling Schedule The temperature decreases according to a geometric cooling schedule:

$$T_{k+1} = \alpha \cdot T_k, \quad \alpha \in (0, 1) \quad (23.4)$$

with typical values $\alpha \in [0.95, 0.999]$. The initial temperature T_0 is set such that the initial acceptance probability for a worst-case perturbation is approximately 0.8:

$$T_0 = -\frac{\Delta f_{\max}}{\ln(0.8)} \quad (23.5)$$

Convergence Criterion The algorithm terminates when one of the following conditions is met:

- Temperature threshold: $T_k < T_{\min}$ (typically 10^{-6})
- No improvement: $f(\mathbf{x}_{\text{best}})$ has not improved for N_{stale} iterations
- Maximum iterations: $k > k_{\max}$

Algorithm

1. Initialize $\mathbf{x}_0$ randomly, set T_0 , $k = 0$
2. Generate neighbour $\mathbf{x}'$ from $\mathbf{x}_k$
3. Compute $\Delta f = f(\mathbf{x}') - f(\mathbf{x}_k)$
4. If $\Delta f \leq 0$ or $\mathcal{U}(0, 1) < \exp(-\Delta f/T_k)$, set $\mathbf{x}_{k+1} = \mathbf{x}'$
5. Otherwise, set $\mathbf{x}_{k+1} = \mathbf{x}_k$
6. Update $T_{k+1} = \alpha \cdot T_k$
7. If convergence not met, $k \leftarrow k + 1$, goto 2

Properties

- Time complexity: $O(k_{\max} \cdot |\mathcal{N}(\mathbf{x})|)$ where $|\mathcal{N}(\mathbf{x})|$ is the neighbourhood evaluation cost
- Probability of finding global optimum approaches 1 as $k_{\max} \rightarrow \infty$ with sufficiently slow cooling
- Space complexity: $O(|\mathbf{x}|)$ for storing the current and best states

Genetic Algorithm (ga)

An evolutionary optimization method that maintains a population of candidate solutions and evolves them through biologically-inspired operators: selection, crossover, and mutation.

Population Representation The population at generation t is a set of N chromosomes:

$$P(t) = \{\mathbf{x}_1^{(t)}, \mathbf{x}_2^{(t)}, \dots, \mathbf{x}_N^{(t)}\} \quad (23.6)$$

Each chromosome $\mathbf{x}_i^{(t)} \in \mathcal{X}$ encodes a complete layout state as a gene vector:

$$\mathbf{x}_i^{(t)} = (g_1, g_2, \dots, g_m) \quad (23.7)$$

where each gene g_j represents a design variable (position, route choice, etc.).

Fitness Function The fitness of chromosome $\mathbf{x}_i$ is derived from the objective function:

$$\text{fit}(\mathbf{x}_i) = \frac{1}{1 + f(\mathbf{x}_i)} \quad (23.8)$$

where $f(\mathbf{x}_i)$ is the constraint violation cost. Higher fitness indicates better solutions.

Selection Parent selection uses tournament selection of size τ :

1. Sample τ individuals uniformly from $P(t)$
2. Select the individual with highest fitness as parent

This is repeated to select N parents for mating.

Crossover Single-point crossover with probability p_c :

$$\text{crossover}(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} (g_1^{(i)}, \dots, g_c^{(i)}, g_{c+1}^{(j)}, \dots, g_m^{(j)}) & \text{if } \mathcal{U}(0, 1) < p_c \\ \mathbf{x}_i & \text{otherwise} \end{cases} \quad (23.9)$$

where $c \sim \mathcal{U}\{1, m-1\}$ is the crossover point.

Mutation Per-gene mutation with probability p_m :

$$g'_j = \begin{cases} g_j + \mathcal{N}(0, \sigma^2) & \text{if } \mathcal{U}(0, 1) < p_m \\ g_j & \text{otherwise} \end{cases} \quad (23.10)$$

where σ is the mutation strength, often adapted over generations.

Elitism The best $\lfloor e \cdot N \rfloor$ individuals are copied unchanged to the next generation, where $e \in [0, 0.2]$ is the elitism rate.

Convergence Criterion The algorithm terminates when:

- Maximum generations: $t > t_{\max}$
- Fitness stagnation: Best fitness has not improved for t_{stale} generations
- Target fitness: $\text{fit}(\mathbf{x}_{\text{best}}) > 1 - \epsilon_{\text{fit}}$

Algorithm

1. Initialize population $P(0)$ randomly
2. Evaluate fitness for all $\mathbf{x}_i^{(t)} \in P(t)$
3. Apply elitism: copy top $e \cdot N$ individuals to $P(t+1)$
4. Select parents via tournament selection
5. Apply crossover with probability p_c
6. Apply mutation with probability p_m
7. Replace population: $P(t) \leftarrow P(t+1)$
8. If convergence not met, $t \leftarrow t+1$, goto 2

Properties

- Time complexity: $O(t_{\max} \cdot N \cdot m)$ for N individuals of length m
- Maintains population diversity, reducing local optima entrapment
- Space complexity: $O(N \cdot |\mathbf{x}|)$ for storing the population
- Naturally parallelizable across individuals

Constraint Programming (cp)

A declarative optimization approach that models the problem as a set of variables, domains, and constraints, using propagation and backtracking to find solutions.

Problem Formulation A constraint satisfaction problem (CSP) is defined as a tuple:

$$\mathcal{P} = (\mathcal{X}, \mathcal{D}, \mathcal{C}) \tag{23.11}$$

where:

- $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ is the set of decision variables
- $\mathcal{D} = \{D(x_1), D(x_2), \dots, D(x_n)\}$ is the set of variable domains
- $\mathcal{C} = \{c_1, c_2, \dots, c_m\}$ is the set of constraints

Domains Each variable x_i has an initial domain $D(x_i) \subseteq \mathbb{R}$ (or discrete set). For PCB layout:

- Position variables: $D(x_i) = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$
- Route variables: $D(x_i) = \{\text{path}_1, \text{path}_2, \dots\}$

Constraint Types Constraints are classified as:

- **Unary:** $c(x_i)$ — restricts a single variable
- **Binary:** $c(x_i, x_j)$ — relates two variables
- **Global:** $c(x_{i_1}, \dots, x_{i_k})$ — involves multiple variables (e.g., `alldiff`)

Arc Consistency (AC-3) For each binary constraint $c(x_i, x_j)$, enforce arc consistency:

$$\forall v_i \in D(x_i), \exists v_j \in D(x_j) \text{ such that } c(v_i, v_j) = \text{true} \quad (23.12)$$

If no such v_j exists, remove v_i from $D(x_i)$. Repeat until no more domains can be reduced.

Propagation When a domain is reduced, propagate effects to all connected constraints:

1. Maintain a worklist W of affected variables
2. For each $x_i \in W$, re-evaluate all constraints involving x_i
3. If any domain $D(x_j)$ is reduced, add x_j to W
4. Continue until W is empty (fixpoint reached) or a domain becomes empty (failure)

Backtracking Search The solver explores the search tree depth-first:

1. Select unassigned variable x_i (branching heuristic: smallest domain first)
2. For each value $v \in D(x_i)$ (best-first ordering):
 - (a) Assign $x_i \leftarrow v$
 - (b) Propagate constraints
 - (c) If propagation succeeds (no empty domains), recurse
 - (d) If propagation fails, backtrack

Optimization Extension For optimization (minimizing $f(\mathbf{x})$), add a bound constraint:

$$f(\mathbf{x}) < f(\mathbf{x}_{\text{best}}) \quad (23.13)$$

This is tightened each time a better solution is found (branch-and-bound).

Convergence Criterion

- Solution found: All variables assigned, all constraints satisfied
- No solution: Search tree exhausted (problem infeasible)
- Optimality: All nodes pruned, current best is optimal

Properties

- Time complexity: Exponential in worst case, but propagation prunes large portions of the search tree
- Guarantees constraint satisfaction (complete when solution exists)
- Space complexity: $O(n \cdot |D_{\max}| + m)$ for domain storage and constraint graph
- Deterministic: Same problem instance yields same solution

Gradient Descent (gd)

A first-order iterative optimization algorithm that finds local minima by moving in the direction of steepest descent of the objective function.

Objective Function Assume the objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, where n is the number of design variables (component positions, trace parameters, etc.).

Gradient Computation The gradient of f at point $\mathbf{x}_k$ is:

$$\nabla f(\mathbf{x}_k) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^\top \quad (23.14)$$

Computed via automatic differentiation or finite differences:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x}_k + h\mathbf{e}_i) - f(\mathbf{x}_k - h\mathbf{e}_i)}{2h} \quad (23.15)$$

where h is a small perturbation and $\mathbf{e}_i$ is the i -th unit vector.

Update Rule The basic gradient descent update is:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \nabla f(\mathbf{x}_k) \quad (23.16)$$

where $\eta_k > 0$ is the learning rate at iteration k .

Learning Rate Schedules

- **Constant:** $\eta_k = \eta$
- **Decay:** $\eta_k = \eta_0 / (1 + \beta k)$
- **Exponential:** $\eta_k = \eta_0 \cdot \gamma^k$
- **Adaptive:** $\eta_{k,i} = \eta_0 / \sqrt{s_i + \epsilon}$ (per-parameter, AdaGrad-style)

where η_0 is the initial learning rate, β, γ are decay parameters, s_i accumulates past squared gradients, and ϵ prevents division by zero.

Momentum Enhancement To accelerate convergence and dampen oscillations:

$$\mathbf{v}_{k+1} = \mu \mathbf{v}_k + \eta_k \nabla f(\mathbf{x}_k) \quad (23.17)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{v}_{k+1} \quad (23.18)$$

where $\mu \in [0, 1]$ is the momentum coefficient (typical: 0.9).

Constraint Handling For constraints $g_j(\mathbf{x}) \leq 0$, use a penalty method:

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + \lambda \sum_{j=1}^m \max(0, g_j(\mathbf{x}))^2 \quad (23.19)$$

where λ is a penalty weight, increased iteratively:

$$\lambda_{k+1} = \min(\lambda_k \cdot \rho, \lambda_{\max}), \quad \rho > 1 \quad (23.20)$$

Convergence Criterion

- Gradient norm: $\|\nabla f(\mathbf{x}_k)\| < \epsilon_{\text{grad}}$
- Parameter change: $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| < \epsilon_{\text{step}}$
- Function change: $|f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k)| < \epsilon_f$
- Maximum iterations: $k > k_{\max}$

Algorithm

1. Initialize $\mathbf{x}_0, \eta_0, \lambda_0, k = 0$
2. Compute $\nabla f(\mathbf{x}_k)$ and $\nabla g_j(\mathbf{x}_k)$
3. Update penalty: $\lambda_{k+1} = \min(\lambda_k \cdot \rho, \lambda_{\max})$
4. Compute search direction: $\mathbf{d}_k = -\nabla \tilde{f}(\mathbf{x}_k)$
5. Line search: find η_k satisfying Armijo condition
6. Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \eta_k \mathbf{d}_k$
7. If convergence not met, $k \leftarrow k + 1$, goto 2

Properties

- Time complexity: $O(k_{\max} \cdot n)$ per iteration for gradient computation
- Converges to local minima; requires convexity or good initialization for global optimality
- Space complexity: $O(n)$ for storing the current point and gradient
- Fast per-iteration, but may require many iterations for convergence

Hybrid (hybrid)

A multi-strategy approach that combines the strengths of different algorithms in a sequential pipeline, adapting the optimization strategy to the problem characteristics at each phase.

Architecture The hybrid solver operates in three sequential phases:

$$\mathbf{x}_{\text{final}} = \text{Phase}_3(\text{Phase}_2(\text{Phase}_1(\mathbf{x}_0))) \quad (23.21)$$

Phase 1: Global Exploration (Simulated Annealing)

- Algorithm: Simulated annealing with reduced iterations ($k_{\text{max}}^{(1)} = k_{\text{max}}/4$)
- Purpose: Explore the solution space broadly, identify promising regions
- Output: $\mathbf{x}_1$ with $f(\mathbf{x}_1) < f_{\text{threshold}}$

The temperature schedule uses faster cooling ($\alpha = 0.95$) to quickly identify basins of attraction.

Phase 2: Local Refinement (Gradient Descent)

- Algorithm: Gradient descent with momentum
- Purpose: Refine the solution from Phase 1 to a local minimum
- Input: $\mathbf{x}_1$ from Phase 1
- Output: $\mathbf{x}_2$ with $\|\nabla f(\mathbf{x}_2)\| < \epsilon_{\text{grad}}$

The penalty weight is initialized from Phase 1's final value and continues to increase.

Phase 3: Constraint Satisfaction (Constraint Programming)

- Algorithm: Constraint propagation and backtracking
- Purpose: Verify and enforce all hard constraints
- Input: $\mathbf{x}_2$ from Phase 2
- Output: $\mathbf{x}_{\text{final}}$ feasible or infeasible report

If $\mathbf{x}_2$ violates hard constraints, CP attempts repair. If repair fails, the solver reports the conflict set.

Adaptive Phase Selection Based on problem characteristics, the hybrid solver may skip phases:

$$\text{skip}(\text{Phase}_i) = \begin{cases} \text{true} & \text{if } n < n_{\text{simple}} \text{ and } i = 1 \\ \text{true} & \text{if all constraints linear and } i = 1 \\ \text{false} & \text{otherwise} \end{cases} \quad (23.22)$$

Phase Transition Criteria Transition from Phase i to Phase $i + 1$ occurs when:

- Improvement rate falls below threshold: $\Delta f / \Delta k < \epsilon_{\text{rate}}$
- Maximum phase iterations reached: $k^{(i)} > k_{\text{max}}^{(i)}$
- Solution quality threshold met: $f(\mathbf{x}) < f_{\text{threshold}}^{(i)}$

Algorithm

1. Initialize $\mathbf{x}_0$, determine phase parameters from problem size n
2. **Phase 1:** Run SA with $k_{\text{max}}^{(1)}$, obtain $\mathbf{x}_1$
3. **Phase 2:** Run GD from $\mathbf{x}_1$ until convergence, obtain $\mathbf{x}_2$
4. **Phase 3:** Run CP from $\mathbf{x}_2$, verify/repair constraints
5. Return $\mathbf{x}_{\text{final}}$ or infeasibility report

Properties

- Time complexity: Sum of phase complexities, typically $O(k_{\text{max}}^{(1)} \cdot |\mathcal{N}| + k_{\text{max}}^{(2)} \cdot n + \text{CP}_{\text{search}})$
- Combines global exploration with local exploitation and constraint guarantees
- Adapts to problem complexity via phase skipping
- Space complexity: Maximum of individual phase requirements

23.2.2 Default Algorithm

If no algorithm is specified, the solver uses simulated annealing (**sa**) as the default.

23.3 Constraint Resolution

The solver resolves physical constraints according to the following rules:

23.3.1 No Conflicts

If all constraints can be satisfied without conflicts, the solution is complete.

23.3.2 Conflicts Between Different Priorities

When constraints with different priorities conflict:

- The higher-priority constraint is satisfied
- The lower-priority constraint is optimized as much as possible without impairing the higher rule
- A warning is printed to the console

23.3.3 Conflicts Between Same Priorities

When constraints with the same priority conflict, behavior depends on the priority level:

Lower Priorities (-d, -nth, -imp, -vimp)

- An error message is printed to the console
- The program continues execution

Higher Priorities (-oeo, -eo, -ne, -nonneg)

- An error message is printed to the console
- The program does NOT continue execution (halts)

23.4 Layout Generation Pipeline

The final layout is generated through a multi-phase pipeline:

23.4.1 Phase 1: Constraint Collection

All physical constraints are collected from the program and organized by type and priority.

23.4.2 Phase 2: Board Setup

The board stackup is defined (layers, thicknesses, materials) and the design area is initialized.

23.4.3 Phase 3: Component Placement

Parts are placed on the board according to:

1. Alignment constraints (`phys.align`)
2. Proximity constraints (`phys.prox`)
3. Layer constraints (`phys.layer`)
4. Creepage constraints (`phys.creep`)

The solver optimizes placement to satisfy all constraints simultaneously, respecting the priority hierarchy.

23.4.4 Phase 4: Trace Routing

Connections are routed as traces on the board:

1. Current capacity constraints (`phys.connectioncurrent`) determine minimum trace width
2. Connection matching constraints (`phys.connmatch`) ensure length/impedance matching
3. Net shrinkage (`phys.shrinknet`) minimizes trace length

23.4.5 Phase 5: Via Placement

Vias are placed to connect traces across layers:

1. Via fill constraints (`phys.viafill`) populate thermal vias
2. System flags (`syst.flag.vias`, etc.) control via availability

23.4.6 Phase 6: Plane Generation

Copper planes are generated on specified layers:

1. Plane constraints (`phys.mkplane`) create ground and power planes
2. Clearances are maintained around non-connected objects

23.4.7 Phase 7: Design Rule Check

The final layout is verified against all constraints:

1. All physical constraints are checked
2. Violations are reported with severity based on priority
3. Critical violations (`-ne`, `-nonneg`) halt generation

23.4.8 Phase 8: File Export

The verified layout is exported to KiCad format:

1. `make.schematic` generates the schematic file (`.kicad_sch`)
2. `make.pcb` generates the PCB file (`.kicad_pcb`)
3. `halt` triggers final export and log file generation

23.5 Ordering of Global Set Constant `*e`

Inside the `*e` pseudo-constant (and when converting any set to a list via `f.list()`), element order is determined by lexicographic sorting of each element's **canonical string representation**.

23.5.1 Canonical String Rules

Each data type produces a deterministic canonical string:

- **int** – decimal string (e.g., `42` → `"42"`)
- **real** – decimal string (e.g., `3.14` → `"3.14"`)
- **bool** – literal (e.g., `True` → `"True"`)
- **string** – quoted (e.g., `"a"` → `'"a"'`)

- **list** – bracket-enclosed, preserving element order (e.g., $[3, 1, 2] \rightarrow "[3, 1, 2]"$)
- **set** – brace-enclosed, elements sorted first (e.g., $\{Q1, R5, Q2\} \rightarrow "\{Q1, Q2, R5\}"$)
- **part** – reference designator (e.g., $R1 \rightarrow "R1"$)
- **pin** – parent ref and pad id (e.g., $U1[3] \rightarrow "U1[3]"$)
- **connection** – sorted pin pair in parentheses (e.g., $(R1[1], U1[+]) \rightarrow "(R1[1], U1[+])"$)
- **net** – name (e.g., $Gnd \rightarrow "Gnd"$)

Elements are then sorted lexicographically by their canonical strings.

23.5.2 Example

```
elec.part "U1": opamp(type="LM741"); des="";
    fp="Package_DIP:DIP-8_W7.62mm"
elec.net "V-"
elec.connect V- U1[v-]
```

```
#c# Canonical strings:
#c# (U1[V-],U1[v-]) -- connection
#c# U1                -- part
#c# U1[+]             -- pin
#c# U1[-]             -- pin
#c# U1[out]           -- pin
#c# U1[v+]            -- pin
#c# U1[v-]            -- pin
#c# V-                -- net
#c#
#c# *e = { (U1[V-],U1[v-]), U1, U1[+], U1[-],
#c#   U1[out], U1[v+], U1[v-], V- }
```


Chapter 24

Flags

System flags control PCB manufacturing capabilities using the `syst.flag.` prefix:

`syst.flag.<flag_name> <value>`

24.1 Available Flags

- `exec_mode` – Execution mode: `lenient` (default) or `strict`. In strict mode, all warnings become errors and halt execution.
- `phy_mode` – Physical constraint enforcement mode (default: `phylenient`):
 - `phystRICT` – All physical constraints must be met exactly. Violations halt execution immediately.
 - `phylenient` – Physical constraints are enforced best-effort. Violations produce warnings but execution continues.
 - `phyrelaxed` – The solver may relax or ignore physical constraints to find any valid layout.
- `vias` – Controls whether vias are allowed
- `slots` – Controls whether slots / PCB cuts are allowed
- `jumpers` – Controls whether jumper wires are allowed
- `solder_reinforced_tracks` – Controls whether solder-reinforced tracks are allowed
- `buried_vias` – Controls whether buried vias are allowed
- `approx_eps` – Additive tolerance for approximate equality operator `~` (default: 1)
- `approx_eps_mult` – Multiplicative tolerance for approximate equality operator `~*` (default: 1)
- `max_loop_iter` – Maximum iterations per while/do-while loop before raising a `Value Error` (default: 100000)
- `show_pcb_part_ref` – Show reference designator on PCB (default: `true`)
- `show_pcb_part_value` – Show component value on PCB (default: `true`)

- `show_pcb_part_tol` – Show tolerance/rating on PCB (default: `true`)
- `show_pcb_part_mpn` – Show MPN/manufacturer on PCB (default: `true`)
- `debug_mode` – Enable debug functions (`dbg.*`). When `false`, all debug functions are no-ops (default: `false`)
- `debug_output` – Default output destination for `dbg.print`: `"console"`, `"log"`, or `"both"` (default: `"console"`)

24.2 PCB Part Text Visibility

Global flags control which part text fields are visible on the PCB. Each field can be toggled independently:

```
syst.flag.show_pcb_part_ref False
syst.flag.show_pcb_part_value True
```

Per-component overrides can be set in the `elec.part` property list (see Section 20.2.1). If a per-component flag is set, it overrides the global flag for that component:

```
elec.part "R3": R(R=220); des="220R"; fp="R_Axial_DIN0309";
    show_ref=True; show_value=False
```

Available per-component flags: `show_ref`, `show_value`, `show_tol`, `show_mpn`.

Part VI

Toolchain

Chapter 25

Compiler and Execution Pipeline

The EDADSL toolchain compiles source code into PCB and schematic files.

25.1 Compilation Phases

1. Lexical Analysis – Tokenization of source code
2. Parsing – Syntax analysis and AST construction
3. Semantic Analysis – Type checking and validation
4. Execution – Program execution with electrical system construction
5. Constraint Solving – Physical layout optimization
6. Code Generation – Output file generation

25.2 Entry Point

Every program must begin with the **start** keyword. The program halts when **halt** is reached or an unrecoverable error occurs.

Chapter 26

Code Generation

26.1 PCB Generation

The `make.pcb` command generates a PCB file based on the circuit and physical constraints:

```
make.pcb
```

This command must be placed after all circuit definitions and physical constraints.

26.2 Schematic Generation

The `make.schematic` command generates a schematic file:

```
make.schematic
```

26.3 Export

The `halt` command terminates execution and exports all created files:

```
halt
```

Chapter 27

Standard Library

The standard library provides built-in component models, footprint libraries, and material definitions.

27.1 Component Models

Built-in models include: **R** (Resistor), **C** (Capacitor), **L** (Inductor), **opamp** (Operational Amplifier), **MOSFET** (MOSFET transistor), **LED** (Light Emitting Diode), **D** (Diode), **Q** (BJT Transistor), **X** (Crystal/Oscillator), **J** (Connector), **F** (Fuse), **Y** (Transformer).

27.2 Footprint Libraries

Footprints are organized by package type: **Package_SMD**, **Package_TH**, **Package_BGA**, **Package_Power**, **Package_Connector**. Each library contains standard footprints following IPC naming conventions.

27.3 Material Library

Materials define physical properties for PCB stackup: **FR4** (fiberglass-epoxy), **copper** (conductor), **ENIG** (surface finish), **soldermask** (insulator).

Chapter 28

External Library

External libraries extend EDADSL with additional component models and footprints.

28.1 Importing Libraries

TODO: Document how external libraries are integrated with EDADSL.

28.2 Supported Formats

EDADSL supports integration with:

- KiCad symbol libraries (`.kicad_sym`)
- KiCad footprint libraries (`.kicad_mod`)
- SPICE models (`.lib`, `.sub`)
- IPC-2581 component databases

Chapter 29

Debugging and Logging

29.1 Debug Mode

Debug functions (`dbg.*`) are enabled by the `debug_mode` flag. When disabled (default), all debug functions are no-ops:

```
syst.flag.debug_mode True
```

29.2 Console Output

The `echo` and `f.print()` functions output to the console:

```
echo $variable  
f.print("Message: " + $variable)
```

29.3 Log File

The `writetolog` keyword appends a message to the log file:

```
writetolog "Checkpoint reached"
```

The log file is exported when `halt` is executed.

29.4 Debug Print

The `dbg.print` function outputs variable value and type. The destination is controlled by the `mode` parameter or the global `debug_output` flag:

```
dbg.print($variable)  
dbg.print($variable, mode="log")  
dbg.print($variable, mode="both")
```

Modes:

- "console" (default) – Output to console
- "log" – Output to debug log file
- "both" – Output to console and debug log file

29.5 Breakpoints

The `dbg.breakpoint` function pauses execution and prints the current state (variables, stack). An optional condition can be provided:

```
dbg.breakpoint()
dbg.breakpoint($voltage > 5.0)
dbg.breakpoint($count == 10)
```

If a condition is provided, execution only pauses when the condition is `True`. When `debug_mode` is `false`, breakpoints are no-ops and execution continues uninterrupted.

29.6 Graph Inspection

The `dbg.graph` function dumps the full electrical graph state (parts, nets, connections, pins):

```
dbg.graph()
```

Output includes all parts with their models and parameters, all nets with their tags, all connections, and all pin assignments.

29.7 Resource Monitoring

The `dbg.resources` function prints current resource usage:

```
dbg.resources()
```

Output includes:

- Memory usage
- CPU time elapsed
- Object counts (parts, nets, connections, pins)

Part VII

Reference

Chapter 30

Exceptions

30.1 Type Errors

Raised when an operation is applied to a value of the wrong type.

30.2 Value Errors

Raised when a function receives an argument outside its domain (e.g., division by zero, invalid list index). Out-of-bounds list or set indexing (e.g., accessing index 10 on a 3-element list) raises a Value Error and halts execution.

30.3 Reference Errors

Raised when referencing an undefined variable, part, or function.

30.4 Constraint Errors

Raised when physical constraints cannot be resolved (see Section 23).

Chapter 31

Error and Warning Model

31.1 Errors

Errors halt program execution. The error message includes the error type and location.

31.2 Warnings

Warnings are printed for constraint conflicts between different priorities. The program continues execution after warnings.

Chapter 32

Future Improvements

Planned features for future versions:

- Differential pair routing constraints
- Impedance-controlled routing
- 3D component placement visualization
- Signal integrity analysis integration
- Power integrity analysis
- Automated thermal relief generation

Chapter 33

Implementations

33.1 Reference Implementation

The reference implementation is written in Rust for performance and safety. Current version: 0.1.0 (pre-release). Supported platforms: Linux (x86_64), macOS (x86_64, aarch64), Windows (x86_64).

33.2 Build and Installation

The reference implementation can be built from source using Cargo:

```
cargo build --release
```

Chapter 34

References

34.1 Language Specification

This document serves as the authoritative specification for the EDADSL language.

34.2 Related Standards

- IPC-2581 — Generic Standard for Printed Board Assembly Products
- IPC-7351 — Land Pattern Naming Conventions for Surface Mount Designs
- KiCad File Formats — Schematic and PCB file format specifications
- SPICE — Simulation Program with Integrated Circuit Emphasis

Appendix A

Standard Libraries

A.1 Material Libraries

A.1.1 PCB Substrate Material Library

Common substrate materials:

- FR4 – Standard glass-reinforced epoxy laminate
- Rogers – High-frequency laminates
- Metal-core – Aluminum or copper core substrates

A.1.2 PCB Conductor Material Library

- Copper – Standard conductor (specified in oz/ft²)

A.1.3 PCB Surface-finish Material Library

- HASL – Hot Air Solder Leveling
- ENIG – Electroless Nickel Immersion Gold
- OSP – Organic Solderability Preservative

A.2 Basic Model Libraries

A.2.1 Passive Component Model Library

- R – Resistor (parameters: R, P, type)
- C – Capacitor
- L – Inductor

A.2.2 Active Component Model Library

- MOSFET – MOSFET transistor
- Diode – Standard diode
- BJT – Bipolar junction transistor

A.2.3 Integrated Circuit Component Model Library

- opamp – Operational amplifier
- Additional models in external libraries

A.2.4 Board Hardware Component Model Library

- Connectors
- Mounting holes

A.3 Additional Language Libraries

EDADSL can be extended with custom libraries containing component models, footprints, and constraint templates. Libraries follow the standard library organization.

A.4 Footprint Libraries

Footprints follow KiCad library naming conventions:

`<Library>:<Footprint>`

Examples:

```
Resistor_THT:R_Axial_DIN0309_L9.0mm_D3.2mm_P12.70mm_Horizontal  
Package_T0_SOT_THT:T0-247-3_Horizontal_TabUp  
Package_DIP:DIP-8_W7.62mm
```

Appendix B

Examples

B.1 LED Circuit

B.1.1 Intent

A simple LED current-limiting circuit.

B.1.2 EDADSL Code

```
start

elec.part "R1": R(R=330, P=0.25);
    des="330 ohm resistor";
    fp="Resistor_THT:R_Axial_DIN0309"
elec.part "D1": LED(color="red");
    des="Red LED";
    fp="LED_THT:LED_D3.0mm"

elec.net "Vcc"
elec.net "Gnd"

elec.connect Vcc R1[1]
elec.connect R1[2] D1[a]
elec.connect D1[c] Gnd

halt
```

B.1.3 Explanation

The circuit connects a 330Ω resistor in series with an LED between Vcc and Gnd. The resistor limits current to approximately 10mA (assuming 5V supply and 2V LED forward voltage).

B.2 Arithmetic and Type Coercion

B.2.1 Intent

Demonstrate basic arithmetic, variable declarations, and implicit type promotion from int to real to complex.

B.2.2 EDADSL Code

```
start

$a [int]= 10
$b [real]= 3.14
$c [complex]= 2 + 3i

$sum t= $a + $b          #c# 13.14 (int + real -> real)
$prod t= $a * $c         #c# 20 + 30i (int * complex -> complex)
$hyp t= f.sqrt($a^2 + $b^2) #c# ~10.49 (real)
$ang t= f.angle($c)      #c# ~0.9828 rad (real)

f.print(f.str("a + b = ", $sum))
f.print(f.str("a * c = ", $prod))
f.print(f.str("hypotenuse = ", $hyp))
f.print(f.str("angle = ", $ang))

halt
```

B.2.3 Explanation

Variables are declared with explicit types. Arithmetic operations implicitly promote to the wider type: int + real yields real, int + complex yields complex. The `f.angle` function returns the argument (phase) of a complex number in radians.

B.3 Conditionals and Classification

B.3.1 Intent

Use nested if/else to classify a resistor value into standard E-series ranges.

B.3.2 EDADSL Code

```
start

$value [real]= 4.7e3

$series [string]= ""
if $value < 1e3 ::{
    $series = "E12 or below"
```

```

} else ::{
  if $value < 100e3 ::{
    $series = "E24 common"
  } else ::{
    $series = "High value"
  }
}

$tolerance [string]= ""
if $value == 1.0e3 ::{
  $tolerance = "tight"
} else ::{
  if $value == 4.7e3 ::{
    $tolerance = "standard"
  } else ::{
    $tolerance = "check datasheet"
  }
}

f.print(f.str($value, " ohms: ", $series, ", ", $tolerance))

halt

```

B.3.3 Explanation

The if/otherwise/else chain classifies the resistor into ranges. Use `otherwise` for else-if conditions. There is no `match/case` statement; equality checks use nested if with `==`. The expected output is 4700.0 ohms: E24 common, standard.

B.4 Lambdas and Higher-Order Functions

B.4.1 Intent

Use lambda expressions with `f.map`, `f.filter`, and `f.reduce` to process a list of resistor values.

B.4.2 EDADSL Code

```

start

$resistors [list]= [100, 220, 330,
  470, 1e3, 2.2e3, 4.7e3, 10e3]

# Compute power dissipation at 5V
$powers [list]= f.map($resistors,
  L::($r [real]) -> { (5.0^2) / $r })
f.print(f.str("Powers: ", $powers))

```

```

# Filter to only resistors under 1k
$small [list]= f.filter($resistors,
  L::($r [real]) -> { $r < 1000 })
f.print(f.str("Under 1k: ", $small))

# Sum all power dissipation
$total_power [real]= f.reduce($powers,
  L::($acc [real], $p [real]) -> { $acc + $p },
  0.0)
f.print(f.str("Total power: ", $total_power, " W"))

# Sort resistors ascending
$sorted [list]= f.sort($resistors,
  L::($a [real], $b [real]) -> { $a - $b })
f.print(f.str("Sorted: ", $sorted))

halt

```

B.4.3 Explanation

Lambdas are declared with `L::` prefix, typed parameters, and `-> { }` body. `f.map` applies a function to each element, `f.filter` selects elements satisfying a predicate, and `f.reduce` folds the list with an accumulator. The sort comparator returns negative if the first argument should come first.

B.5 Complex Arithmetic and FFT

B.5.1 Intent

Perform complex arithmetic, generate a signal, and compute its frequency spectrum using FFT.

B.5.2 EDADSL Code

```

start

# Complex arithmetic
$z1 [complex]= 3 + 4i
$z2 [complex]= 1 - 2i

$sum      t= $z1 + $z2      # 4 + 2i
$product  t= $z1 * $z2      # 11 - 2i
$mag      t= f.abs($z1)     # 5.0
$conj_z   t= f.conj($z2)    # 1 + 2i

f.print(f.str("|z1| = ", $mag))

```

```

f.print(f.str("z1 * z2 = ", $product))
f.print(f.str("conj(z2) = ", $conj_z))

# Generate a 256-point signal: 50Hz + 120Hz
$N [int]= 256
$fs [real]= 1000.0
$t [list]= [0:1:$N - 1]
$signal [list]= f.map($t,
  L::($n [int]) -> { f.sin(2 * c.pi * 50
    * $n / $fs)
    + 0.5 * f.sin(2 * c.pi * 120
    * $n / $fs) })

# Compute FFT and extract magnitudes
$spectrum [list]= f.fft($signal)
$mags [list]= f.map($spectrum,
  L::($z [complex]) -> { f.abs($z) })

# Find the dominant frequency
$max_mag [real]= f.max($mags)
$peak_index [int]= f.argmax($mags)
$freq [real]= ($peak_index * $fs) / $N

f.print(f.str("Dominant frequency: ", $freq, " Hz"))
f.print(f.str("Magnitude: ", $max_mag))

halt

```

B.5.3 Explanation

Complex numbers support all arithmetic operators. The `f.fft` function returns a list of complex values; `f.map` with `f.abs` extracts magnitudes. The dominant frequency is recovered by finding the index of the peak magnitude and scaling by f_s/N . The expected dominant frequency is 50 Hz.

B.6 Narray Matrix Operations

B.6.1 Intent

Use `ndarray` syntax and matrix operators to solve a system of linear equations.

B.6.2 EDADSL Code

```

start

# Solve A * x = b where:
#   A = [[2, 1], [5, 7]]

```

```

#   b = [11, 13]
$A [list]= [[2, 1], [5, 7]]
$b [list]= [11, 13]

# Compute inverse and solve
$A_inv [list]= f.inv($A)
$x [list]= $A_inv .* $b

f.print(f.str("Solution x = ", $x))

# Verify: A .* x should equal b
$check [list]= $A .* $x
f.print(f.str("Verification A .* x = ", $check))

# Matrix properties
f.print(f.str("det(A) = ", f.det($A)))
f.print(f.str("rank(A) = ", f.rank($A)))

halt

```

B.6.3 Explanation

Ndarrays are detected automatically from rectangular all-numeric lists; no type annotation is needed. The `.*` operator performs matrix multiplication, and `f.inv` computes the matrix inverse. Functions `f.det` and `f.rank` return the determinant and rank respectively. The expected solution is $x = [6.2, -1.4]$.

B.7 Closures and Lambdas

B.7.1 Intent

Demonstrate lambdas capturing outer variables and lambda composition for filtering and mapping.

B.7.2 EDADSL Code

```

start

$resistors [list]= [100, 330, 1e3, 2.2e3, 4.7e3, 10e3]

# Filter: keep resistors above 1k (captures $threshold)
$threshold [real]= 1000
$high [list]= f.filter($resistors,
  L::($v [real]) -> { $v > $threshold })
f.print(f.str("Above 1k: ", $high))

# Filter: keep resistors above 4.7k

```

```

$threshold = 4700
$vhhigh [list]= f.filter($resistors,
    L::($v [real]) -> { $v > $threshold })
f.print(f.str("Above 4.7k: ", $vhhigh))

# Element-wise add scalar to each list element
$offset [int]= 10
$shifted [list]= f.map([10, 20, 30],
    L::($i [int]) -> { $i + $offset })
f.print(f.str("Shifted: ", $shifted))

halt

```

B.7.3 Explanation

Lambdas capture variables from the enclosing scope. Here `$threshold` is captured by reference: reassigning it before calling `f.filter` changes which values pass the filter. The `f.map` lambda captures `$offset` to add a constant to each element.

B.8 Design-Rule-Driven Component Placement

B.8.1 Intent

Declare components with parametric constraints and let the solver handle placement.

B.8.2 EDADSL Code

```

start

elec.part "U1": opamp(type="generic", GBP=1e6, slew=5)
    ; des="Generic op-amp"
    ; fp="Package_DIP:DIP-8_W7.62mm"

elec.part "Rf": R(R=10e3, P=0.125, tol=1)
    ; des="Feedback resistor"
    ; fp="Resistor_THT:R_Axial_DIN0309"

elec.part "Rin": R(R=1e3, P=0.125, tol=1)
    ; des="Input resistor"
    ; fp="Resistor_THT:R_Axial_DIN0309"

elec.part "C1": C(C=100e-12, V=50)
    ; des="Compensation capacitor"
    ; fp="Capacitor_THT:C_Axial_DIN0207"

elec.net "Input"
elec.net "Output"

```



```

elec.net "Vcc"
elec.net "Gnd"

elec.connect Input Rin[1]
elec.connect Rin[2] U1[2]      # inverting input
elec.connect Rf[1] U1[2]
elec.connect Rf[2] U1[1]      # output
elec.connect C1[1] U1[2]
elec.connect C1[2] U1[1]
elec.connect Output U1[1]
elec.connect U1[4] Vcc
elec.connect U1[7] Gnd

# Constraint: keep feedback path short
phys.prox {Rf} {U1} max=15m -imp

halt

```

B.8.3 Explanation

Components are declared with parametric models and footprint assignments using `; separators`. The `phys.prox` keyword specifies a physical layout constraint that the placement solver must satisfy. This example inverts the op-amp with gain $G = -R_f/R_{in} = -10$ and includes compensation.

B.9 Signal Processing Pipeline

B.9.1 Intent

Build a complete signal processing pipeline: generate, filter, transform, and analyze.

B.9.2 EDADSL Code

```

start

# Step 1: Generate a noisy 100Hz signal
$N [int]= 512
$fs [real]= 1000.0
$t [list]= [0:1:$N - 1]
$clean [list]= f.map($t,
    L::($N [int]) -> { f.sin(2 * c.pi
        * 100 * $n / $fs) })

# Step 2: Add noise (simple LCG pseudo-random)
$noise [list]= f.map($t,
    L::($N [int]) -> { 0.3 * f.sin(2 * c.pi
        * 7 * $n) })

```

```

$noisy [list]= f.map($t,
  L::($i [int]) -> { $clean[$i] + $noise[$i] })

# Step 3: Compute FFT of noisy signal
$spectrum [list]= f.fft($noisy)
$mags [list]= f.map($spectrum,
  L::($z [complex]) -> { f.abs($z) })

# Step 4: Find peak frequency
$peak_idx [int]= f.argmax($mags)
$peak_freq [real]= ($peak_idx * $fs) / $N
f.print(f.str("Detected frequency: ", $peak_freq, " Hz"))

# Step 5: Compute signal statistics
$sig_rms [real]= f.rms($noisy)
$sig_peak [real]= f.peak($noisy)
f.print(f.str("RMS: ", $sig_rms, ", Peak: ", $sig_peak))

# Step 6: Build magnitude spectrum pairs
$bin_width [real]= $fs / $N
$spec_pairs [list]= []
for $i E [0:1:$N / 2 - 1] ::{
  $freq_val [real]= $i * $bin_width
  $mag_val [real]= $mags[$i]
  $spec_pairs = $spec_pairs
    + [[ $freq_val, $mag_val]]
}

f.print(f.str("Spectrum bins (first 5): ",
  $spec_pairs[0:5]))

halt

```

B.9.3 Explanation

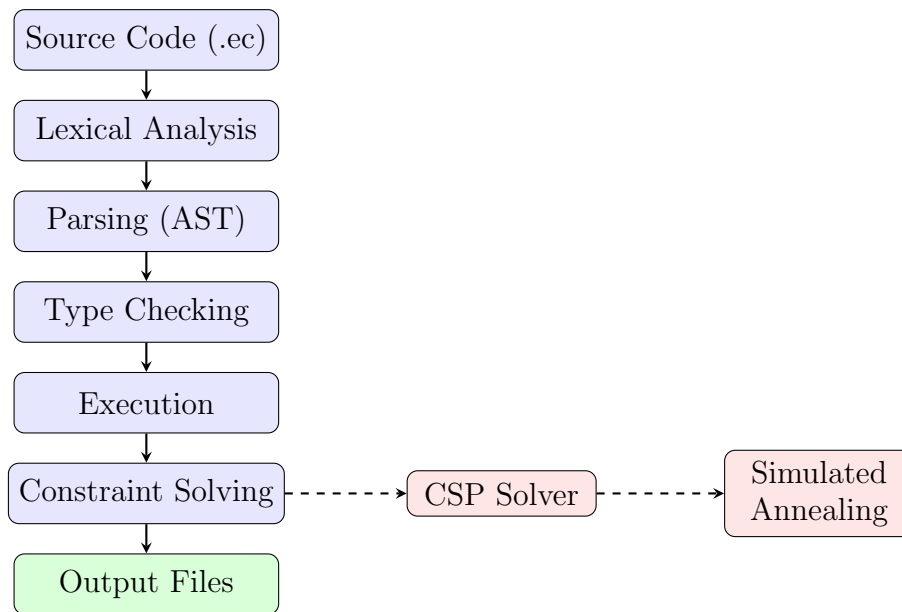
This example chains together multiple language features: `f.map` for list comprehensions, complex FFT, higher-order functions for magnitude extraction, signal statistics (`f.rms`, `f.peak`), and list slicing. The pipeline demonstrates a realistic signal analysis workflow from raw data through frequency-domain representation.

Appendix C

Diagrams

C.1 Execution-phase Diagram

The following diagram shows the execution phases of an EDADSL program, from source code through constraint solving to final output.



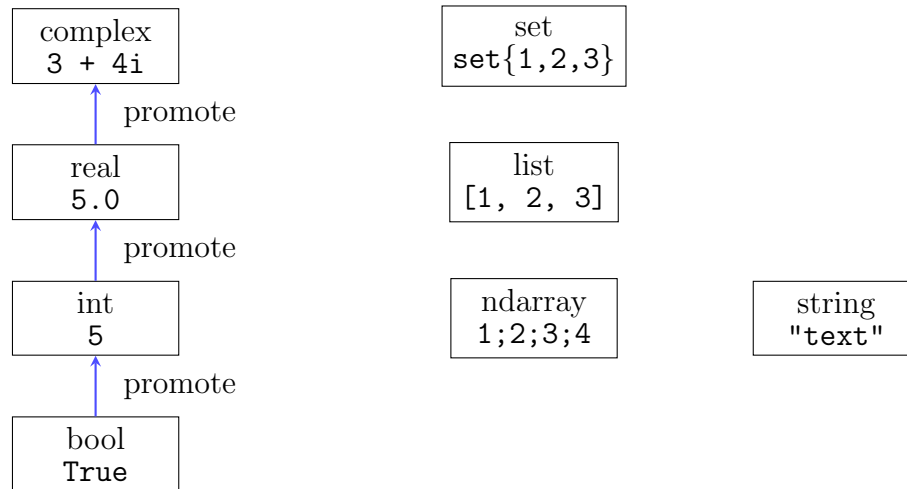
Phase Descriptions:

- **Lexical Analysis** – Tokenizes the source file into keywords, identifiers, literals, and operators.
- **Parsing** – Builds an Abstract Syntax Tree (AST) from the token stream.
- **Type Checking** – Validates types, performs implicit promotions, and resolves overloads.
- **Execution** – Evaluates expressions, executes control flow, and evaluates functions. Components and nets are registered.

- **Constraint Solving** – Physical constraints are solved using CSP propagation and simulated annealing to determine optimal placement and routing.
- **Output Files** – Generates manufacturing files (Gerbers, drill files, BOM, netlists).

C.2 Type Hierarchy Diagram

The following diagram shows the EDADSL type hierarchy, including numeric promotion, collection types, and their relationships.

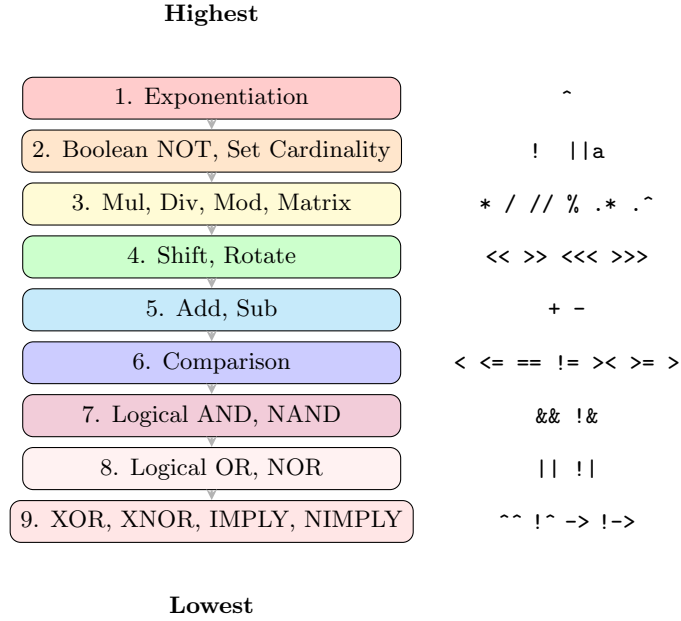


Key Relationships:

- Numeric types promote automatically: `bool` $\rightarrow$ `int` $\rightarrow$ `real` $\rightarrow$ `complex`
- An `ndarray` is a `list` that is rectangular and all-numeric; detected automatically
- `list` element types are polymorphic; `ndarray` elements are restricted to `int` or `real`
- `set` is unordered and unique; `list` is ordered and allows duplicates

C.3 Operator Precedence Diagram

The following diagram visualizes operator precedence from highest (top) to lowest (bottom).



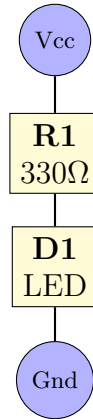
Precedence Table:

1. Exponentiation (^) – right-associative
2. Boolean negation (!), Set cardinality (||a)
3. Multiplication (*), Division (/), Floor Division (//), Modulo (%), Matrix (.* .^)
4. Shift (<< >>), Rotate (<<<[w] >>>[w])
5. Addition (+), Subtraction (-)
6. Comparison (< <= == != > >= >)
7. Logical AND (&&), NAND (!&)
8. Logical OR (||), NOR (!|)
9. XOR (^^), XNOR (!^), IMPLY (->), NIMPLY (!->)

C.4 Example Circuit: LED Current-Limiting

The following diagram shows the LED circuit from Appendix B alongside its EDADSL code.

Schematic



EDADSL Code:

```
elec.part "R1": R(R=330)
    des="330 ohm"
    fp="Resistor_THT:R_Axial"
elec.part "D1": LED(color="red")
    des="Red LED"
    fp="LED_THT:LED_D3.0mm"

elec.net "Vcc"
elec.net "Gnd"

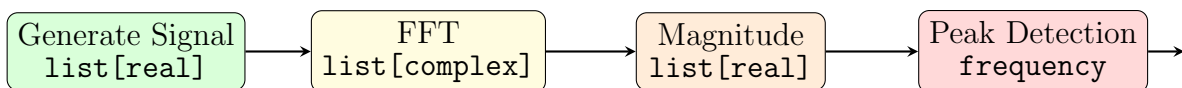
elec.connect Vcc R1[1]
elec.connect R1[2] D1[a]
elec.connect D1[c] Gnd
```

Circuit Description: A 330Ω resistor (R1) limits current through a red LED (D1) between Vcc (5V) and Gnd. Expected current: $I = (5V - 2V)/330\Omega \approx 9.1\text{mA}$.

C.5 FFT Pipeline Diagram

The following diagram shows the signal processing pipeline for FFT analysis.

FFT Pipeline



EDADSL Code:

```

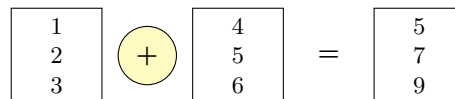
$signal [list]= f.map([0:1:$N - 1],
    L::($n [int]) -> { f.sin(2 * c.pi
        * 100 * $n / $fs) })
$spectrum [list]= f.fft($signal)
    # list[complex]
$mags      [list]= f.map($spectrum,
    L::($z [complex]) -> { f.abs($z) })
    # list[real]
$peak_idx [int]= f.argmax($mags)
$freq     [real]= ($peak_idx * $fs) / $N    # = 100 Hz

```

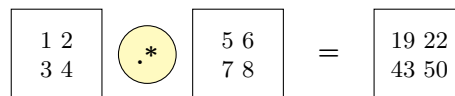
C.6 Narray Operations

The following diagram visualizes key ndarray operations: element-wise operations, matrix multiplication, and broadcasting.

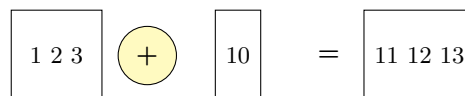
Element-wise +



Matrix Multiply



Broadcasting



EDADSL Code:

```

[1, 2, 3] + [4, 5, 6]    #c# = [5, 7, 9]
[[1,2],[3,4]] .* [[5,6],
    [7,8]]              #c# = [[19,22],[43,50]]
[1, 2, 3] + 10          #c# = [11, 12, 13]

```

Operations:

- **Element-wise** – Standard operators (+ - * / ^) applied element-by-element
- **Matrix Multiply** – .* computes matrix product (rows × columns)
- **Broadcasting** – Scalars are broadcast to match the ndarray shape
- **Transpose** – .T flips rows and columns; .T in-place

C.7 Custom Model Definition and Usage

C.7.1 Intent

Define a custom SiC MOSFET model with a kelvin source pin, then use it in a half-bridge circuit.

C.7.2 EDADSL Code

```
# Define custom SiC MOSFET model
elec.makemodel mosfet/C3M0016120K1:{
pins:
    g      -> 4; T = input
    s      -> 2; T = power
    d      -> 1; T = power
    kelvin_s -> 3; T = output

des = "1200 V, 125A, 16 mOhm, T0-247-4"
dts = "https://assets.wolfspeed.com/..."
fp  = "Package_T0_SOT_THT/T0-247-4_Vert"
}

# Use custom model in a half-bridge
elec.part "Q1": mosfet/C3M0016120K1();
    des="High-side SiC MOSFET";
    fp="Package_T0_SOT_THT/T0-247-4_Vert";
    show_ref=True; show_value=True

elec.part "Q2": mosfet/C3M0016120K1();
    des="Low-side SiC MOSFET";
    fp="Package_T0_SOT_THT/T0-247-4_Vert";
    show_ref=True; show_value=True

# Define nets
elec.net "Vbus" "hv"
elec.net "Vphase"
elec.net "Vgate_high"
elec.net "Vgate_low"
elec.net "Gnd"

# Gate drive
elec.connect Vgate_high Q1[g]
elec.connect Vgate_low  Q2[g]

# Half-bridge output
elec.connect Vphase Q1[d] Q2[d]
```



```

# Power rails
elec.connect Vbus Q1[s]
elec.connect Gnd  Q2[s]

# Kelvin connections for gate drive return
elec.connect Vgate_low Q1[kelvin_s]
elec.connect Gnd       Q2[kelvin_s]

# Unite into single physical package
phys.unite { Q1, Q2 }

```

C.7.3 Explanation

The `elec.makemodel` keyword defines a new component model with named pins, pin types, and optional metadata. Pin types (`input`, `power`, `output`) classify the electrical role of each pin. The `des`, `dts`, and `fp` fields attach documentation and footprint information to the model.

When used in `elec.part`, the custom model is referenced by its full name (`mosfet/C3M0016120K1`). Per-component PCB text flags (`show_ref`, `show_value`) override the global defaults. The `phys.unite` statement merges Q1 and Q2 into a single physical package on the PCB.